

多核编程：进程、线程与同步

巴黎高科计算机科学硕士

巴黎萨克雷高性能计算硕士

作者：Gaël Thomas

进程的定义

- 进程是程序运行的一个实例。
 - 允许并行执行不同的程序（例如：Fortnite 与 Chrome）。
 - 允许同一个程序多次执行（例如：两个用户分别运行各自的 Emacs 实例）。
 - 操作系统负责：
 - 管理进程的生命周期（启动与停止）；
 - 提供进程间通信机制（信号、管道、套接字等）；
 - 定期在处理器上调度进程；
 - 隔离各个进程（默认无共享内存）。
- > 总的来说，进程大致相当于一台完整机器的虚拟化。

从调用栈帧到线程

在程序执行期间，每当函数开始执行时，进程会创建一个“调用帧（call frame）”，其中包含：

- 函数的参数；
- 局部变量；
- 指向调用者的链接。

函数执行完毕后，调用帧会被释放。

```c

```
int add(int x, int y) {
 int z = x + y;
```

```

return z;
}

int main(int argc, char** argv) {
printf("%d\n", add(1, 2));
}
'''

```

栈由多个调用帧组成，例如 main 调用 add。

“下一条要执行的指令”决定了线程当前的执行位置。

---

### ### 线程的定义

- 线程是 CPU 可执行的执行上下文。
- 一个线程包含：
  - 调用帧的栈（例如 main 调用 add）；
  - 下一条待执行的指令。

操作系统负责在各 CPU 上调度线程。

---

### ### 进程与线程

- 一个进程包含：
  - 内存空间（数据段、代码段、堆等）；
  - 一个或多个线程（每个线程拥有独立的栈与下一条指令）。
- 一个进程启动时总是只有一个主线程。
- 为提高并行性，进程可以创建更多线程。操作系统可将它们调度到多个 CPU 上并行执行。
- 当进程中最后一个线程结束时，整个进程终止。

---

### ### 线程标识

- 类型：`pthread\_t`
- 获取当前线程 ID：`pthread\_self()`

---

### ### 创建线程

```
``c
int pthread_create(pthread_t* tid, pthread_attr_t* attr,
void* (*start_routine)(void*), void* arg);
``
```

说明：

- 创建并启动一个新线程。
- 新线程从 `start\_routine` 函数开始执行。
- `start\_routine` 接收参数 `arg`。
- `pthread\_create` 会将新线程 ID 填入 `\*tid`。
- `pthread\_attr\_t` 用于设定线程属性（调度策略、栈指针等）。

示例：

```
``c
void* f(void* arg) { printf("f 正在运行\n"); return NULL; }

int main() {
pthread_t tid;
pthread_create(&tid, NULL, f, NULL);
printf("main 与 f 并行执行\n");
}
``
```

---

### ### 线程终止

线程的终止方式：

- 显式调用 `pthread\_exit(void\* retval)` ；
- `start\_routine` 函数执行完毕 ；
- 当主函数返回或某个线程调用 `exit` 时，系统会终止该进程的所有线程。

---

### 等待线程结束

```
```c
int pthread_join(pthread_t thread, void** retval);
```
```

示例：

```
```c
void* f(void* arg) {
    printf("f 正在运行\n");
    return (void*)0x42;
}

int main() {
    pthread_t tid;
    void* retval;
    pthread_create(&tid, NULL, f, NULL);
    printf("main 与 f 并行执行\n");
    pthread_join(tid, &retval);
    printf("f 返回值为 %p\n", retval);
}
```
```

---

### 分离（Detached）模式

- 默认情况下，线程为可连接（joinable）模式：

- 线程结束后系统保留其返回值（占用系统资源）。
- 其他线程可通过 `pthread\_join` 获取返回值。
- 分离模式（detached）：
- 线程退出时系统立即释放相关资源；
- 无法再获取返回值。

切换方式：

- 调用 `pthread\_detach(pthread\_t tid)`；
- 或在创建时通过 `pthread\_attr\_t` 指定。

---

### ### 共享变量与不一致性

同一进程的多个线程共享相同的内存空间。

- 当一个线程修改变量时，其他线程也能看到变化；
- 并发访问可能导致数据不一致。

示例：

```
```c
int balance = 1000;

void credit() {
    int tmp = balance;
    tmp = tmp + 100;
    balance = tmp;
}

void debit() {
    int tmp = balance;
    tmp = tmp - 1;
    balance = tmp;
}
```
```

如果线程 1 和线程 2 并行执行，可能丢失一次加 100 的更新。

---

### ### 避免不一致的原则

要防止同时执行访问同一共享变量的代码段。

- 我们称这种互斥执行的指令序列为\*\*互斥区（mutual exclusion）\*\*。

定义：\*\*临界区（critical section）\*\*是以互斥方式执行的代码段。

- 临界区之间顺序执行；

- 临界区中的代码原子地执行。

---

### ### 互斥的实现

\*\*互斥锁（mutex）\*\*：一种用于互斥的锁。

- 状态：忙（busy）或空闲（free）。

- 同一时刻仅有一个线程可持有该锁。

两个主要操作：

- `lock` 获取锁（若忙则等待）；

- `unlock` 释放锁。

这两个操作看似原子执行。

```c

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
```

```
int balance = 1000;
```

```
void credit() {
```

```
    pthread_mutex_lock(&m);
```

```
    int tmp = balance; tmp += 100; balance = tmp;
```

```
    pthread_mutex_unlock(&m);
```

```
}
```

```

void debit() {
pthread_mutex_lock(&m);
int tmp = balance; tmp -= 1; balance = tmp;
pthread_mutex_unlock(&m);
}
'''

```

监视器（Monitor）

允许线程等待某个条件为真。通过互斥锁和条件变量实现。

```

```c
char* msg = NULL;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void send() {
pthread_mutex_lock(&m);
msg = "Hello!";
pthread_cond_signal(&c);
pthread_mutex_unlock(&m);
}

void recv() {
pthread_mutex_lock(&m);
while (msg == NULL)
pthread_cond_wait(&c, &m);
printf("Message: %s\n", msg);
pthread_mutex_unlock(&m);
}
'''

```

接口说明：

- `pthread\_cond\_wait(&cond, &mutex);`：释放互斥锁、在条件上等待、被唤醒后重新获取锁。
- `pthread\_cond\_signal(&cond);`：唤醒一个等待线程。
- `pthread\_cond\_broadcast(&cond);`：唤醒所有等待线程。

---

### 小结

**\*\*线程生命周期：\*\***

- `pthread\_create`：创建线程；
- `pthread\_self`：获取线程 ID；
- `pthread\_exit`：退出线程；
- `pthread\_join`：等待线程结束。

**\*\*同步机制：\*\***

- `pthread\_mutex\_lock` / `pthread\_mutex\_unlock`：加锁与解锁；
- `pthread\_cond\_wait`：等待条件变量；
- `pthread\_cond\_signal` / `pthread\_cond\_broadcast`：唤醒等待线程。

# 多核编程中的锁实现 (Lock Implementations)

## 原子操作 (非正式定义)

- 在高层次上，**原子操作**是看起来“瞬间执行”的操作。
- `pthread_mutex_lock` 就是一个典型的原子操作，它执行的逻辑大致如下：

```
1 enum { FREE, BUSY };
2 typedef struct { int state; } pthread_mutex_t;
3 #define PTHREAD_MUTEX_INITIALIZER = { FREE };
4
5 void pthread_mutex_lock(pthread_mutex_t* mutex) {
6 while(mutex->state == FREE) { }
7 mutex->state = BUSY;
8 }
```

另一条线程在此处无法获取锁，因为 `pthread_mutex_lock` 是以原子方式执行的。

## 如何构建原子操作？

我们无法用 `pthread_mutex_lock` 来实现 `pthread_mutex_lock`，因为这就是我们要实现的函数本身！

## 目录

1. 面包房算法 (Bakery Algorithm)
  - (a) 第一个机器模型
  - (b) 算法本身
2. 背景知识
3. 各类锁算法

## 如何构建原子操作？(一)

在一切之前，我们需要一个机器模型。

此阶段我们考虑一个简单的机器模型：

- 能够以原子方式读写一个机器字 (32 位或 64 位)

示例：

```
1 int x = 0;
2 Thread 1: x = 0x101;
3 Thread 2: tmp = x;
```

这里 `tmp` 的值只能是 0 或 0x101 (不会是 0x100 或 0x001)。

## 如何构建原子操作？(二)

仍然是假设：

- 机器能原子地读写一个机器字

但请注意：一般情况下这个假设**并不成立**！（在 Pentium 上是成立的，但并非所有现有或未来处理器都成立）

## 如何构建原子操作？(三)

继续扩展模型：

我们假设一个抽象的简单机器模型：

- 原子地读写一个机器字
- 不对指令进行重排序

示例：

```
1 int x = 0;
2 int y = 0;
3 Thread 1:
4 x = 0x101;
5 y = 0x202;
6 Thread 2:
7 t2 = y;
8 t1 = x;
```

在这种情况下，如果  $t2 = 0x202$ ，那么必定  $t1 = 0x101$ 。

## 如何构建原子操作？(四)

仍然是上述模型，但要注意：

在一般情况下，这个假设**仍不一定成立**！（对于两次写操作在 Pentium 上是对的，但对于一次写后读，可能不成立）

示例：

```
1 int x = 0;
2 int y = 0;
3 Thread 1:
4 x = 0x101;
5 y = 0x202;
6 Thread 2:
7 t2 = y;
8 t1 = x;
```

## 目录（再次）

### 1. 面包房算法（Bakery Algorithm）

#### (a) 机器模型

(b) 算法

2. 背景

3. 锁算法

## 面包房算法 (Lamport 1974)

基本原理:

- 模拟一家“面包店”: 顾客(线程)排队取号等待服务。
- 每个顾客有一个号码。
- 号码更小的顾客先被服务。

问题: 如何选取号码?

- 思路: 向其他顾客询问他们的号码, 选择最大值 + 1。
- 新问题: 如果两个顾客同时取号, 可能获得相同号码。
- 解决: 假设大家礼貌地按照“年龄(线程 ID)”顺序, ID 小的先服务。

## 面包房算法 (Lamport 1974) 实现

我们假设采用前述的简单机器模型。

```
1 int entering[N]; /* 初始化为 false */
2 int num[N]; /* 初始化为 0 */
3
4 void lock(int self) { /* 线程编号 self 调用 lock */
5 entering[self] = true;
6 num[self] = 1 + max(num[0], ..., num[N-1]);
7 entering[self] = false;
8
9 restart:
10 for(int i=0; i<N; i++) {
11 if(entering[i]) { } /* 等待直到线程 i 选好了号码 */
12 if(num[i] && num[i] < num[self]) goto restart;
13 if(num[i] && num[i] == num[self] && i < self) goto restart;
14 }
15 }
16
17 void unlock(int self) { num[self] = 0; } /* 退出临界区 */
```

## 为什么需要 entering 变量?

假设我们没有 entering 变量:

- 进程 2 和进程 3 同时进入。
- 进程 2 计算出 max+1, 但在写入 num 前被抢占。

- 进程 3 执行并计算出相同的 `max+1`。
- 进程 3 进入临界区，但在释放锁之前被抢占。
- 进程 2 被调度并进入临界区（因为  $2 < 3$ ）。
- 因此会出现两个进程同时在临界区的情况！

加入 `entering` 可以阻止线程 3 进入临界区，因为线程 3 会看到 `entering[2] = true`。

## 面包房算法的性质

- 这个算法非常优雅。
- 它只依赖于**写操作的原子性**（以及指令不重排序）。
- 但算法较慢：
  - 每个线程至少需要读取 4 个内存位置：
    1. 计算 `max`；
    2. 两次访问 `entering`；
    3. 检查自己是否拥有最小号码。
  - 因此算法的读取复杂度是  $O(N)$ ，其中  $N$  是线程数。

## 我们需要硬件的帮助

为了优化锁的实现，我们需要特殊的硬件指令。但在此之前，我们要先理解处理器的行为：

`[label=()]`

1. 缓存一致性协议（Cache Protocol）
2. 加载与存储的原子性
3. 内存顺序（Memory Ordering）

## 目录

1. 面包房算法
2. 背景知识
  - (a) 缓存协议
  - (b) 加载与存储的原子性
  - (c) 内存顺序
3. 各类锁算法

## 缓存协议 (Cache Protocol)

- 处理器的缓存保存了主存中某些位置的副本。
- 对于大多数 Pentium 处理器，一条缓存行大小为 64 字节。
- 缓存实现了一种读写锁机制：
  - 读时：缓存行处于“共享状态”（多个读者）。
  - 写时：缓存行处于“独占状态”（仅一个写者）。

示例：

```
1 int tab[2];
2 Thread 1: tab[0] = 42;
3 Thread 2: tab[1] = 666;
```

缓存行 1: [42, 0] 缓存行 2: [0, 666]

若多个线程同时写同一缓存行，几乎不可能保证所有写都被保留。

## 缓存协议的实现

在执行存储（store）操作时：

- 缓存协议会将缓存行置为“独占状态”，以保证一致性。
- 如果没有这种排他机制，处理器可能丢失部分写入。

```
1 int tab[2];
2 Thread 1: tab[0] = 42;
3 Thread 2: tab[1] = 666;
```

如果多个写操作同时发生，几乎不可能保留全部写入结果。

## 缓存协议的内部机制

- 当执行 **load** 时：
  - 若该行不在缓存中，则从其他核心或主存中加载。
  - 确保其他核心不持有该行的独占副本。
  - 将该行标记为共享状态。
- 当执行 **store** 时：
  - 若该行不在缓存中，则从其他核心或主存加载；
  - 使其他核心的副本失效；
  - 将该行标记为独占状态。
- 若该行在缓存中但处于共享状态：
  - 同样会使其他核心的副本失效；
  - 并标记为独占状态。

## 缓存协议的影响

- 若多个线程在不同核心上写同一缓存行，会导致总线饱和。
- 对锁算法的启示：
  - 应尽量避免多个线程写同一个内存位置。

## 目录（再次）

1. 面包房算法
2. 背景知识
  - (a) 缓存协议
  - (b) 加载与存储原子性
  - (c) 内存顺序
3. 锁算法

## 加载与存储的原子性（Load and Store Atomicity）

- 对一个机器字的加载（load）或存储（store）并不总是原子的：
  - 在 Pentium 处理器上是原子的；
  - 但未来的处理器上不一定如此。
- 为了在 C 中确保加载与存储的原子性：
  - `atomic_load(&var)`：保证加载原子性；
  - `atomic_store(&var, value)`：保证存储原子性；
  - 此时变量需声明为 `_Atomic`，例如：

```
1 int _Atomic var;
```

- 注意：这些操作也会影响指令重排序。

## 内存顺序（Memory Ordering）

- 处理器可能会对指令进行乱序执行（只要能保证线程自己读到自己的最后一次写）。
- 处理器可能重排序以下操作：
  - 两个不同地址的存储（store → store）；
  - 两个不同地址的加载（load → load）；
  - 不同地址的“写后读”（store → load）；
  - 不同地址的“读后写”（load → store）。

## 不同语言与处理器的内存模型

### Pentium: TSO (Total Store Order)

- 保证 64 位读写原子性；
- 不重排序：
  - 加载在加载之后；
  - 存储在存储之后；
  - 存储在加载之后；
- 但可能重排序“加载在存储之后” (load-after-store)。

示例：在 Pentium 上正确的代码

```
1 int hasMessage = false;
2 char* message = NULL;
3
4 Thread 1:
5 message = "hello";
6 hasMessage = true;
7
8 Thread 2:
9 while(!hasMessage) { }
10 printf(message);
```

### Java 内存模型

- 允许任意重排序（除非涉及 `volatile` 或锁操作）。

### ARM 内存模型

- 比 TSO 更弱。
- 在 Java 或 ARM 平台上，可能出现 `printf` 段错误。

示例：

```
1 int hasMessage = false;
2 char* message = NULL;
3
4 Thread 1:
5 message = "hello";
6 hasMessage = true;
7
8 Thread 2:
9 while(!hasMessage) { }
10 printf(message);
```

## 使用汇编指令防止重排序

某些处理器提供特殊的“内存栅栏 (memory fence)”指令：

- `mfence`：全栅栏，防止任何加载或存储重排；
- `lfence`：仅防止加载重排（典型用途：`rdtsc`）；
- `sfence`：仅防止存储重排（典型用途：`clwb`）。

## 在 C 中防止重排序

开发者可以通过显式指定内存顺序语义：

- `value = atomic_load_explicit(&addr, order);`
- `atomic_store_explicit(&addr, value, order);`

其中 `order` 可取：

- `memory_order_relaxed`
- `memory_order_consume`
- `memory_order_acquire`
- `memory_order_release`
- `memory_order_acq_rel`
- `memory_order_seq_cst`（默认值）

## Relaxed 语义

- 任意重排序皆可能；
- 实际上与普通非原子操作相同。

示例：

```
1 int _Atomic x = 0;
2 int _Atomic y = 0;
3
4 Thread 1:
5 atomic_store_explicit(&x, 0x101, memory_order_relaxed); // A
6 atomic_store_explicit(&y, 0x101, memory_order_relaxed); // B
7
8 Thread 2:
9 r2 = atomic_load_explicit(&y, memory_order_relaxed); // C
10 r1 = atomic_load_explicit(&x, memory_order_relaxed); // D
```

可能的结果：`r1 = 0` 且 `r2 = 0x101`。

说明：

- 若 B 先于 A 执行 调度顺序 B C D A；
- 若 D 先于 C 执行 调度顺序 D A B C。

重要：此时 `r0`, `r1` 只能取 0 或 0x101；若非原子操作，还可能取 0x100 或 0x001。

## Release-Acquire 语义

- 规则：
  - 在 `release` 存储之前的所有可见效果，在对应的 `acquire` 加载前可见；
  - 在 `acquire` 加载之后的效果，不会“穿透”到之前。

示例：

```
1 int _Atomic x = 0; int y = 0; int z = 0;
2
3 Thread 1:
4 z = 1; // A
5 y = 17; // B
6 atomic_store_explicit(&x, 42, memory_order_release); // C
7
8 Thread 2:
9 r1 = atomic_load_explicit(&x, memory_order_acquire); // D
10 r2 = y; // E
11 r3 = z; // F
```

若 `r1 == 42`，则必有 `y == 17` 且 `z == 1`。

## Release-Consume 语义

- 类似于 Release-Acquire，但仅适用于存在“依赖关系”的变量；
- 实现复杂且常被编译器误处理；
- 因此目前**不推荐使用**。

示例：

```
1 int _Atomic x = 0; int y = 0; int z = 0;
2
3 Thread 1:
4 x = 42; // A
5 y = 42; // B
6 atomic_store_explicit(&z, x, memory_order_release); // C
7
8 Thread 2:
9 r1 = atomic_load_explicit(&z, memory_order_consume); // D
10 r2 = y; // E
11 r3 = x; // F
```

如果 `r1 == 42`，则 `r3 == 42`（因为 `z` 与 `x` 之间有依赖），但 `r2` 可能仍为 0。

## 顺序一致性 (Sequential Consistency) 语义

- 等价于 Release-Acquire；
- 但还要求所有线程看到的原子操作顺序一致。

示例：

```

1 int _Atomic x = 0; int _Atomic y = 0;
2
3 Thread 1: atomic_store(&x, 1); // A
4 Thread 2: atomic_store(&y, 1); // B
5 Thread 3: r1 = atomic_load(&x); r2 = atomic_load(&y);
6 Thread 4: r3 = atomic_load(&x); r4 = atomic_load(&y);

```

若  $r1 = 1$  且  $r2 = 0$ ，则对于线程 3，A 在 B 之前执行。由于顺序一致性，线程 4 也必须看到相同顺序；因此  $r3 = 0$  且  $r4 = 1$  不可能。

但若仅使用 Release-Acquire，这种情况可能出现。

## 备注

在后续讲义中，为简化代码：

- 若写  $x = 0x101$ ，表示 `atomic_store(&x, 0x101);`
- 默认假设语义为“顺序一致性”（即使弱语义也能保证正确）。

在实验中你需要：

- 显式使用 `atomic_load/atomic_store`；
- 尝试识别是否可使用更弱的语义（如 Acquire-Release 或 Relaxed）。

## 现在我们准备好了……

我们现在已经能够用 C 实现面包房算法。

为了实现更高效的锁算法，我们仍需要一些原子性的“加载-修改-存储”（load-modify-store）操作，它们能在单个原子步骤中：

- 读取一个值；
- 修改该值；
- 写回该值。

常见的三种操作是：

- `atomic_exchange`
- `atomic_fetch_add`
- `atomic_compare_exchange_strong`

若需显式指定内存顺序，可使用后缀 `_explicit`。

## 目录

1. 面包房算法
2. 背景知识
3. 各类锁算法

- (a) 自旋锁 (Spinlock)
- (b) 票据锁 (Ticket Lock)
- (c) MCS 锁

## 工具: `atomic_exchange`

`atomic_exchange` 原子地交换一个变量的值:

```
1 type atomic_exchange(type _Atomic* addr, type value) {
2 type res = *addr;
3 *addr = value;
4 return res;
5 }
```

上述过程以原子方式执行。

## 加载-修改-存储的实现

在 Pentium 上, 其机制依赖于缓存协议:

- 获取缓存行的独占访问权;
- 在执行指令期间“锁定”该缓存行;
- 其他核心必须等待, 直到锁释放。

**重要:**

- 这种原子性不需要全局同步;
- 它只是暂时锁住 L1 缓存行;
- 因此原子操作本身性能代价极小;
- 真正的成本来自:
  1. 写操作导致其他副本失效;
  2. 内存排序限制导致乱序执行受阻。

## 自旋锁 (Spinlock)

自旋锁是最简单的锁实现:

- 原理: 当锁处于 BUSY 状态时自旋等待;
- 仅依赖 `atomic_exchange` 操作。

```
1 enum { FREE, BUSY };
2 int _Atomic lock = FREE;
3
4 void lock_func(int _Atomic* lock) {
5 while(atomic_exchange(lock, BUSY) != FREE) { }
6 }
```

```

7
8 void unlock_func(int _Atomic* lock) {
9 atomic_store(lock, FREE);
10 }

```

若使用 acquire-release 语义，可保证正确性。

## 性能分析

- 当锁几乎总是空闲时，自旋锁效率极高；
- 但在竞争严重时极其低效：
  - 因为持锁变量所在的缓存行会在各核心间不断来回失效。

回忆缓存协议：写操作会导致该缓存行独占，从而使其他副本失效。

## 工具：atomic\_fetch\_add

该操作原子地对某内存位置加上一个值并返回原值：

```

1 type atomic_fetch_add(type _Atomic* addr, type n) {
2 type res = *addr;
3 *addr += n;
4 return res;
5 }

```

## 票据锁 (Ticket Lock)

票据锁是一种非常高效的锁实现，被广泛用于 Linux 内核中。

- 基于 atomic\_fetch\_add；
- 模拟邮局取号的机制：
  - 顾客取票（号码递增）；
  - 服务窗口的显示屏号也递增；
  - 当票号与屏号相等时获得服务。
- 概念上与面包房算法相似。

```

1 struct ticket_lock {
2 int _Atomic ticket;
3 int _Atomic screen;
4 }; /* 初始化为 (0, 0) */
5
6 void lock(struct ticket_lock* t) {
7 int my = atomic_fetch_add(&t->ticket, 1);
8 while(atomic_load(&t->screen) < my) { }
9 }
10
11 void unlock(struct ticket_lock* t) {

```

```

12 atomic_fetch_add(&t->screen, 1);
13 }

```

### 特性分析

- 线程使用 load 自旋，而不是 load-modify-store；
- 因此避免了缓存行频繁失效带来的总线冲突；
- 但所有线程仍在轮询同一内存位置；
- 仍有进一步优化空间。

### 工具：atomic\_compare\_exchange\_strong

类似 exchange，但仅当变量值匹配预期时才交换。

```

1 bool atomic_compare_exchange_strong(
2 type _Atomic* addr, type* expected, type value) {
3 if(*addr == *expected) { /* 成功 */
4 *addr = value;
5 return true;
6 } else { /* 失败 */
7 *expected = *addr; /* 更新预期值 */
8 return false;
9 }
10 }

```

### 线程局部存储 (\_\_Thread\_local)

- 我们经常需要线程私有的全局变量，例如：
  - 线程编号；
  - 每线程独有的数据结构。
- 关键字 \_\_Thread\_local 定义线程本地变量：

```

1 __Thread_local int myId;

```

### MCS 锁 (Mellor-Crummey & Scott, ASPLOS' 91)

#### 基本原理：

- 构建一个 FIFO 队列，等待锁的线程依次排列；
- 当前持锁者释放锁时，仅唤醒下一个等待者。

#### 优点：

- 完全公平；
- 每个等待者自旋于自己的变量；
- 持锁线程只需唤醒其后继。

### 缺点:

- 在线程数多于 CPU 核数时, 可能出现“车队效应”(convoy effect)。

实现 (C 版):

```
1 struct node {
2 struct node* _Atomic next;
3 bool _Atomic isFree;
4 };
5
6 _Thread_local struct node my;
7 struct node* _Atomic lock = NULL;
8
9 void my_lock() {
10 my.next = NULL;
11 my.isFree = false;
12 struct node* p = atomic_exchange(&lock, &my);
13 if(p) {
14 atomic_store(&p->next, &my);
15 while(!atomic_load(&my.isFree)) { }
16 }
17 }
```

NULL 表示锁空闲; 若锁已被占用, p 为前驱节点, 将自己加入等待队列。

解锁函数:

```
1 void my_unlock() {
2 struct node* expected = &my;
3 if(!atomic_load(&my.next) &&
4 atomic_compare_exchange_strong(&lock, &expected, NULL))
5 return;
6 while(!atomic_load(&my.next)) { }
7 atomic_store(&my.next->isFree, true);
8 }
```

逻辑说明:

- 若自己仍是队头且无等待者, 则直接释放;
- 否则等待下一个线程注册;
- 注册完成后, 将其标记为可运行 (isFree = true)。

## 课程总结

- 面包房算法 (Bakery Algorithm)
- 原子加载与存储 (atomic\_load / atomic\_store)
- 内存顺序模型:
  - Relaxed
  - Release-Acquire

- Release-Consume
- Sequential Consistency
- 原子 “加载-修改-存储” 操作:
  - `atomic_exchange` ——实现自旋锁;
  - `atomic_fetch_add` ——实现票据锁;
  - `atomic_compare_exchange` ——实现 MCS 锁。

完。

# 多核编程：非阻塞算法（前 20 页）

译自 Gaël Thomas 课件

## 目录

### 第 2 页：锁算法的局限性

**Amdahl 定律：**

- $T$ ：程序执行时间；
- $p$ ：可并行化的代码比例；
- 并行时执行时间为  $T \times (1 - p + p/n)$ ；
- 加速比  $a = \frac{1}{1 - p + p/n}$ ；
- 当  $n \rightarrow \infty$  时，极限  $a \rightarrow \frac{1}{1 - p}$ 。

**数值示例：**

- $p = 75\% \Rightarrow a \rightarrow 4$ （当  $n \rightarrow \infty$ ），在 32 核时约为 3.7；
- $p = 95\% \Rightarrow a \rightarrow 20$ （当  $n \rightarrow \infty$ ），在 32 核时约为 12.55，在 128 核时约为 17.42。

**结论：**我们必须努力让剩下的那一点串行部分也能并行化！

### 第 3 页：非阻塞算法

非阻塞算法的目标：构建在多线程环境下不会出现阻塞的算法。

**三种类型：**

- **无等待 (Wait-free)：**每个操作在有限步内完成；
- **无锁 (Lock-free)：**若不断调用该操作，则该操作会不断成功完成；
- **无干扰 (Obstruction-free)：**若线程独自执行，则能在有限步内完成。

三者强度关系如下：

$\text{Wait-free} \Rightarrow \text{Lock-free} \Rightarrow \text{Obstruction-free}$

反向不成立。

**注意：**只要一个操作中使用了锁，它就不再是无干扰的。

## 第 4 页：非阻塞数据结构分类

非阻塞算法通常用于以下几类结构：

1. 栈 (Stack)
2. 队列 (Queue)
3. 链表 (Linked List)

## 第 5 页：栈的操作

栈支持以下操作：

- `push(Element e)`：压入一个元素；
- `Element pop()`：弹出一个元素。

## 第 6–7 页：使用锁的栈实现

```
1 class Node {
2 Node next;
3 Element element;
4 }
5
6 class Stack {
7 Node head;
8 }
9
10 synchronized void push(Element e) {
11 Node n = new Node(head, e);
12 head = n;
13 }
14
15 synchronized Element pop() {
16 Node n = head;
17 head = n.next;
18 return n.element;
19 }
```

这种实现方式简单，但加锁会导致线程相互阻塞，难以扩展到多核。

## 第 8–10 页：无锁栈 (Scott 1991)

核心思想：对栈顶指针 `head` 使用原子比较交换 (CAS, Compare-And-Swap)。

```
1 class Node {
2 Node next;
3 Element element;
4 }
5
6 class Stack {
```

```
7 Node head;
8 }
9
10 void push(Element e) {
11 do {
12 Node n = new Node(head, e);
13 } while (atomicCAS(&head, n.next, n) != n.next);
14 }
15
16 Element pop() {
17 do {
18 Node n = head;
19 if (n == null) error("空栈");
20 } while (atomicCAS(&head, n, n.next) != n);
21 return n.element;
22 }
```

## 第 11 页：无锁栈的性质

- **无锁 (Lock-free)**: 若多个线程不断执行 push 或 pop, 这些操作会持续成功。因为一个成功的 CAS 会使得其他线程的 CAS 失败, 从而保证系统整体前进。
- **非无等待 (Not Wait-free)**: 可能存在线程反复被其他线程干扰, 使自己的 CAS 始终失败。

## 第 12 页：非阻塞数据结构概览

1. 栈 (Stack)
2. 队列 (Queue)
3. 链表 (Linked List)

## 第 13 页：队列的基本操作

队列支持两种操作：

- enqueue(Element e): 将元素添加到队尾；
- dequeue(): 从队头取出一个元素。

## 第 14–16 页：使用锁的队列实现

```
1 class Queue {
2 Node head = null;
3 Node tail = null;
4 }
5
6 class Node {
7 Node next;
```

```
8 Element element;
9 }
10
11 synchronized void enqueue(Element e) {
12 Node n = new Node(null, e);
13 if (tail != null) tail.next = n;
14 else head = n;
15 tail = n;
16 }
17
18 synchronized Element dequeue() {
19 Node n = head;
20 head = n.next;
21 if (head == null) tail = null;
22 return n.element;
23 }
```

这种实现方式简单直观，但加锁限制了并发性能。

## 第 17–18 页：无锁队列的设计原则

设计原则：

- 始终保留一个虚拟节点（fake node）以简化初始化；
- 尾指针延迟更新（lazy update）；
- 始终满足以下三个不变式：
  1. 若头节点之后存在节点，则该节点是第一个入队的节点；
  2. 从 head 和 tail 出发的链表至少共享一个节点；
  3. 这两个链的最后一个节点即是最新入队的节点。

尾指针可能暂时落后于实际尾部，但稍后会被自动更新。

## 第 19 页：无锁队列结构定义

```
1 class Queue {
2 Node head = new Node(null, null); // 虚拟节点
3 Node tail = head;
4 }
5
6 class Node {
7 Node next;
8 Element element;
9 }
```

初始状态：

$$Head \rightarrow [Fake] \leftarrow Tail$$

## 第 20 页：无锁队列入队操作 (enqueue)

```
1 void enqueue(Element e) {
2 Node node = new Node(null, e);
3 do {
4 Node old = tail;
5 while (old.next != null) {
6 CAS(&tail, old, old.next);
7 old = tail;
8 }
9 } while (CAS(&old.next, null, node) != null);
10 CAS(&tail, old, node);
11 }
```

执行过程：

1. 创建新节点；
2. 若发现尾节点 `old.next` 不为空，则通过 CAS 推进 `tail`；
3. 当找到真正尾节点时，将其 `next` 指向新节点；
4. 最后尝试更新尾指针。

结果结构：

$$Head \rightarrow [Fake] \rightarrow A \leftarrow Tail$$

## 第 21 ~ 23 页：无锁队列入队过程（续）

入队过程反复尝试，直到成功为止。

```

1 void enqueue(Element e) {
2 Node node = new Node(null, e);
3 do {
4 Node old = tail;
5 while (old.next != null) {
6 CAS(&tail, old, old.next);
7 old = tail;
8 }
9 } while (CAS(&old.next, null, node) != null);
10 CAS(&tail, old, node);
11 }

```

步骤说明：

1. 反复读取当前尾节点；
2. 若发现 `old.next` 非空，说明 `tail` 落后，需要通过 CAS 推进；
3. 找到真正的尾节点后，将其 `next` 指向新节点；
4. 最后尝试用 CAS 更新尾指针（若失败则无碍，下次会被其他线程推进）。

最终结果示意：

$$Head \rightarrow [Fake] \rightarrow A \leftarrow Tail$$

## 第 24 ~ 35 页：并行入队（C 与 B 同时入队）

假设线程 1 入队元素 C，线程 2 入队元素 B。

情形：

- 两个线程都从当前的 `tail` 出发；
- 通过 CAS 操作竞争更新最后节点的 `next`；
- 当其中一个线程成功时，另一个线程检测到变化后，重新读取尾节点并前移；
- 若尾指针落后，线程会“帮忙”推进。

这是 lock-free 算法的关键思想：**线程间协作性推进 (cooperative helping)**。

并发情况下的链表变化示意：

$$Head \rightarrow [Fake] \rightarrow A \rightarrow C \rightarrow B \leftarrow Tail$$

无论执行顺序如何，入队顺序的一致性始终保持。

注意：

- CAS 的失败仅代表竞争，没有数据错误；
- 每个入队操作最终都能成功；
- 尾指针的延迟更新不会影响逻辑正确性。

## 第 36 ~ 39 页：出队操作 (Dequeue)

出队操作从队列头部移除一个元素。

```
1 Element dequeue() {
2 do {
3 Node res = head;
4 if (res.next == null)
5 return null; // 队列为空
6 } while (CAS(&head, res, res.next) != res);
7
8 return res.next.element;
9 }
```

执行逻辑：

1. 读取当前头节点；
2. 若 `head.next` 为空，说明队列为空；
3. 使用 CAS 将 `head` 前移到下一个节点；
4. 若 CAS 失败（有其他线程先一步成功），重新尝试；
5. 最终返回新头节点后的元素。

图示说明：

出队前：

$$Head \rightarrow [Fake] \rightarrow A \rightarrow C \rightarrow B \rightarrow Tail$$

出队一次后：

$$Head \rightarrow A \rightarrow C \rightarrow B \rightarrow Tail$$

原虚拟节点被替换为新的头节点。

## 第 40 页：无锁队列性质总结

**无锁特性 (Lock-free)：**

- 如果多个线程持续执行 `enqueue()` 或 `dequeue()` 操作；
- 那么这些操作中必定有一个能成功完成；
- 因此系统整体不会陷入阻塞。

**非无等待 (Not Wait-free)：**

- 单个线程可能因为竞争而反复 CAS 失败；
- 因此无法保证每个线程在有限步内完成；
- 但整体系统仍然持续前进 (progress)。

**小结：**

- 无锁队列依赖 **原子 CAS 操作** 保证正确性；
- 通过“虚拟节点 + 尾指针延迟更新”简化并发结构；

- 每次入队出队都只需要常数步（平均意义上）；
- 算法满足 lock-free，但不满足 wait-free。

## 第 41 页：非阻塞数据结构回顾

本课程中介绍的非阻塞数据结构主要有三类：

1. 栈 (Stack)：结构最简单，CAS 直接作用在栈顶指针上；
2. 队列 (Queue)：需要保持头尾两个方向上的不变式；
3. 链表 (Linked List)：最复杂，因为插入和删除都可能发生在中间位置，并且有并发线程同时操作同一位置。

后面几页将专门讨论链表的无锁实现。

## 第 42 页：链表的问题

在并发环境中，链表的主要困难在于：

- 插入和删除往往发生在**同一个位置**；
- 例如，一个线程正在尝试在 A 和 C 之间插入 X，另一个线程同时要删除 B，这会造成竞争；
- 如果只用普通的指针操作，很难既保证正确性，又做到无锁。

示意结构：

$$A \rightarrow B \rightarrow C$$

此时若有人要在 B、C 之间插入 X，就会与删除 B 的线程发生冲突。

## 第 43 页：同一位置的插入与删除

进一步解释这个问题：

- 假设链表为  $A \rightarrow B \rightarrow C$ ；
- 线程 1：想在 B 和 C 之间插入 X；
- 线程 2：想删除 B；
- 如果两个线程都只看到了“B 还在”，就都可能以 B 为基准做 CAS；
- 但其中一个线程的操作会让另一个线程的“前提条件”失效，从而需要重试。

因此，简单的“next 指针 CAS”还不够，需要额外的信息来判断节点在遍历过程中是否被删除。

## 第 44 页：并发插删的根本冲突

问题的本质是：

- 链表的“当前位置”不是永远稳定的；
- 当我们正在遍历链表、打算在某处插入时，这个位置的节点可能已经被其他线程删除了；
- 如果看不到这种“节点被删除”的状态，就会把一个已经失效的节点当成有效节点来操作；
- 这就会破坏链表结构。

所以，需要一种办法在指针里“带上状态”。

## 第 45 页：再看插入位置

再看同一个例子：

- 链表： $A \rightarrow B \rightarrow C$ ；
- 我们要在 B 和 C 之间插入 X；
- 如果在我们准备插入时，有别的线程把 B 删掉了，那我们这次插入就不能再基于 B 来做；
- 必须能**发现**“B 被删了”，然后**重新开始遍历**；
- 这正是 Tim Harris 算法的要点。

## 第 46 页：Harris 算法的基本思想（2001）

Tim Harris 在 2001 年提出的无锁链表算法有几个关键点：

- 每个节点都有一个“颜色”或“标记”；
- **白色**表示节点**仍然存在**；
- **灰色**表示节点**已经被删除**（逻辑删除）；
- 如果在插入过程中，发现相关节点的颜色变化了（说明被删了），就**重新开始**；
- 删除并不是一次性做完的，删除线程只需要把节点“标记成灰色”，真正把它从链表里摘掉可以由**任意后续线程**完成。

## 第 47 页：为什么要把颜色放到指针里

如果我们只对 next 做 CAS，就看不到“颜色”这种额外状态。

解决办法是：**把颜色（删除标记）塞进指针本身**。常见做法是用指针的最低一位表示“是否被删除”。

- 指针的高位部分是真正的节点地址；
- 最低位（如第 0 位）表示“这个节点是否被标记为删除”；
- 这样，读一次 next，既得到了“下一个节点是谁”，也得到了“当前节点是不是被删了”；
- 然后就可以在遍历时顺手把被删节点清理掉。

## 第 48 页：有色指针的定义

下面是一个典型的“把标记塞进指针”的写法（C 风格）：

```
1 typedef uintptr_t coloredPointer;
2
3 Node* pointer(coloredPointer ptr) {
4 return (Node*)(ptr & -2); // 去掉最低位，得到真实指针
5 }
6
7 int mark(coloredPointer ptr) {
8 return ptr & 1; // 取最低位，看是不是被标记
9 }
```

思路说明：

- 要删除一个节点：不是马上把它从链表上摘掉，而是把它的 `next` 标记成“已删”；
- 以后任何一个线程在遍历时，只要发现这个标记，就会尝试把它“真正删掉”（也就是把它从链上跳过去）；
- 这种方式也叫“机会式删除”或者“遍历时清理”。

节点结构可以这样写：

```
1 class Node {
2 coloredPointer next;
3 Element element;
4 }
```

## 第 49 页：删除操作——遍历阶段（1）

删除的总体结构如下（伪代码）：

```
1 void del(coloredPointer* plist, int value) {
2 restart:
3 coloredPointer* pred = plist;
4 while (!found) {
5 Node* cur = pointer(*pred);
6 // ... 继续往后找 ...
7 pred = &cur->next;
8 }
9 }
```

说明：

- `plist` 是链表头指针的地址；
- `pred` 表示当前节点的前驱指针位置（也就是指向“指向当前节点的那个 `next`”）；
- 我们一边走一边更新 `pred`，找到要删的值或走到比它大的位置。

## 第 50 页：删除操作——遍历阶段（2）

遍历时的状态可以这样想象：

- `pred` 指向前一个节点的 `next`；
- `cur` 是当前正在看的节点；
- 然后我们要判断 `cur` 是否是要删的那个值；
- 若不是，就向后走：`pred = &cur->next`。

这一页的重点是：删除其实也是一次普通的遍历，只是多了“顺便清理删掉的节点”这一步。

## 第 51 页：删除操作——遍历阶段 (3)

继续遍历的过程中，可能遇到两种情况：

1. 当前节点还在（白色）：那就正常往后走；
2. 当前节点已经被其他线程标记为删除（灰色）：那我们要尝试把它从链上摘掉。

因此，删除逻辑不仅要找“我要删的值”，还要顺手清理前面别人删过但没来得及摘掉的节点。

## 第 52 页：删除操作——遍历阶段 (4)

当我们发现一个节点被标记为删除（也就是 `mark(cur->next)` 为真）时，我们要做的是：

- 用 CAS 把 `pred` 直接指向 `cur` 的下一个真实节点；
- 如果 CAS 成功，说明我们把这个“灰节点”清理掉了；
- 如果 CAS 失败，说明在我们清理的同时，`pred` 或链表结构被别的线程改了，只好回到 `restart` 重新来。

## 第 53 页：删除操作——遍历阶段 (5)

伪代码可写成这样（已经带清理逻辑了）：

```
1 void del(coloredPointer* plist, int value) {
2 restart:
3 coloredPointer* pred = plist;
4 while (!found) {
5 Node* cur = pointer(*pred);
6
7 if (mark(cur->next)) {
8 // 当前节点已被删除，尝试跳过它
9 if (CAS(pred, cur, pointer(cur->next)) != cur)
10 goto restart;
11 else
12 continue;
13 }
14
15 pred = &cur->next;
16 }
17 }
```

## 第 54 页：删除操作——机会式清理

这一页的核心句子是：

**“遍历到已删除节点时，要机会式地把它真正移除。”**

也就是说，删除是分两步的：

1. 某个线程把节点标记为删除（逻辑删除）；
2. 以后任意一个遍历到这个节点的线程，都可以把它真正从链表中移除（物理删除）。

这样就不需要专门的垃圾回收线程来清理这些节点了。

## 第 55 页：CAS 失败的两种典型原因

当我们尝试

$$CAS(pred, cur, pointer(cur \rightarrow next))$$

时，这个 CAS 可能失败，原因主要有两种：

- **原因 1：** `pred` 在此期间也被别的线程标记为删除了；
- **原因 2：** 有别的线程在 `pred` 和 `cur` 之间插入了新的节点。

不论是哪一种情况，这一页的做法都是：**回到 restart 重新遍历**，因为你手里的“链表视图”已经不是最新的了。

## 第 56 页：从链表中实际移除

如果 CAS 成功了，就表示我们已经把这个灰色节点从链表中真正移除了，这时可以继续向后遍历。

此时链表结构会变得更“干净”，后续的插入和删除也会更容易成功。这是无锁链表的一个重要特点：**遍历的过程中结构会被逐渐清理干净**。

## 第 57 页：继续遍历的效果

当清理动作成功时，链表会变成“没有那个被删节点”的样子，其效果就像是：

- 删除线程只做了逻辑删除；
- 但下一个路过的线程顺手做了物理删除；
- 整个系统共同维护了链表的一致性；
- 所以，即使某次删除来不及做完，也不会留下永久的脏数据。

## 第 58 页：删除操作的继续说明

在并发场景中，有时你会看到这样的顺序：

1. 线程 A 标记了节点 X 为删除；
2. 线程 B 正好也在遍历这里，发现 X 被删了；
3. 线程 B 做了真正的 CAS，把 X 跳过去；
4. 线程 A 以为还要删 X，但是这时它的 CAS 就会失败；
5. 线程 A 发现失败，就重新来。

这正是无锁算法里典型的“谁看到谁清”的模式。

## 第 59 页：为什么要重启

这一页的重点是：只要发现链表结构和我们假设的不一致，就必须重启遍历。因为：

- 链表可能在我们手里“过期”了；
- 我们不能基于一个过期的前驱节点做插入或删除；
- 重启虽然代价高一点，但能保证正确性；
- 无锁算法的进度性靠的是“总有人能成功”，而不是“每个人一次就成”。

## 第 60 ~ 64 页：真正的删除——标记要删的值

接下来是“真正要删某个值”的部分。伪代码如下：

```
1 void del(coloredPointer* plist, int value) {
2 restart:
3 coloredPointer* pred = plist;
4 while (!found) {
5 Node* cur = pointer(*pred);
6
7 // 找到了要删的值
8 if (cur->value == value) {
9 coloredPointer n;
10 // 把当前节点的 next 标记为 1 (逻辑删除)
11 do {
12 n = cur->next;
13 } while (CAS(&cur->next, n, n | 1) != n);
14 found = 1;
15 }
16
17 // 遇到已经删除的节点，就尝试把它跳过去
18 if (mark(cur->next)) {
19 if (CAS(pred, cur, pointer(cur->next)) != cur)
20 goto restart;
21 else
22 continue;
23 }
24
25 pred = &cur->next;
26 }
27 }
```

说明：

- 第一阶段：把要删的节点标记成“灰色”；
- 第二阶段：像前面几页那样，把它从链表中跳过；
- 如果中间任何一步 CAS 失败，就回到 restart。

## 第 65 ~ 67 页：并发插入导致的重启

如果你删除某个值的同时，有别的线程在这个位置插入了新节点，那么你的 CAS 也会失败。例如：

- 链表里有 12, 17, 22;
- 你要删 17;
- 在你删的过程中，别的线程插入了 14;
- 你手里的前驱还是 12，但现在真正的前驱应该是 14 了;
- 所以你的 CAS 会失败，你需要 `goto restart`;
- 不过你已经把 17 标记成了删除状态，所以下一次遍历的人会把它清掉。

这说明：**标记成功 立刻从链表里消失**，真正的删除是“延迟完成”的。

## 第 68 ~ 69 页：要删的值不存在的情况

如果你要删的值根本不在链表里，会怎么样？伪代码里的这几行负责这个逻辑：

```
1 if (cur == NULL || value < cur->value)
2 return 0; // 没找到
```

这是典型的“有序链表”删除写法：

- 链表是按值递增的;
- 一旦当前值比要删的值大，就说明没必要再往后找了;
- 直接返回没找到即可;
- 这一步也要检查删除标记，遇到灰节点要清理。

## 第 70 ~ 71 页：插入操作——遍历

插入操作和删除操作的套路几乎一样，也是：

1. 从头开始遍历;
2. 一边走一边清理被标记删除的节点;
3. 找到合适的插入位置;
4. 用 CAS 把新节点插进去;
5. CAS 失败就重启。

示例代码：

```
1 void add(coloredPointer* plist, int value) {
2 restart:
3 coloredPointer* pred = plist;
4 while (true) {
5 Node* cur = pointer(*pred);
```

```
6
7 // 插入位置：到了链表末尾，或者当前值比要插的值大
8 if (cur == NULL || value < cur->value) {
9 if (CAS(pred, cur, new Node(cur, value)) != cur)
10 goto restart;
11 else
12 return;
13 }
14
15 // 如果当前节点被标记删除，就尝试清理
16 if (mark(cur->next)) {
17 if (CAS(pred, cur, pointer(cur->next)) != cur)
18 goto restart;
19 else
20 continue;
21 }
22
23 pred = &cur->next;
24 }
25 }
```

## 第 72 ~ 74 页：插入中的 CAS 失败原因

插入时 CAS 失败的原因和删除时非常像：

- 前驱节点在此期间被别的线程标记删除了；
- 有别的线程在你前面插入了一个新节点；
- 因为你是按“当前视图”来的，而真实链表已经变了；
- 所以最安全的办法仍然是：**回到 restart 重新遍历一次。**

这种写法看起来“重复”，但好处是：**代码逻辑局部且简单**，不需要大锁，也不需要全局顺序。

## 第 75 页：插入成功后的链表

当 CAS 成功时，链表结构就是我们期望的样子了：

- 新节点的 next 指向原来位置的节点；
- 前驱的 next 指向新节点；
- 若有其他线程也想在这个位置插入，它会因为 CAS 失败而重试；
- 因此链表最终还是有序、无环、可遍历的。

这也说明：**无锁并不等于不要重试**，恰恰相反，无锁算法里重试是很常见的。

## 第 76 页：本章要点总结

本节关于非阻塞算法的主要结论如下：

- 非阻塞算法有三个强度层次：无等待 (wait-free)、无锁 (lock-free)、无干扰 (obstruction-free)，从强到弱，反向不成立；
- 本课程示范了三种典型的无锁数据结构：栈、队列、链表；
- 栈的无锁实现最简单，直接对栈顶做 CAS；
- 队列的无锁实现要保持头尾的不变式，并且让尾指针延迟更新；
- 链表的无锁实现最复杂，需要：遍历时清理、逻辑删除 + 物理删除分离、以及把标记塞进指针；
- 每个无锁算法都可以找到一个**线性化点**——也就是操作真正“生效、对所有线程可见”的那一行代码（通常是 CAS 那一行）；
- 虽然这些结构都能做到不加锁就前进，但它们的实现难度也确实比加锁高很多，因此在工程中往往会配合更高层的并发抽象或库来用。

## 1 事务内存概述 (Transactional Memory Overview)

### 1.1 锁机制的局限性 (Limitations of Lock-Based Algorithms)

传统并发程序依赖互斥锁 (lock) 来保护共享数据, 但这种方法在多核环境下存在显著缺陷:

- **大临界区 (large critical section) 限制性能**: 当多个线程争用同一锁时, 只有一个线程能进入临界区, 从而导致可伸缩性下降。
- **细粒度锁 (fine-grain locking) 与无锁算法的缺陷**:
  - 代码复杂、难以维护;
  - 不易复用——很多锁顺序或不变量仅存在于开发者脑中;
  - Bug 难以发现;
  - 不可组合 (一个数据结构只能匹配一个专用算法);
  - 难以形式化验证正确性。

**事务内存的基本想法 (Idea of Transactional Memory)** 事务内存 (Transactional Memory, TM) 旨在提供一种 **更高层次的并发抽象接口**, 使开发者能像写顺序程序一样编写并发代码。

- 提供统一的高层 API (atomic block);
- 简化开发、减少锁管理;
- 性能目标: 尽可能接近无锁算法。

### 1.2 事务内存的基本原理 (Principle of Transactional Memory)

**核心概念: 原子块 (Atomic Block)** 事务内存定义了一个基本构造:

`atomic { ... }` ——一个在逻辑上“瞬间执行”的代码块。

**优势**

- 无需知道需要获取哪些锁, 也不用考虑锁的顺序;
- 避免常见并发错误 (如死锁、饥饿)。

**示例** 假设有两个原子块:

```
atomic {
 tmp = x;
 tmp = tmp + 1;
 x = tmp;
}
```

```
atomic {
 tmp = x;
 tmp = tmp * 2;
 x = tmp;
}
```

系统可能按不同顺序调度事务：

- 先执行第一个，再执行第二个 结果  $x = 44$ ；
- 或先执行第二个，再执行第一个 结果  $x = 43$ 。

不论顺序如何，系统都能保证每个事务在外部看来是“原子”执行的。

### 1.3 从锁到事务内存 (From Locks to TM)

传统同步 (如 Java `synchronized`):

```
synchronized(o) {
 if(!x) {
 x = true;
 doSomething();
 }
}
```

改写为事务形式：

```
atomic {
 if(!x) {
 x = true;
 doSomething();
 }
}
```

#### 高层视角

- 每个临界区 (critical section) 都可替换成一个 `atomic` 块；
- 因此底层锁机制可被完全移除。

### 1.4 从条件变量到事务内存 (From Condition Variables to TM)

在传统模型中，经常使用锁与条件变量等待某事件：

```
synchronized(o) {
 while(!x)
 o.wait();
}
```

```
synchronized(o) {
 x = true;
 o.notify();
}
```

事务内存提供了类似机制——`retry` (Harris, 2005):

```
atomic {
 while(!x)
 retry;
```

```
}

atomic {
 x = true;
}
```

其语义为：

- 当条件 `!x` 为真时，事务挂起；
- 一旦变量 `x` 被其他事务修改并提交，挂起事务会被唤醒并重新执行；
- 无需显式锁与条件变量即可实现等待机制。

## 2 事务的可组合性与运行时设计 (Composability and Runtime Design)

### 2.1 可组合性 (Composability)

事务内存的一大优势是：**事务可以自由组合而保持原子性**。如果 A 和 B 都是原子块 (atomic blocks)，它们可以安全地被组合成一个更复杂的事务。

**示例：队列 (Queue) 操作**

```
atomic Elmt deq();
atomic void enq(Elmt e);

atomic move(Queue dst, Queue src) {
 Elmt e = src.deq();
 dst.enq(e);
}
```

在这个例子中：

- `deq()` 表示出队操作；
- `enq()` 表示入队操作；
- `move()` 将一个元素从队列 `src` 移动到 `dst`。

无论内部调用多少个原子块，组合后的 `move()` 在语义上仍是一个**原子事务**。因此，事务内存天然具有**可组合性 (composability)**。

### 2.2 可组合性与重试 (Composability and Retry)

事务内存支持通过 `retry` 来实现“条件等待”。例如，一个带界限的阻塞队列：

```
void enq(Elmt e) {
 atomic {
 if(queue.size() == MAX_SIZE)
 retry; // 队列满则挂起
 queue.addLast(e);
 }
```

```

 }
}

Elmt deq() {
 atomic {
 if(queue.empty())
 retry; // 队列空则挂起
 return queue.removeLast();
 }
}

```

### 运行机制

- 当 `retry` 被执行时，事务会自动挂起；
- 系统会监控事务的 `read set`；
- 一旦读集中的变量被其他事务修改，挂起的事务会被唤醒并重新执行；
- `retry` 具备“可组合性”：若嵌套事务中有 `retry`，则整个外层事务都会重新启动。

## 2.3 通过替代实现可组合性 (Composability by Alternative)

有时我们希望事务尝试一个动作，如果失败或需重试，则执行另一种逻辑。为此，事务内存提供了 `orElse` 结构。

### 示例：非阻塞出队 (Non-blocking Dequeue)

```

Elmt deqNoWait() {
 atomic {
 return deq(); // 若队列为空会触发 retry
 } orElse {
 return null; // 否则立即返回 null
 }
}

```

### 语义说明

- 系统首先执行第一个 `atomic` 块；
- 如果该块触发 `retry`，则立即执行 `orElse` 块；
- 若多个 `orElse` 连续定义，则按顺序依次尝试；
- 当所有分支都触发 `retry` 时，整个事务重新开始。

因此：

`orElse` 提供了一种声明式的“优先级回退机制”，用于构建非阻塞、可恢复的并发结构。

## 2.4 事务运行时的设计 (Design of a TM Runtime)

事务内存运行时 (runtime) 主要有两种设计思路：

### 1. 悲观策略 (Pessimistic Solution)

- 在事务开始时获取一个全局锁；
- 结束时释放；
- 优点：实现简单；
- 缺点：相当于顺序执行，**极不高效**。

### 2. 乐观策略 (Optimistic Solution)

- 不加锁，直接执行事务；
- 在提交阶段检查是否与其他事务冲突；
- 若检测到冲突，则中止 (abort) 并重试。

**冲突的定义 (What is a Conflict?)** 当事务无法“看起来像是瞬间执行”时，就发生冲突。常见情形包括：

- 事务 Y 读到了事务 X 尚未提交的中间状态；
- 事务 X 在执行期间，其读集中的变量被事务 Y 修改。

**解决方法** 在冲突出现时，系统必须：

- 中止读者 (reader)、写者 (writer) 或双方；
- 重新执行其中之一；
- 保证所有可见状态始终与某种“顺序化执行”等价。

## 3 事务冲突与更新模型 (Conflicts and Update Models)

### 3.1 读写冲突 (Read-Write Conflicts)

设两个事务 X 与 Y 共享变量 A，可能出现以下两类冲突：

#### 1. 双写 + 读者 (Double Write with a Reader)

- 事务 X 连续对变量 A 写入 b 与 c；
- 事务 Y 在此期间读取了 b；
- 如果事务是原子的，b 在外部应当从未“存在”过；
- 因此 Y 观察到了一个临时状态——构成冲突。

#### 2. 双读 + 写者 (Double Read with a Writer)

- 事务 X 对 A 写入 b；
- 事务 Y 先读到 a，又读到 b；
- 表明 Y 在执行期间，A 被 X 修改；
- 违背“原子执行”假设，因此也是冲突。

**处理方式** 在这种情况下，系统必须选择：

- 中止读者；
- 中止写者；
- 或同时中止双方；

以维持所有事务的原子性和一致性。

### 3.2 两种主要的事务实现设计 (Two Main Possible Designs)

事务内存的底层实现主要有两种方式，区别在于写入更新的时机：

#### 1. 延迟更新 (Deferred Update, Redo Log)

- 事务执行时，写操作仅记录到 **重做日志 (redo log)**；
- 直到提交 (commit) 时，才将日志内容写回主内存；
- 优点：
  - 避免 “双写冲突”；
  - 仅需处理 “双读冲突”；
- 缺点：
  - 提交阶段工作量大；
  - 回滚 (abort) 几乎无代价。

#### 2. 立即更新 (Immediate Update, Undo Log)

- 写操作立即修改主内存；
- 同时将旧值保存至 **回滚日志 (undo log)**；
- 若事务中止，则恢复旧值；
- 优点：
  - 提交开销小；
- 缺点：
  - 同时面临 “双读” 和 “双写” 冲突；
  - 需要频繁回滚。

### 3.3 延迟更新事务示例 (Deferred-Update TM Example)

**示例代码**

```
atomic {
 tmp = x;
 tmp = tmp + 1;
 x = tmp;
}
```

```
atomic {
 tmp = x;
 tmp = tmp * 2;
 x = tmp;
}
```

### 执行过程

1. 初始状态:  $x = 20$
2. 事务 T1: 读取  $x=20$ , 计算  $tmp=21$ , 记录到 redo log;
3. 事务 T2: 读取  $x=20$ , 计算  $tmp=40$ , 记录到 redo log;
4. 若 T1 先提交:
  - 将 redo log 写入内存;
  - 主存更新为  $x=21$ ;
5. 若 T2 随后提交:
  - 检测到冲突 (读集包含被修改的  $x$ );
  - T2 中止 (abort)。

### 关键点

- 每个事务维护独立的读集 (read set) 与写集 (write set);
- 提交时检查: 若读集变量被他人修改  $\rightarrow$  冲突;
- 仅在成功提交时才真正修改主内存。

## 3.4 立即更新事务示例 (Immediate TM Example)

### 示例代码

```
atomic {
 z = y + 7;
}
```

```
atomic {
 y = x * 2;
}
```

### 执行示意

- 初始状态:  $x=3, y=20, z=30$
- 第一个事务:
  - 读  $y=20$ ;
  - 计算  $z=27$ ;

- 第二个事务：
  - 写  $y=6$ ;
- 若冲突（第二个事务修改了第一个事务的读集）：
  - 第一个事务回滚；
  - 系统利用 undo log 恢复原始状态；
  - 最终内存：  $x=3$ ,  $y=6$ ,  $z=30$ 。

## 比较

- 延迟更新：在提交时写主存，适合高冲突场景；
- 立即更新：实时写主存，适合高提交率场景。

### 3.5 冲突检测机制 (Conflict Detection Strategies)

事务系统中存在两种冲突检测策略：

- **Eager Detection (即时检测)**：
  - 每次读写操作时立即检查；
  - 优点：冲突发现早；
  - 缺点：需要为每次访问插桩 (instrumentation)，开销较大。
- **Lazy Detection (延迟检测)**：
  - 仅在提交阶段检测冲突；
  - 优点：减少检测开销；
  - 缺点：事务可能在无效数据下继续运行；
  - 特别是在“双读冲突”情况下，可能造成暂时不一致。

**总结** 选择策略取决于应用场景：

- 即时检测适用于对一致性要求极高的系统；
- 延迟检测更适合追求性能的通用应用。

## 4 事务内存的实现技术与朴素算法 (Implementation Techniques and Naive Algorithm)

### 4.1 事务内存的主要实现方式 (Implementation Techniques)

根据底层实现，事务内存 (TM) 可以分为三种类型：

#### 1. 硬件事务内存 (Hardware Transactional Memory, HTM)

- 利用处理器缓存 (CPU cache) 来构建延迟更新型事务；
- 通常采用延迟检测 (lazy detection)，即通过专门指令在事务末尾检查冲突；
- 优点：执行效率极高；
- 缺点：受限于缓存大小，仅适合较小的事务。

## 2. 软件事务内存 (Software Transactional Memory, STM)

- 通过编译器或运行时系统注入代码 (code instrumentation) 实现;
- 优点: 事务大小无限制;
- 缺点: 执行较慢, 存在额外开销。

## 3. 混合事务内存 (Hybrid Transactional Memory, HyTM)

- 若事务规模较小, 则使用硬件执行;
- 若超出硬件能力或冲突过多, 则自动切换到软件执行;
- 优点: 结合 HTM 与 STM 的长处;
- 缺点: 硬件与软件之间的切换成本较高。

## 4.2 朴素算法 (Naive STM Algorithm)

**设计思想** 该算法结合了:

- 延迟更新 (deferred update);
- 纯延迟冲突检测 (lazy conflict detection);
- 在提交阶段使用全局锁。

**基本原理** 每个内存单元都附带一个整数计数器 (version counter):

- **读操作 (Read):** 读取变量的值, 并记录其计数器;
- **写操作 (Write):** 写入到本地日志 (local log), 不立即修改主存;
- **提交阶段 (Commit):**
  1. 检查所有读过的变量, 其计数器是否未被修改;
  2. 若未变化:
    - 提交成功;
    - 将本地写入同步到主内存;
    - 并增加这些变量的计数器;
  3. 若检测到变化:
    - 中止事务 (abort);
    - 丢弃本地日志。

### 核心思想

只有当读到的所有变量版本 (计数器) 在提交时仍未变化, 事务才能被认为是“原子执行”的。

### 4.3 朴素 STM 执行示例 (Execution Example)

假设内存中包含五个变量，每个变量都有一个计数器。

变量名	计数器	初始值
a	13	17
b	2	26
c	7	10
d	21	8
e	83	17

#### 事务代码

```
atomic {
 a = a + b;
 c = a - e;
 b = c;
}
```

#### 执行步骤

1. **读取阶段 (Read Phase)**: 读取 a, b, c, e 的值与计数器, 保存到本地日志;
2. **计算阶段 (Execution Phase)**: 在本地内存中计算:

$$a = 17 + 26 = 43, \quad c = 43 - 17 = 26, \quad b = 26$$

3. **提交阶段 (Commit Phase)**:

- 再次读取主存中的计数器;
- 若与之前一致 → 提交成功;
- 更新主存并递增计数器;
- 否则 → 中止事务。

#### 提交后的主存状态

变量名	新值	新计数器
a	43	14
b	26	3
c	26	8
d	8	21
e	17	83

#### 结果分析

- 如果在事务执行期间, 没有其他事务修改这些变量;
- 则所有计数器在提交前保持不变;
- 事务成功提交;
- 主存与日志同步, 保证一致性。

### 关键点总结

- STM 通过计数器机制检测冲突；
- 若读集的任意变量版本被更改，则事务必须中止；
- 若验证通过，则日志中的更新原子地写回主存；
- 提交后，新的计数器版本表示“时间向前推进”。

## 5 僵尸事务问题与一致性破坏 (Zombie Transactions and Consistency Violations)

### 5.1 僵尸事务的定义 (Definition of Zombie Transactions)

**概念** 僵尸事务 (Zombie Transaction) 是指：一个事务在其依赖的数据已经被其他事务修改或删除后，仍继续运行，而没有被系统检测并中止的情况。

这类事务会导致：

- 空指针异常 (Null Pointer Exception)；
- 数据不一致；
- 违反逻辑约束；
- 程序崩溃或产生错误结果。

### 5.2 示例一：引用失效 (Invalid Reference Example)

#### 事务代码

```
atomic {
 if (x != null)
 x.f();
}
```

```
atomic {
 x = null;
}
```

#### 问题说明

- 第一个事务读取  $x$  (非空)，准备执行  $x.f()$ ；
- 第二个事务随后将  $x$  设置为 `null`；
- 如果第一个事务没有重新验证  $x$ ，它仍会尝试调用  $x.f()$ ；
- 结果：程序抛出 `NullPointerException`。

这种情况下，第一个事务在逻辑上已经“死去”，但仍在执行，因此被称为：

**Zombie Transaction (僵尸事务) .**

### 5.3 错误原因与直接后果 (Root Cause and Effects)

**根本原因** 事务 A 在执行时：

1. 读取  $x$ ;
2. 检查  $x \neq \text{null}$ ;
3. 在检查后未再次验证;
4. 期间事务 B 改变了  $x$ ;
5. 事务 A 仍继续执行  $x.f()$ ;

此时，A 访问了已失效的对象引用 (stale reference)，STM 系统却未检测出冲突，从而引发异常。

**后果**

- 系统可能崩溃;
- 逻辑不变量被破坏;
- 数据库或内存状态不再一致。

### 5.4 示例二：违反不变量 (Invariant Violation Example)

**事务代码**

```
atomic {
 t1 = x;
 t2 = y;
 p = 1 / (t1 - t2);
}
```

```
atomic {
 x = 217;
 y = 4;
}
```

**初始状态**

$x = 4, \quad y = 5, \quad \text{且约束为 } x \neq y$

**执行过程**

1. 事务 A 读取  $x = 4, y = 5$ ;
2. 事务 B 修改为  $x = 217, y = 4$  并提交;
3. 事务 A 继续执行，计算  $p = 1/(t1 - t2) = 1/(4 - 5) = -1$ ;
4. 若 B 更改的值导致  $t1 = t2$ ，则可能出现除以零错误。

**结果分析** 即使事务 A 在读取每个变量时验证过版本号，也仍无法检测出中间不一致状态，因为它没有保证两次读取的值来自同一时刻。

$\Rightarrow$  事务 A 读取了不一致的快照。

这正是“僵尸事务”的第二种表现：事务使用了部分旧值、部分新值，违反了系统应维护的全局一致性约束。

## 5.5 根本问题与挑战 (Underlying Problem)

**核心问题** STM 缺乏一个全局时间概念 (global notion of time)。因此事务无法判断：

“我读到的这些变量，是否来自同一时刻的一致快照？”

**结果**

- 即便每次读取都验证版本号；
- 事务仍可能读取到“跨时刻”的数据；
- 从而形成逻辑层面的僵尸事务。

**影响**

- 导致严重的不一致；
- 破坏事务原子性；
- 在部分 STM 实现中甚至会触发系统崩溃。

**下一步改进方向** 为了解决这个问题，系统需要一个机制确保：

每个事务只能读取同一“时间点”的一致快照。

这引出了后续章节的核心改进——全局时钟 (Global Clock) 机制。

# 6 全局时钟机制与完整 STM 实现 (Global Clock and Full STM Implementation)

## 6.1 全局时钟思想 (Idea of the Global Clock)

**动机** 为了解决僵尸事务问题，我们需要确保：

事务在执行期间所读取的所有数据，必须来自同一时间点的一致快照。

**核心思想** 引入一个全局单调递增的时间戳 (Global Clock)，并为每个变量维护版本号 (counter)。其关键规则如下：

- 每个变量的版本号  $v$  必须小于当前全局时钟  $G$ ；
- 每个事务在开始时记录一个本地时钟  $t_{local} = G$ ；
- 事务执行期间，所有被读取的变量版本必须  $\leq t_{local}$ ；
- 提交时，写入的变量版本更新为当前全局时钟值；
- 每次提交成功后，系统全局时钟  $G$  自增。

**保证** 通过这种机制，所有事务的执行在逻辑上形成一个全序 (total order)，即每个事务都看起来像在某个时间点“瞬间执行”。

## 6.2 完整算法流程 (Full STM Algorithm)

一个完整的软件事务内存 (STM) 实现包括以下阶段:

### 1. 事务开始 (Begin Transaction)

```
local_clock = global_clock;
read_set = {};
write_set = {};
```

### 2. 读操作 (Read Operation)

1. 读取变量值;
2. 检查该变量版本号是否  $\geq$  local\_clock;
3. 若是, 说明变量在事务开始后被修改  $\rightarrow$  冲突, 中止;
4. 否则, 将该变量加入读集。

### 3. 写操作 (Write Operation)

1. 将修改写入本地写集 (write set);
2. 不立即更新主存;
3. 延迟到提交阶段统一处理。

### 4. 提交阶段 (Commit Phase)

1. 对所有读集变量再次检查版本号;
2. 若任一变量版本  $\geq$  local\_clock  $\rightarrow$  冲突, 中止;
3. 否则:
  - 将写集中的值写回主存;
  - 为这些变量赋予新的版本号 = 当前 global\_clock;
  - 最后 global\_clock 自增;

### 5. 中止 (Abort)

- 丢弃本地写集;
- 等待随机退避 (backoff) 后重启事务。

## 6.3 示例: 带全局时钟的冲突检测 (Example with Global Clock)

示例代码

```
atomic {
 t1 = x;
 t2 = y;
 p = 1 / (t1 - t2);
}
```

```
atomic {
 x = 217;
 y = 4;
}
```

#### 初始状态

$$x = 4, \quad y = 5, \quad \text{global\_clock} = 90$$

#### 事务 A (读者)

- 读取  $x = 4, y = 5$ ;
- 记录  $\text{local\_clock} = 90$ ;
- $\text{read\_set} = \{x, y\}$ 。

#### 事务 B (写者)

- 修改  $x = 217, y = 4$ ;
- 提交成功;
- 更新计数器  $x.\text{counter} = 100, y.\text{counter} = 100$ ;
- $\text{global\_clock} = 101$ 。

#### 事务 A 提交时检查

- 重新读取读集中变量的计数器;
- 检测到  $x.\text{counter} = 100 \geq 90$ ;
- 冲突! 中止事务 A;
- 重新执行。

**结果** 事务 A 被正确地中止, 从而避免了僵尸事务和除以零等错误。通过全局时钟, 系统保证:

所有事务的读集均来自同一一致快照。

### 6.4 全局时钟 STM 的关键步骤总结 (Key Properties)

- **Begin:** 保存当前全局时钟;
- **Read:** 检查变量版本号, 确保不超过本地时间;
- **Write:** 写入本地缓存;
- **Commit:** 验证 + 写回 + 全局时钟自增。

该机制确保每个事务在全序关系下“可序列化执行”(serializable)。

## 6.5 内存结构设计 (Memory Layout)

在实际实现中，内存通常组织为以下结构：

- 内存中的每个变量为一个结构体：  
(value, counter)
- 提交时以原子方式更新指针（例如通过 CAS 操作）；
- 提交后旧版本的结构体仍可能被其他线程读取；
- 因此需要垃圾回收（Garbage Collector, GC）回收过期版本。

## 6.6 完整实现伪结构 (Full Implementation Structure)

以下为简化的类结构（或对应的 C 结构体）：

```
class Value {
 int value;
 int counter;
}

class Memory {
 static Value values[];
 static int clock;
}

class TX {
 Set<int> readSet;
 Map<int, int> writeSet;
 int clock;
}
```

### 事务开始

```
void TX.begin() {
 clock = Memory.clock;
 readSet = new HashSet();
 writeSet = new HashMap();
}
```

### 写操作

```
void TX.write(int idx, int value) {
 writeSet.put(idx, value);
}
```

### 读操作

```
int TX.read(int idx) {
 if (writeSet.contains(idx))
 return writeSet.get(idx);
}
```

```

Value value = Memory.values[idx];
if (value.counter >= clock) abort();
readSet.add(idx);
return value.value;
}

```

### 提交

```

void TX.commit() {
 synchronized(Memory.values) {
 for (int idx : readSet)
 if (Memory.values[idx].counter >= clock) abort();

 for (entry : writeSet) {
 Value v = new Value(entry.value, Memory.clock);
 Memory.values[entry.key] = v;
 }
 Memory.clock++;
 }
}

```

### 机制总结

- 事务开始时记录全局时钟；
- 读操作检测变量版本；
- 提交时验证读集；
- 更新写集并推进全局时钟；
- 若检测到冲突则中止并重试。

## 7 退避策略、I/O 事务问题与性能总结 (Backoff, I/O, and Performance Summary)

### 7.1 事务冲突与退避策略 (Transactional Conflicts and Backoff)

**问题** 在 STM 系统中，多个事务可能频繁冲突并互相中止，例如：

- 事务 T1 修改了事务 T2 读取的变量；
- 事务 T2 又修改了事务 T1 的变量；
- 双方不断重启，形成 **活锁 (livelock)**。

**解决方法：指数退避 (Exponential Backoff)** 为了避免持续冲突，每当事务中止时，引入随机延时再重启。

```

int backoff(int n) {
 Thread.sleep(1 + (int)(n * Math.random()));
 return n < 512 ? n << 1 : n;
}

```

```

}

void doTransaction() {
 int n = 16;
 try {
 tx.begin();
 ... // 执行事务逻辑
 tx.commit();
 } catch (TXAbort e) {
 n = backoff(n);
 doTransaction();
 }
}
}

```

### 机制说明

- 初始等待时间为  $n = 16$ ;
- 每次中止后，随机等待  $[1, n]$  毫秒;
- 每次失败后将等待时间翻倍（指数增长），上限 512;
- 显著降低线程同时重启导致的冲突概率。

**总结** 退避 (backoff) 不能消除冲突，但能**减少活锁的出现**并显著提升性能。

## 7.2 事务与输入/输出操作 (Transactions and I/O Operations)

### 示例

```

atomic {
 if (x > 42)
 launchMissile();
}

```

**问题** 如果事务在执行 `launchMissile()` 后被中止 (abort)，系统无法“撤销”发射导弹这一外部动作。这是事务内存与外部世界交互时最危险的情况之一。

### 原因

- STM 的回滚机制仅能恢复内存状态;
- I/O 操作具有**不可逆性** (side effects);
- 因此事务不能安全地包含任何实际 I/O。

### 常见解决方案

1. **推迟执行 I/O (Deferred I/O)**: 将 I/O 操作加入队列，待事务成功提交后再执行;
2. **标记事务为“不可中止” (Unabortable)**: 在即将执行 I/O 时禁止该事务被回滚;
3. **事务外部化 I/O (Externalization)**: 将 I/O 操作移出事务，由独立线程在确认后执行。

**结论** I/O 操作必须以特别方式处理，否则会破坏 STM 的原子性与一致性。许多 STM 实现（如 Haskell STM）甚至明确禁止事务中出现任何 I/O。

### 7.3 性能分析与实验结果 (Performance and Evaluation)

**实验背景** 作者测试了以下场景：

100 个线程并发递增同一个共享计数器，每个线程执行 10,000 次递增操作。

**实验结果**

执行模式	时间 (秒)
STM (无退避)	3.00
STM (带退避)	0.48
锁 (单锁实现)	0.19

**结果分析**

- STM 带退避显著优于不带退避版本；
- 但传统锁机制仍然最快；
- 表明当前 STM 实现仍存在显著的性能开销。

### 7.4 总结与展望 (Summary and Outlook)

**事务内存的优势**

- 编程模型简洁，无需显式加锁；
- 无死锁 (deadlock-free)；
- 无饥饿 (no starvation)；
- 支持高层抽象：
  - 可组合性 (Composability)；
  - `retry / orElse`；
  - 自动回滚与重试；

**事务内存的挑战**

- 性能仍落后于细粒度锁；
- 对 I/O 操作支持不完善；
- 硬件事务 (HTM) 受缓存大小限制；
- 混合事务 (HyTM) 在硬件与软件间切换代价高。

**总体结论**

事务内存 (TM) 为并发编程提供了优雅、简洁且安全的抽象。它极大简化了开发者在锁、条件变量与死锁管理上的负担。然而，在性能敏感场景下，传统锁机制仍然更具优势。

**一句话总结**

“事务内存简化并发逻辑，但仍以性能为代价。”

# 1 第 1–10 页：多核与性能趋势、动机

## 1.1 课程概览

多核编程：非统一内存架构 (NUMA)。课程背景为巴黎综合理工计算机硕士与巴黎萨克雷高性能系统硕士课程，主讲为 Gaël Thomas。

## 1.2 我们为什么需要计算能力 (第 2 页)

- 分析大规模数据集；
- 执行复杂的计算；
- 同时服务大量客户端。

## 1.3 计算能力的核心在 CPU (第 3 页)

现代计算性能提升长期依赖于 CPU 的演进。

## 1.4 (旧的) 性能趋势与摩尔定律 (第 4–6 页)

晶体管数量快速增长 (摩尔定律)，而主频继续提高会导致功耗急剧增加。幸运的是，晶体管仍在增长，但我们需要新的提升路径。

## 1.5 当今：靠增加核心数提升性能 (第 7 页)

单核主频提升受限，通过增加核心数来拓展总算力成为主流。

## 1.6 多核编程的挑战 (第 8–10 页)

- 并行化并不等于自动加速；
- 并行算法需要可扩展性；
- 仍需面对复杂的内存架构 (为后续 NUMA 铺垫)。

### 示例代码 (第 8 页)

Listing 1: 在不同机器上的运行时间对比

```
1 #include <stdlib.h>
2 #define N 100000000
3 int main(int argc, char **argv) {
4 int* a = malloc(sizeof(int) * N);
5 for(int i=1; i<N; i++) {
6 a[i] = a[i] * a[i-1];
7 }
8 }
```

## 2 第 11–20 页：NUMA 基础与内存延迟

### 2.1 内存访问延迟差异（第 12 页）

在一台 48 核 / 4 插槽、128GB 内存的 AMD 机器上进行基准测试，结果显示核心访问不同节点的内存延迟差异极大：

- 核心 0 分配、核心 0 访问：约 5 个周期；
- 核心 3 分配、核心 0 访问：约 50 个周期；
- 核心 15 分配、核心 0 访问：约 275 个周期；
- 核心 20 分配、核心 0 访问：约 380 个周期。

说明内存访问延迟与核心位置及内存分配节点强相关。

### 2.2 缓存层次与延迟（第 13 页）

缓存层次结构示意：

缓存层级	典型访问延迟（周期）
L1 缓存	~5
L2 缓存	~15
L3 缓存	~50
主内存	数百周期

### 2.3 单总线难以扩展与复杂架构（第 14–18 页）

当系统规模增大时，单一总线（bus）无法支撑所有核心的并发访问，导致性能瓶颈。因此，现代服务器采用复杂的多节点互联架构。

以 Supermicro X11QPH+ 为例：

- 4× Intel Xeon GOLD 6130（每颗 16 核 / 32 线程）；
- 总计 64 核 / 128 超线程；
- 内存 256GB；
- 性能约为笔记本的 32 倍。

在 NUMA 架构中，每个节点有自己的本地内存，不同节点之间通过互联总线（interconnect）通信。

#### 访问延迟示例（48 核 AMD）

访问类型	跳数	延迟（周期）
本地访问	0-hop	155
一次远程访问	1-hop	275
两次远程访问	2-hop	380

延迟差异可达约 2.5×。

## 2.4 节点过载的情况（第 19 页）

当所有核心同时访问同一节点（即便访问不同缓存行）：

- 平均访问延迟可达  $\sim 870$  个周期；
- 相当于本地访问的 6 倍；
- 主要原因是内存控制器与互联链路的过载。

## 2.5 需要内存放置策略（第 20 页）

在 NUMA 架构中，我们必须制定内存放置策略，以实现：

- 避免单个 NUMA 域过载；
- 减少互联链路拥塞；
- 增强内存访问的局部性 (locality)。

目标是让线程优先访问所在节点的本地内存，以获得最低访问延迟。

# 3 第 21–30 页：NUMA 放置策略与页表

## 3.1 制定 NUMA 放置策略（第 21 页）

NUMA 放置策略的第一步是选择数据的物理地址。整个物理地址空间被分割到不同的 NUMA 节点上：

```
节点 0: 0 —— n-1
节点 1: n —— 2n-1
节点 2: 2n —— 3n-1
节点 3: 3n —— 4n-1
...
```

每个节点负责自己的一段物理地址区间。

## 3.2 利用页表机制实现放置（第 22 页）

每个进程都有独立的虚拟地址空间，通过页表 (page table) 映射到不同节点的物理页：

虚拟地址空间  $\rightarrow$  页表  $\rightarrow$  物理地址空间 (Node 0, 1, 2)

因此可以通过修改页表来控制每页的物理位置。

## 3.3 示例：保留虚拟地址空间（第 23 页）

Listing 2: mmap 分配虚拟地址空间

```
1 struct x* x = mmap(0, sizeof(*x), ...);
```

此时只是预留虚拟空间，还未绑定物理页。

### 3.4 示例: 绑定到指定节点 (第 24 页)

Listing 3: mbind 绑定页面到特定 NUMA 节点

```
1 struct x* x = mmap(0, sizeof(*x), ...);
2 mbind(x, sizeof(*x), 2); // 从节点 2 分配
```

绑定后, 该虚拟地址对应的物理页来自 Node 2 的内存。

### 3.5 主要研究问题 (第 25 页)

1. NUMA 效应在实践中是否显著?
2. 如果显著, 我们能否通过策略来降低其影响?

### 3.6 理论分析 (第 26–27 页)

**Cache 不友好的应用:** 假设 50% 指令访问内存, 命中比例 L1/L2/L3 各为 30%。若访问完全局部: ~32 周期/指令; 若全部跨节点: ~156 周期/指令。⇒ 性能下降 385%。

**Cache 友好的应用:** 命中比例 L1/L2/L3 各为 70%。局部访问: ~7 周期/指令; 跨节点访问: ~17 周期/指令。⇒ 性能下降约 137%。

### 3.7 第一项研究: NUMA 管理行为 (第 28 页)

目标:

- 理解 Linux 如何处理 NUMA;
- 观察应用在不同 NUMA 策略下的性能。

研究对象: 来自 NPB、Parsec、Mosbench、X-stream、YCSB 的 29 个应用。

### 3.8 手工调优策略 (第 29–30 页)

通过手动将不同虚拟内存区段绑定到不同节点:

虚拟地址空间

段 1 → Node 0

段 2 → Node 2

**优点:** 精细化控制性能。**缺点:** 工程量巨大, 不具通用性, 仅对特定硬件/应用有效。

## 4 第 31–40 页: Linux 策略与 Carrefour

### 4.1 Linux 中的手工调优 (第 31 页)

在 Linux 中可通过以下系统调用手动控制线程和内存的放置:

- `setaffinity(core set)`: 为整个进程设置可运行核心集合;
- `pthread_setaffinity(core set)`: 为单个线程设置绑定核心;
- `mbind(virtual range, node set)`: 将地址范围绑定到一组节点;

页粒度通常为 4 KiB, 从而精确控制线程与数据的物理位置。

## 4.2 交错策略 Interleaved (第 32–33 页)

**机制:** 按节点轮流分配页面 (Round-robin)。

虚拟地址空间 → 页表 → Node0, Node1, Node2, ...

**优缺点:**

- 优点: 均衡所有节点的负载;
- 缺点: 产生大量远程访问, 容易使互联链路饱和;

→ 平衡负载但牺牲局部性。

## 4.3 首次触摸策略 First-touch (第 34–37 页)

**机制:** 依赖 Linux 的惰性映射。由第一个访问该页面的线程决定页面所在的物理节点。

Listing 4: 首次触摸示意

**示例:**

```
1 struct x* x = mmap(0, sizeof(*x), ...);
2 x->a = 42; // 第一次访问触发在当前节点分配页
```

**过程:** 首次访问时分配物理页, 并更新页表:

线程在 Node1 上 → 访问虚拟页 → 系统在 Node1 分配物理页

**优缺点:**

- 优点: 若线程只访问自己分配的内存, 局部性完美;
- 缺点: 若由其他线程分配, 则导致热点与过载;

## 4.4 Carrefour 策略 (第 38–40 页)

**来源:** Dashti 等人, ASPLOS 2015。

**目标:** 动态平衡各节点负载, 防止互联链路 (interconnect) 拥塞。

**机制:** Carrefour 监控节点与链路状态, 按需执行页面迁移:

- 若节点过载 → 将部分页迁移至未过载节点;
- 若链路拥塞 → 将页迁移到最常访问它的节点;

**优缺点:**

- 优点: 自动适应访问热点, 减少争用;
- 缺点: 若应用访问模式频繁变化, 会产生无效迁移;

**实验策略组合：** 研究中比较了 4 种组合（均为 4 KiB 页）：

1. First-touch;
2. First-touch + Carrefour;
3. Interleaved;
4. Interleaved + Carrefour。

这些组合在后续章节中被评估与对比。

## 5 第 41–50 页：NUMA 策略评估与预测

### 5.1 NUMA 策略的性能评估（第 41 页）

以 **Linux First-touch (FT)** 为基准，比较四种策略组合的相对加速比。图表显示：

- “4K win” → 交错策略 (Interleaved) 最优；
- “4K/Carrefour win” → 交错 + Carrefour；
- “FT win” → 首次触摸最优；
- “FT/Carrefour win” → 首次触摸 + Carrefour；

结果发表于 EuroSys 2017。

### 5.2 总体结论（第 42 页）

**第一结论：**所有 NUMA 策略都重要。不同应用在不同策略下获得最佳性能，因此不存在“通用最优解”。

### 5.3 第二项研究：策略预测（第 43 页）

目标：

- 预测应用最适合的 NUMA 策略；
- 自动选择最优策略；
- 理解应用访问分布与不平衡程度。

### 5.4 如何预测（第 44 页）

测量在 First-touch 下的内存访问**不平衡度**：

- 定义：各节点访问次数的相对标准差；
- 完全均衡 (8 节点)：0%；
- 完全集中 (1 节点)：283%；

## 5.5 不平衡度等级（第 45 页）

不平衡度	活跃节点数	分类
0%	8	完美平衡
40%	7	轻度不平衡
83%	5	中度不平衡
185%	2	严重不平衡
283%	1	极端集中

## 5.6 低不平衡：First-touch 即最优（第 46–47 页）

当不平衡度低（0–40%）时，First-touch 已具良好局部性与均衡性。**推荐策略：**继续使用 First-touch。性能仅比最优策略慢约 1%。

## 5.7 中等不平衡：适合 Carrefour（第 48–49 页）

访问较均衡但局部性仍不完美时，可让 Carrefour 动态迁移优化。**推荐策略：**First-touch / Carrefour。性能仅比最优慢约 2%。

## 5.8 高不平衡：适合 Interleaved / Carrefour（第 50 页）

访问集中在少数节点，First-touch 无法平衡。交错策略能分散负载，Carrefour 进一步提升局部性。**推荐策略：**Interleaved / Carrefour。性能仅比最优慢约 2%。

# 6 第 51–60 页：预测结论与 Spark 案例

## 6.1 策略预测总结（第 51–52 页）

通过测量在 First-touch 策略下的访问不平衡度，可以合理预测最适合的 NUMA 策略。该方法能自动选择最优配置，同时避免人工调试的复杂性。

## 6.2 第三项研究：数据分析应用 Spark（第 53 页）

研究对象：分布式内存分析系统 Spark。测试应用：**PageRank**（数据集：Friendster）。配置：

- JVM 堆大小 40GB；
- 使用 Parallel Scavenge (PS) 垃圾回收器；
- 评估不同核心数下的可扩展性。

## 6.3 可扩展性问题（第 54 页）

结果：

- 理想情况下，核心数  $\times 2 \rightarrow$  性能应  $\times 2$ ；
- 实际上，当核心数超过 12 时，性能停止增长甚至下降；
- 说明 Spark 在 12 核以上出现明显可扩展性瓶颈。

## 6.4 垃圾回收器瓶颈（第 55–56 页）

时间分解分析：

- 应用执行时间随核心数减少（表现出并行加速）；
- GC 时间基本不变，成为主要瓶颈；
- Parallel Scavenge 吞吐量在 12 核后几乎不再提升。

结论：GC 是 Spark 在多核下的性能限制因素。

## 6.5 第一类优化：同步机制（第 57–58 页）

优化方向：

1. 移除冗余同步；
2. 使用 Linux futex 取代自旋锁；
3. 改进无锁队列（lock-free queue）；

结果：吞吐量提升，但未根本改善扩展性。参考：Gidra 等人，ASPLOS 2013。

## 6.6 第二类优化：NUMAGiC（第 59–60 页）

NUMAGiC 旨在使垃圾回收器具备 NUMA 感知能力。

问题：传统 GC 线程可访问任意节点的内存：

Node 0 内存

Node 1 内存

GC Thread 0 运行在 Node 0 → 访问 Node 1 内存 → 远程访问延迟高

思路：NUMAGiC 采用分布式设计：

- 每个节点的 GC 线程只访问本节点对象；
- 不直接访问远程内存；
- 若需跨节点操作，通过消息传递与其他 GC 协作。

核心原则：

“既然内存是分布式的，垃圾回收也应分布式化。”

每个节点只处理本地对象，GC 线程之间通过消息协作实现全局回收。

# 7 第 61–70 页：NUMAGiC 细节与消息优化

## 7.1 远程访问问题（第 61 页）

在传统 GC 中，一个 GC 线程可能跨节点访问任意内存区域：

Node 0 内存

Node 1 内存

GC Thread 0 运行在 Node 0 → 访问 Node 1 对象 → 远程访问

远程访问 = 性能杀手。因此必须减少跨节点访问。

## 7.2 NUMAGiC 的理念 (第 62 页)

核心思想：

“分布式内存 → 分布式垃圾回收设计。”

让每个节点有自己的 GC 线程，仅访问本地内存。NUMAGiC 将集中式 GC 改造成节点内协作、节点间通信的模型。

## 7.3 工作方式 (第 63–64 页)

机制：

- 将远程内存访问替换为**消息传递**；
- 每个节点的 GC 线程仅扫描本地对象；
- 若需访问其他节点对象，则发送消息请求该节点的 GC。

示意：

```
Node 0 GC0
 |
 | 消息传递
Node 1 GC1
```

## 7.4 分布式通信模型 (第 65–66 页)

所有 GC 线程形成一个分布式网络：

```
Node 0 Node 1 Node 2 Node 3
```

```
GC0 GC1 GC2 GC3
```

每个线程仅操作本地内存，通过消息协作完成全局回收。

## 7.5 优点与问题 (第 67–68 页)

优点：

- 极大减少远程访问；
- 更贴合 NUMA 架构；
- 分担节点间负载；

问题：

- 消息发送本身也有代价；
- 若跨节点引用多，消息数量暴涨，性能反而下降。

## 7.6 性能下降与原因 (第 69 页)

实验发现：

- 当跨节点引用过多时，消息量爆炸；
- 发送消息比远程访问还昂贵；

**结论：**必须减少跨节点引用 (inter-node references)。

## 7.7 减少跨节点引用的策略（第 70 页）

**关键观察：**一个线程通常连接的是它自己分配的对象 → 引用多为局部。  
在 Spark 中：

仅约 1% 的引用发生在由不同线程分配的对象之间。

**启发式策略：**

- 对象在分配后应保留在分配节点；
- 避免跨节点引用；
- 减少跨节点通信量，从而减少消息数量；

示意：

Node 0 对象群 A

Node 1 对象群 B

→ 对象主要在同节点内相互引用

## 8 第 71–80 页：NUMAGiC 自适应算法与虚拟化问题

### 8.1 NUMAGiC 的局限：并行性不足（第 71 页）

即使减少了跨节点引用，仍可能出现性能下降：

- 消息变少，但系统的并行性也随之降低；
- 某些节点在 GC 时忙碌，其他节点则闲置；

示意：

Node 0 —— 正在 GC

Node 1 —— 空闲等待

**问题：**少消息 高并行。

### 8.2 自适应算法的引入（第 72 页）

NUMAGiC 通过引入自适应机制在“局部性”与“并行性”之间动态权衡。

**思想：**

- 当节点有工作可做 → 保持本地模式（Local Mode）；
- 当节点空闲 → 切换为“盗取模式”（Thief Mode），帮助其他节点；

示意：

Node 0 回收内存 → Node 1 空闲 → Node 1 偷取 Node 0 的工作

### 8.3 两种模式（第 73 页）

模式	条件	行为
本地模式 (Local)	有本地任务	仅处理本节点对象
盗取模式 (Thief)	节点空闲	从远程节点“偷取”任务帮助 GC

**效果：**兼顾局部性（减少远程访问）与并行性（提高 CPU 利用率）。

## 8.4 协同回收示意 (第 74 页)

Node 0 —— 执行 GC

Node 1 —— 切换至 Thief 模式 → 访问 Node 0 对象 → 协助回收

节点间协同缩短整体回收时间。

## 8.5 GC 吞吐量结果 (第 75 页)

实验环境: Spark, 堆 40GB。

横轴: 核心数; 纵轴: GC 吞吐量 (GB/s)。比较方案:

1. PS (原生 Parallel Scavenge);
2. SynchroPS (同步优化版);
3. SynchroPS + Interleaved;
4. NUMAGiC;

结果: NUMAGiC 吞吐量最高, 并呈线性扩展趋势。

## 8.6 对应用性能的影响 (第 76 页)

指标: 任务完成时间。

策略	相对性能
PS (默认)	最慢
SynchroPS	略快
SynchroPS + Interleaved	明显改善
NUMAGiC	最快, 任务完成时间减半

来源: Gidra 等人, ASPLOS 2015。

## 8.7 第三项研究结论 (第 77 页)

1. NUMA 对性能有显著影响;
2. 可以通过新策略 (如 NUMAGiC) 显著提升 JVM 应用性能;
3. 将分布式系统思维引入 NUMA 管理是有效的方向。

## 8.8 第四项研究: 虚拟化环境 (第 78 页)

问题: NUMA 架构下, 虚拟机管理器 (hypervisor) 的性能表现如何?

实验:

- 29 个应用: Parsec、NPB、MosBench、X-stream、YCSB;
- 观察虚拟化层在多核下的开销;

## 8.9 虚拟化开销结果 (第 79 页)

平台: 48 核服务器 + Xen hypervisor。

- 1 核时几乎无开销;
- 48 核时性能下降高达 9.5 倍;

结论: NUMA 无感知的虚拟化层是主要瓶颈。

## 8.10 性能下降根源（第 80 页）

Xen 未考虑 NUMA 拓扑：

- vCPU 访问远程内存；
- 多虚拟机争用同一节点；
- 内存控制器与互联链路拥塞；

**结论：**虚拟化中的内存访问延迟是性能下降的核心原因。

## 9 第 81–90 页：XenNUMA、vNUMA 与 XPV + 总结

### 9.1 XenNUMA 的设计（第 81 页）

**目标：**在 Xen 虚拟机管理器中实现通用的 NUMA 策略。**功能：**

- 支持三种策略：Interleaved、First-touch、Carrefour；
- 在 Xen 与 Linux 之间新增接口：
  - Linux 可选择每个进程的 NUMA 策略；
  - Xen 能获知进程页的物理位置；
  - Xen 能在首次访问时从正确节点分配页；
- 重写 Xen 内存子系统以支持 NUMA 放置。

### 9.2 XenNUMA 性能（第 82 页）

实验环境：

- 48 vCPU 对应 48 物理核心；
- 比较标准 Xen 与 XenNUMA；

**结果：**

- 性能提升最高可达 700%；
- 12/29 应用虚拟化开销低于 50%，23/29 应用整体虚拟化开销下降；

**结论：**NUMA 感知虚拟化可显著提升性能。**来源：** Voron 等人，EuroSys 2017。

### 9.3 XenNUMA 的局限（第 83 页）

虽然性能改善明显，但仍存在问题：

- XenNUMA 隐藏硬件拓扑，导致：
  1. 应用无法使用高级 NUMA 优化（如 NUMAGiC）；
  2. 无法利用 NUMA 感知内存分配器（如 TCMalloc、JEMalloc）；

**结论：**隐藏拓扑使上层系统优化受限。

## 9.4 vNUMA: 暴露拓扑 (第 84–85 页)

vNUMA 尝试向虚拟机暴露 NUMA 拓扑:

- 起初暴露正确拓扑;
- 但 Hypervisor 为平衡负载会迁移 vCPU;
- vNUMA 不会更新拓扑信息;
- OS/运行库基于过期拓扑做优化 → 反而性能下降。

结论: 静态拓扑暴露同样不理想。

## 9.5 XPV: 扩展半虚拟化 (第 86 页)

XPV (eXtended ParaVirtualization) 结合了 XenNUMA 与 vNUMA 的优点:

- 向客户 OS 暴露真实 NUMA 拓扑;
- 当拓扑变化 (例如 vCPU 迁移) 时, 发送通知;
- OS/运行库据此更新策略;

仅需在 Xen 与客户系统中添加少量代码即可实现。

## 9.6 XPV 与 XenNUMA 性能比较 (第 87 页)

在固定拓扑下, XPV 比 XenNUMA 性能提升最高达 **130%**。原因: XPV 暴露拓扑, 使 OS 与运行库能自主优化访问局部性。

## 9.7 XPV 在动态拓扑下的效果 (第 88 页)

实验: 当 Xen 动态迁移 vCPU 以平衡负载时, XPV 能根据通知及时更新策略。结果: 性能提升最高达 **127%** (相较无通知机制)。来源: Bui 等人, EuroSys 2019。

## 9.8 总结与启示 (第 89 页)

- NUMA 对性能的影响在各层次都显著;
- 使用通用策略 (First-touch, Interleaved, Carrefour) 即可显著提升性能;
- 可通过访问不平衡度预测最优策略;
- 特定运行时 (JVM、数据库、锁库) 需定制方案 (NUMAGiC 等);
- 虚拟化层也可通过 NUMA 感知 (XenNUMA/XPV) 减轻影响。

## 9.9 结语 (第 90 页)

本课程总结:

- NUMA 是现代多核系统必须正视的现实;
- 只有应用、操作系统与虚拟化层协同感知并优化内存拓扑, 才能释放硬件全部潜能;
- NUMA 管理是连接体系结构与高性能编程的重要桥梁。

——课程结束——

主讲: Gaël Thomas 翻译整理: 小法的 Overleaf 中文版

# 非易失性主存 (NVMM)

## 定义

- 字节可寻址：处理器可通过内存总线直接使用普通的加载/存储指令访问；
- 持久性与 SSD 相同；
- 性能介于 DDR4 与 NVMe (通过 PCIe 的 SSD) 之间。

## 对比

传统易失性内存 (DDR4) + 持久存储 (Flash) = 非易失性主存 (NVMM)

# NVMM 性能

## 延迟 (Latency)

- 比 DDR4 慢约 2 倍；
- 比 NVMe 快约 1000 至 10000 倍。

## 吞吐量 (Throughput)

- 比 DDR4 低 2-4 倍；
- 比 NVMe 高 4-10 倍。

类型	读延迟	写延迟	顺序带宽	随机读	随机写	带宽
DDR4	80ns	60ns	~100Gbps	100ns	60ns	~100Gbps
Optane DC	160ns	60ns	40Gbps	300ns	120ns	14Gbps
NVMe	250µs	250µs	3Gbps	250µs	250µs	3Gbps

# 在硬件与系统中的使用

## 硬件层面

- BIOS 将 NVMM 识别为普通内存；
- 标记为 “非易失”。

## 系统层面 (Linux)

- 显示为一个设备；
- 使用 直接访问文件系统 (DAX) 格式化；
- 绕过 I/O 缓存，允许直接 load/store；
- 可通过常规 I/O 函数 (read, write, mmap) 访问；
- 当使用 mmap 时可直接访问 NVMM。

## Linux 下 NVMM 使用示例

### 文件系统对比

- 直接访问文件系统 (ext4-dax): 绕过内核页缓存;
- 非直接访问文件系统 (ext4): 使用页缓存。

```
/dev/pmem0 -> 挂载到 /mnt/nvmm
/dev/sdb -> 挂载到 /mnt/disk

int fd = open("/mnt/[nvmm,disk]/myfile", O_RDWR | O_CREAT);
ftruncate(fd, 1024*1024);
char* addr = mmap(NULL, 1024*1024, PROT_READ | PROT_WRITE,
 MAP_SHARED_VALIDATE | MAP_SYNC, fd, 0);
addr[0] = 'a';
```

在 NVMM 上为直接写入; 在普通磁盘上则写入页缓存, 之后再同步到磁盘。

### 崩溃管理 (Crash Management)

- 崩溃可能随时发生;
- NVMM 的状态仍然存在;
- 但状态未必一致;
- 程序需在恢复时清理状态。

```
struct id {
 char name[256];
};

struct id* id = mmap(...);
strcpy(id->name, "Pikachu");
```

若在 strcpy 期间崩溃, 则 id->name 可能处于不一致状态。

### 解决方案: 使用事务 (Transactions)

```
struct id {
 char name[256];
 bool committed;
};

struct id* id = mmap(...);

if(!id->isCommitted) { /* recover */
 strcpy(id->name, "Pikachu");
 id->committed = true;
}
```

## 乱序执行 (Out-of-order Execution)

现代处理器可能乱序执行指令：`id->committed` 的写入可能先于 `strcpy`。

为保证顺序，需要使用内存屏障（memory fence）。但普通内存屏障仅保证缓存中的顺序，缓存行仍可能乱序刷新。

```
memory_fence();
```

## 防止乱序执行的新指令

- **pwb(addr)**: 将包含 `addr` 的缓存行加入刷新队列（FIFO 顺序）；
- **pfence()**: 确保之前的 store 与 pwb 在之后的 store/pwb 之前执行。

```
if(!id->isCommitted) {
 strcpy(id->name, "Pikachu");
 pwb(id->name);
 pfence();
 id->committed = true;
 pwb(&id->committed);
}
```

这样：

- `id->name` 所在的缓存行先刷新；
- `id->committed` 后刷新；
- 恢复时若 `committed==true`，说明 `name` 已持久化。

## 更强的保证：持久化同步

- **pwb/pfence** 仅保证刷新顺序；
- 不保证缓存行真的写入 NVMM；
- 某些场景需要更强的持久性保证；
- 例如：**耐久线性化 (durable linearizability)**。

引入第三条指令：

- **psync()**: 行为类似 `pfence()`，并确保缓存行真正写入 NVMM。

```
strcpy(id->name, "Pikachu");
pwb(id->name);
pfence();
id->committed = true;
pwb(&id->committed);
psync();
atomic_store(&visible, true);

while(!atomic_load(&visible)) { }
printf("%s is durable\n", id->name);
```

## 在 Pentium 上的实现

- **pwb** 使用 **clwb** (Cache Line Write Back) 实现；
- **pfence** 使用 **sfence** (Store Fence) 实现；
- **psync** 也使用 **sfence**；
- Pentium 采用 TSO (总存储顺序) 模型；
- 崩溃时 CPU 残余能量足以将缓存行刷新至 NVMM。

## 总结

- NVMM 提供接近易失内存成本的持久性；
- 在 Linux 中可通过 **ext4-dax** 文件系统直接访问；
- 三条关键指令：
  - **pwb**: 将缓存行加入刷新队列；
  - **pfence**: 防止 store/pwb 重排；
  - **psync**: 确保数据写入 NVMM。

## 1 线程与同步 (Threads and Synchronizations)

### 1.1 实验一：我的第一个线程 (My first threads)

#### 实验目标

使用 `pthread_create` 创建多个线程，并观察：

1. 为什么有时候主线程在所有子线程输出之前就退出？
2. 为什么创建很多线程时，有时输出的 “Hello world” 数量少于线程数？
3. 使用 `pthread_exit` 之后，行为有何变化？
4. 如何保证主线程在所有子线程输出完毕之后再打印 “Main quitting”？

#### 实验要求

1. 编写一个 C 程序，程序第一个命令行参数为线程数 N，使用 `pthread_create` 创建 N 个线程。
2. 每个线程向终端输出：

[tid] Hello, World i!!!

其中：

- tid 为 `pthread_self` 返回的线程标识；
- i 为主线程对每个线程分配的递增编号（第一个线程为 0，第二个为 1，以此类推）。

3. 主函数结束时打印 Main quitting。
4. 多次以 3 个线程运行程序，观察并回答：为什么有时候会先打印 Main quitting 再打印所有的 “Hello world”？
5. 以 20 个线程多次运行程序，观察并回答：为什么经常打印的 “Hello world” 少于 20 次？
6. 在 main 的结尾增加一次 `pthread_exit` 调用，观察并回答：为什么此时以 20 个线程运行，总是能输出 20 次 “Hello world”？
7. 进一步修改程序，确保在所有 “Hello world” 输出之后，才打印 Main quitting。

#### 参考实现：hello.c

下面的代码是一个完整的示例，展示了如何创建 20 个线程，等待它们结束，然后再退出主线程。

Listing 1: hello.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4
5 #define N 20
6
7 void* hello(void* _arg) {
8 int* p = _arg;
```

```
9 printf("[%llx] Hello from %d\n", (unsigned long long)pthread_self(), *p);
10 free(p);
11 return NULL;
12 }
13
14 int main(int argc, char* argv[]) {
15 pthread_t tid[N];
16 void* retval;
17
18 for(int i=0; i<N; i++) {
19 int* p = malloc(sizeof(*p));
20 *p = i;
21 pthread_create(&tid[i], NULL, hello, p);
22 }
23
24 for(int i=0; i<N; i++)
25 pthread_join(tid[i], &retval);
26
27 printf("Quitting\n");
28
29 pthread_exit(NULL);
30 return 0;
31 }
```

### 现象解释 (简要思路)

- 如果主线程不 join 子线程，也不 pthread\_exit，主线程一旦返回 main，整个进程就结束，尚未完成输出的线程会被直接杀掉，因此会看到“Main quitting”在前，甚至有线程来不及打印。
- 当线程数较多时，调度、创建开销导致部分线程还没来得及运行或完成，进程就结束了，所以“Hello world”数量少于线程数。
- 使用 pthread\_exit 时，主线程只终止自己，而进程会继续，直到所有非分离 (non-detached) 的线程结束，从而保证所有线程打印完。
- 使用 pthread\_join 等待所有线程后，再打印 Quitting/Main quitting，即可保证主线程最后退出。

## 2 共享计数器 (A Shared Counter)

### 实验目标

在上一部分的多线程程序基础上：

1. 添加一个全局变量 counter，初始为 0。
2. 每个线程在打印自己的“Hello”之后，对 counter 自增。
3. 在主线程调用 pthread\_exit 之前，打印 counter 的值。
4. 当线程数为 100 时，观察并分析：为什么 counter 的值经常小于 100？

5. 修改程序，使用适当的同步原语（如互斥锁 `pthread_mutex_t`），保证 `counter` 的最终值等于线程数。
6. 进一步修改程序，只在 `counter` 等于线程数时才打印 `Main quitting`。

## 现象与原因（概述）

- 多个线程同时对共享变量 `counter` 执行读-改-写操作时，如果没有同步机制，会产生数据竞争 (data race)，导致丢失更新，最终结果小于理论值。
- 使用互斥锁保护对 `counter` 的访问，使每次自增成为原子（互斥）操作，可以保证最终结果正确。
- 可以用条件变量或忙等的方式，让主线程在 `counter == N` 时打印 “Main quitting”。

（此处你可以在 Overleaf 里自己增加一个带互斥锁的版本代码，比如 `hello_counter.c`。）

Listing 2: `countermutex.c`

```

1 //
2 // Created by liyif on 2025/11/5.
3 //
4 #include<stdio.h>
5 #include<pthread.h>
6 #include<stdlib.h>
7
8 #define N 10000
9 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
10 static int counter = 0;
11
12 void* worker(void* arg)
13 {
14 printf("hello world\n");
15 pthread_mutex_lock(&mutex);
16 counter++;
17 pthread_mutex_unlock(&mutex);
18 return NULL;
19 }
20
21 int main()
22 {
23 pthread_t pthread[N];
24 for(int i=0; i<N; i++)
25 {
26 pthread_create(&pthread[i], NULL, worker, NULL);
27 }
28 for(int i=0; i<N; i++)
29 {
30 pthread_join(pthread[i], NULL);
31 }
32 printf("Bye %d\n", counter);
33 pthread_exit(NULL);
34 }
```

### 3 生产者/消费者 (Producer/Consumer)

本实验实现一个生产者/消费者模型。主线程作为 **生产者**，从标准输入读取行并将其放入队列中。其他线程作为 **消费者**，从队列中按照 FIFO 顺序取出这些行并打印。

队列使用链表实现：

- 链表的头结点 (**head**) 是最早入队的元素 (first in)。
- 链表的尾结点 (**tail**) 是最新入队的元素 (last in)。

为了避免每次入队都从头遍历到尾，我们同时维护头指针和尾指针。

#### 3.1 单生产者单消费者队列 (Unicast Queue)

要求

- 只有一个消费者线程从队列中取消息。
- 当队列为空时，消费者线程需要调用 `pthread_cond_wait` 进入睡眠，等待生产者唤醒。
- 消费者取出并处理一个节点后，需要释放该节点的内存。

参考实现：spsc.c

Listing 3: spsc.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <pthread.h>
5 #include <stdint.h>
6
7 #define MAX_LENGTH 256
8 #define xstr(s) str(s)
9 #define str(s) #s
10
11 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
12 pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
13
14 struct node {
15 struct node* next;
16 char msg[MAX_LENGTH];
17 };
18
19 struct node* head = NULL;
20 struct node* tail = NULL;
21
22 void* consumer(void* _arg) {
23 int i = (int)(uintptr_t)_arg;
24
25 for(;;) {
26 pthread_mutex_lock(&mutex);
```

```
27 while(head == NULL)
28 pthread_cond_wait(&cond, &mutex);
29
30 struct node* n = head;
31 head = n->next;
32 if(!head)
33 tail = NULL;
34 pthread_mutex_unlock(&mutex);
35
36 printf("[%d] Receive: %s\n", i, n->msg);
37 free(n);
38 }
39
40 return 0;
41 }
42
43 void producer() {
44 for(;;) {
45 struct node* cur = malloc(sizeof(*cur));
46
47 scanf("%"xstr(MAX_LENGTH)"s", cur->msg);
48
49 if(strncmp(cur->msg, "exit", MAX_LENGTH) == 0)
50 exit(0);
51
52 cur->next = NULL;
53
54 pthread_mutex_lock(&mutex);
55 if(tail)
56 tail->next = cur;
57 else
58 head = cur;
59 tail = cur;
60 pthread_cond_signal(&cond);
61 pthread_mutex_unlock(&mutex);
62 }
63 }
64
65 int main(int argc, char** argv) {
66 if(argc != 2) {
67 fprintf(stderr, "Usage: %s nb-threads\n", argv[0]);
68 exit(1);
69 }
70
71 int n = atoi(argv[1]);
72 pthread_t tids[n];
73
74 for(int i=0; i<n; i++)
75 pthread_create(&tids[i], NULL, consumer, (void*)(uintptr_t)i);
```

```
76
77 producer();
78
79 for(int i=0; i<n; i++) {
80 void* retval;
81 pthread_join(tids[i], &retval);
82 }
83
84 printf("Main quitting\n");
85
86 return 0;
87 }
```

### 说明

- 生产者不断从标准输入读取字符串, 如果读到 `exit` 就直接 `exit(0)` 结束进程。
- 每次生产一个新的节点, 将其加到链表尾部, 然后使用 `pthread_cond_signal` 唤醒一个等待的消费者。
- 消费者在队列为空时使用 `pthread_cond_wait` 睡眠, 等待被唤醒后再从队列头部取出一个节点。

## 4 共享内存中的原子广播 (Atomic Broadcast in Shared Memory) (可选, 1/2)

### 多播队列 (Multicast Queue)

在这一部分, 我们希望实现一个 **多播队列**:

- 每个消费者必须接收并打印每一条消息。
- 消息仍然需要按照 FIFO 顺序被消费。
- 需要注意在所有消费者处理完某个节点之后再释放该节点。

## 5 共享内存中的原子广播 (Atomic Broadcast in Shared Memory) (可选, 2/2)

在实现广播之后, 我们进一步希望:

- 各个消费者可以完全异步地消费队列中的节点;
- 即: 某个消费者可以消费更靠后的节点, 而不必等待前面节点被所有其他消费者消费完;
- 从而实现消费者之间的完全解耦。

## 参考实现: mpmc.c

Listing 4: mpmc.c

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <pthread.h>
5 #include <stdint.h>
6 #include <stdbool.h>
7 #include <unistd.h>
8
9 #define MAX_LENGTH 256
10 #define xstr(s) str(s)
11 #define str(s) #s
12
13 struct node {
14 char msg[MAX_LENGTH];
15 struct node* next;
16 size_t cpt;
17 pthread_mutex_t mutex;
18 pthread_cond_t cond;
19 };
20
21 struct thread_arg {
22 struct node* fake;
23 size_t id;
24 };
25
26 struct node* new_node(size_t n) {
27 struct node* node = malloc(sizeof(*node));
28
29 node->next = NULL;
30 node->cpt = n;
31 pthread_mutex_init(&node->mutex, NULL);
32 pthread_cond_init(&node->cond, NULL);
33
34 return node;
35 }
36
37 struct thread_arg* new_arg(size_t id, struct node* fake) {
38 struct thread_arg* arg = malloc(sizeof(*arg));
39
40 arg->fake = fake;
41 arg->id = id;
42
43 return arg;
44 }
45
46 void* consumer(void* _arg) {

```

```

47 struct thread_arg* arg = _arg;
48 struct node* cons = arg->fake;
49 size_t id = arg->id;
50
51 free(arg);
52
53 for(;;) {
54 bool was_free = false;
55
56 pthread_mutex_lock(&cons->mutex);
57 while(cons->next == NULL)
58 pthread_cond_wait(&cons->cond, &cons->mutex);
59 char buf[MAX_LENGTH];
60 strncpy(buf, cons->msg, MAX_LENGTH);
61 struct node* tmp = cons->next;
62 if(--cons->cpt == 0) {
63 was_free = true;
64 free(cons);
65 }
66 pthread_mutex_unlock(&cons->mutex);
67
68 cons = tmp;
69
70 if(strncmp(buf, "exit", MAX_LENGTH) == 0) {
71 printf("==== [%zu] exiting =====\n", id);
72 pthread_exit(NULL);
73 } else {
74 printf("[%zu] receive: %s%s\n", id, buf, was_free ? " (free the node)" : "");
75 // simulate different speeds, consumer i is faster than consumer i+1
76 //usleep((id-1)*100000);
77 }
78 }
79 }
80
81 // here, the thread_arg is just here if we want to test with multiple producers
82 void producer(struct thread_arg* _arg) {
83 struct thread_arg* arg = _arg;
84 struct node* prod = arg->fake;
85 size_t id = arg->id;
86
87 free(arg);
88
89 char* poeme[] =
90 {
91 "Avec", "ses", "quatre", "dromadaires",
92 "Don", "Pedro", "d' Alfaroubeira",
93 "courut", "le", "monde", "et", "l' admira",
94 "il", "fit", "ce", "que", "je", "voudrais", "faire",
95 "si", "j' avais", "quatre", "dromadaires",

```

```
96 "exit"
97 };
98 size_t i = 0;
99
100 for(;;) {
101 //char msg[MAX_LENGTH];
102 //scanf("%xstr(MAX_LENGTH)"s", msg);
103 char* msg = poeme[i++];
104
105 struct node* next = new_node(prod->cpt);
106
107 pthread_mutex_lock(&prod->mutex);
108 printf("[%zu] produce: %s\n", id, msg);
109 prod->next = next;
110 strncpy(prod->msg, msg, MAX_LENGTH);
111 pthread_cond_broadcast(&prod->cond);
112 pthread_mutex_unlock(&prod->mutex);
113
114 if(strncmp(msg, "exit", MAX_LENGTH) == 0)
115 return;
116
117 prod = next;
118 }
119 }
120
121 int main(int argc, char** argv) {
122 if(argc != 2) {
123 fprintf(stderr, "Usage: %s nb-threads\n", argv[0]);
124 exit(1);
125 }
126
127 int n = atoi(argv[1]);
128 pthread_t tids[n];
129
130 struct node* list = new_node(n);
131
132 for(int i=0; i<n; i++)
133 pthread_create(tids + i, NULL, consumer, new_arg(i + 1, list)); // 1...n are the
134 consumers
135
136 producer(new_arg(0, list)); // 0 is the producer
137
138 for(int i=0; i<n; i++) {
139 void* retval;
140 pthread_join(tids[i], &retval);
141 }
142
143 printf("Main quitting\n");
144 }
```

## 1 实验目标

- 自旋锁 (spinlock)
- Ticket Lock
- MCS Lock
- POSIX 互斥锁 (pthread\_mutex)
- 基于 futex 的自制 POSIX 风格锁 (可选)

主要比较它们在不同参数 (线程数、临界区停留时间、计算区停留时间) 下的性能表现。

## 2 基准测试程序 (Benchmark) 设计

### 2.1 命令行参数

程序需要接收以下参数:

1. n: 线程数量 (整数)
2. it: 每个线程执行的迭代次数 (整数)
3. csd: 临界区停留时间 (纳秒)
4. cd: 临界区外“计算”时间 (纳秒)
5. algo: 锁算法名称 (字符串)

### 2.2 线程行为

主线程创建 n 个线程, 并等待它们结束。每个线程执行相同的 benchmark 函数 (这里叫 f), 在其中执行 it 次循环, 每次循环做:

1. 获取锁 (根据 algo 的不同选择不同实现);
2. 在临界区停留 csd 纳秒;
3. 释放锁;
4. 在非临界区 (模拟计算) 停留 cd 纳秒。

Listing 1: lock.h

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5 #include <time.h>
6 #include <string.h>
7
8 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
9
```

```
10 static int n;
11 static int iter;
12 static long long csd; // 以纳秒为单位
13 static long long ds; // 以纳秒为单位
14 static char *algo;
15
16 typedef struct {
17 long long lock_time_ns; // 本线程在 pthread_mutex_lock 上花费的总时间
18 long long unlock_time_ns; // 本线程在 pthread_mutex_unlock 上花费的总时间
19 } thread_stats_t;
20
21 // 当前时间 (CLOCK_MONOTONIC), 返回纳秒
22 static inline long long now_ns(void)
23 {
24 struct timespec ts;
25 clock_gettime(CLOCK_MONOTONIC, &ts);
26 return (long long)ts.tv_sec * 1000000000LL + ts.tv_nsec;
27 }
28
29 // 忙等 delay_ns 纳秒 (按照题目提示用 clock_gettime)
30 static void busy_wait_ns(long long delay_ns)
31 {
32 long long start = now_ns();
33 while (now_ns() - start < delay_ns) {
34 // busy wait
35 }
36 }
37
38 void* worker(void *arg)
39 {
40 thread_stats_t *stats = (thread_stats_t *)arg;
41 stats->lock_time_ns = 0;
42 stats->unlock_time_ns = 0;
43
44 for (int i = 0; i < iter; i++)
45 {
46 long long t1, t2, t3, t4;
47
48 // 计时: 获取锁
49 t1 = now_ns();
50 pthread_mutex_lock(&mutex);
51 t2 = now_ns();
52 stats->lock_time_ns += (t2 - t1);
53
54 // 临界区内等待 csd 纳秒
55 busy_wait_ns(csd);
56
57 // 计时: 释放锁
58 t3 = now_ns();
```

```

59 pthread_mutex_unlock(&mutex);
60 t4 = now_ns();
61 stats->unlock_time_ns += (t4 - t3);
62
63 // 临界区外等待 ds 纳秒
64 busy_wait_ns(ds);
65 }
66
67 return NULL;
68 }
69
70 int main(int argc, char *argv[])
71 {
72 if (argc < 6) {
73 fprintf(stderr, "Usage: %s n iter csd_ns ds_ns algo\n", argv[0]);
74 return 1;
75 }
76
77 n = atoi(argv[1]);
78 iter = atoi(argv[2]);
79 csd = atoll(argv[3]); // 按纳秒读进来
80 ds = atoll(argv[4]);
81 algo = argv[5];
82
83 if (strcmp(algo, "POSIX") != 0) {
84 fprintf(stderr, "Unsupported algo: %s (only POSIX for now)\n", algo);
85 return 1;
86 }
87
88 pthread_t *threads = malloc(sizeof(pthread_t) * n);
89 thread_stats_t *stats = malloc(sizeof(thread_stats_t) * n);
90 if (!threads || !stats) {
91 perror("malloc");
92 return 1;
93 }
94
95 for (int i = 0; i < n; i++) {
96 pthread_create(&threads[i], NULL, worker, &stats[i]);
97 }
98
99 for (int i = 0; i < n; i++) {
100 pthread_join(threads[i], NULL);
101 }
102
103 long long total_lock_ns = 0;
104 long long total_unlock_ns = 0;
105 for (int i = 0; i < n; i++) {
106 total_lock_ns += stats[i].lock_time_ns;
107 total_unlock_ns += stats[i].unlock_time_ns;

```

```
108 }
109
110 printf("Total lock time : %.6f ms\n", total_lock_ns / 1e6);
111 printf("Total unlock time : %.6f ms\n", total_unlock_ns / 1e6);
112
113 free(threads);
114 free(stats);
115
116 return 0;
117 }
```

等待可以使用 `clock_gettime(CLOCK_MONOTONIC, ...)` 配合 busy-wait 方式实现。

## 2.3 结果统计

在程序结束时，需要统计并输出：

- 每个线程执行完 `it` 次循环所花费的时间；
- 所有线程的平均每次循环时间（单位：纳秒）；
- 扣除临界区外延迟（`cd`）和临界区延迟（`csd`）后的锁开销估计；
- 验证共享变量 `z` 是否等于 `n * it`，用于检查正确性。

## 3 锁算法说明

实验支持如下算法名（通过命令行参数 `algo` 指定）：

- `none`：不加锁（这里只在题面里提到，用于 sanity check，本实现中直接从 `locks.c` 开始，没有实现 `none`）
- `posix`：使用 `pthread_mutex_t` 实现的 POSIX 互斥锁
- `spinlock`：自旋锁，使用 `atomic_exchange` 实现
- `ticket`：Ticket Lock，先进先出队列式锁
- `mcs`：MCS Lock，链表排队式锁，避免 cache 抖动
- `futex`：使用 Linux `futex` 系统调用实现的“被动等待”锁

其中至少要实现 `spinlock`、`ticket`、`mcs` 中的一个，最好全部实现以便比较性能。

## 4 futex 接口（可选 POSIX 锁实现）

题目提供了基于 Linux `futex` 系统调用的接口示例，可以通过系统调用 `SYS_futex` 实现用户空间与内核的等待/唤醒机制。

下面是题目中最简单的 `futex.c` 示例（在完整实现中，我们用了更封装的 `futexlock.c`，见后面）：

Listing 2: futex.c 示例

```

1 #define _GNU_SOURCE
2 #include <unistd.h>
3 #include <sys/syscall.h>
4 #include <linux/futex.h>
5
6 long futex(void *addr1, int op, int val1, struct timespec *timeout,
7 void *addr2, int val3) {
8 return syscall(SYS_futex, addr1, op, val1, timeout, addr2, val3);
9 }

```

高层思想：

- `pthread_mutex_lock`: 先尝试用原子操作将锁从 FREE 改为 BUSY; 若失败, 则使用 `futex(FUTEX_WAIT)` 在内核里睡眠, 直到被唤醒;
- `pthread_mutex_unlock`: 将锁状态改回 FREE, 如果有等待线程, 则使用 `futex(FUTEX_WAKE)` 唤醒它们;
- 这种方式与自旋锁相比, 是“被动等待”(不占用 CPU), 因此在高竞争情况下更节能。

## 5 完整实现：锁与基准测试 (locks.c)

下面是你给出的完整 `locks.c`, 包含:

- Spinlock
- Ticket Lock
- MCS Lock
- POSIX `pthread_mutex` 封装
- 基于 `futex` 的自制锁的封装
- 基准测试主逻辑与统计输出

Listing 3: locks.c

```

1 #include <stdatomic.h>
2 #include <stdbool.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <pthread.h>
6 #include <string.h>
7 #include <stdint.h>
8 #include <time.h>
9 #include "futexlock.h"
10
11 #define asm_pause() asm volatile("pause")
12
13 uint32_t z = 0;

```

```

14
15 uint32_t n;
16 uint32_t it;
17 uint32_t csd;
18 uint32_t cd;
19
20 void (*ze_lock)();
21 void (*ze_unlock)();
22
23 /*
24 * spin lock
25 */
26 int slock = 0;
27
28 void spin_lock() {
29 while(atomic_exchange(&slock, 1)) { asm_pause(); }
30 }
31
32 void spin_unlock() {
33 atomic_store(&slock, 0);
34 }
35
36 /*
37 * ticket lock
38 */
39 struct {
40 uint64_t _Atomic counter;
41 uint64_t _Atomic screen;
42 } ticket = { 0, 0 };
43
44 void ticket_lock() {
45 uint64_t my = atomic_fetch_add(&ticket.counter, 1);
46 while(atomic_load(&ticket.screen) < my) { asm_pause(); }
47 }
48
49 void ticket_unlock() {
50 atomic_fetch_add(&ticket.screen, 1);
51 }
52
53 /*
54 * MCS lock
55 */
56 struct node { struct node* _Atomic next; bool _Atomic is_free; };
57 _Thread_local struct node my;
58 struct node* _Atomic lock = NULL;
59
60 void mcs_lock() {
61 my.next = NULL;
62 my.is_free = false;

```

```

63
64 struct node* p = atomic_exchange(&lock, &my);
65 if(p) {
66 atomic_store(&p->next, &my);
67 while(!atomic_load(&my.is_free)) { asm_pause(); }
68 }
69 }
70
71 void mcs_unlock() {
72 struct node* expected = &my;
73 if(!atomic_load(&my.next) &&
74 atomic_compare_exchange_strong(&lock, &expected, NULL))
75 return;
76 while(!atomic_load(&my.next)) { asm_pause(); }
77 atomic_store(&my.next->is_free, true);
78 }
79
80 /*
81 * POSIX lock
82 */
83 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
84
85 void posix_lock() {
86 pthread_mutex_lock(&mutex);
87 }
88
89 void posix_unlock() {
90 pthread_mutex_unlock(&mutex);
91 }
92
93 /*
94 * Home-made POSIX lock
95 */
96 futex_lock_t flock;
97
98 void test_futex_lock() { futex_lock(&flock); }
99 void test_futex_unlock() { futex_unlock(&flock); }
100
101 /*
102 * Benchmark
103 */
104
105 int _Atomic nb_started = 0;
106 struct {
107 pthread_t tid;
108 double elapsed;
109 }* stats;
110
111 double gettime() {

```

```

112 struct timespec ts;
113 clock_gettime(CLOCK_MONOTONIC, &ts);
114 return ts.tv_sec*1e0 + ts.tv_nsec*1e-9;
115 }
116
117 double delay(double from, uint32_t d) {
118 if(d) {
119 double res;
120
121 do {
122 res = gettimeofday();
123 } while(res < from + d*1e-9);
124
125 return res;
126 } else
127 return from;
128 }
129
130 void* f(void* _arg) {
131 uint32_t self = (uint32_t)(uintptr_t)_arg;
132 atomic_fetch_add(&nb_started, 1);
133 while(atomic_load(&nb_started) < n) { asm_pause(); }
134
135 double start = gettimeofday();
136 double cur = start;
137
138 for(int i=0; i<it; i++) {
139 ze_lock();
140 z++;
141 cur = delay(cur, csd);
142 ze_unlock();
143 cur = delay(cur, cd);
144 }
145
146 stats[self].elapsed = gettimeofday() - start;
147
148 return 0;
149 }
150
151 int main(int argc, char** argv) {
152 if(argc < 6) {
153 fprintf(stderr, "Usage: %s nb-threads it csd cd algo\n", argv[0]);
154 exit(1);
155 }
156 n = atoi(argv[1]);
157 it = atoi(argv[2]);
158 csd = atoi(argv[3]);
159 cd = atoi(argv[4]);
160 char* algo = argv[5];

```

```

161
162 stats = calloc(n, sizeof(*stats));
163
164 printf("=== test with %s lock ===\n", algo);
165 if(strcmp(algo, "spinlock") == 0) {
166 ze_lock = spin_lock;
167 ze_unlock = spin_unlock;
168 } else if(strcmp(algo, "ticket") == 0) {
169 ze_lock = ticket_lock;
170 ze_unlock = ticket_unlock;
171 } else if(strcmp(algo, "mcs") == 0) {
172 ze_lock = mcs_lock;
173 ze_unlock = mcs_unlock;
174 } else if(strcmp(algo, "posix") == 0) {
175 ze_lock = posix_lock;
176 ze_unlock = posix_unlock;
177 } else if(strcmp(algo, "futex") == 0) {
178 ze_lock = test_futex_lock;
179 ze_unlock = test_futex_unlock;
180 } else {
181 fprintf(stderr, "unknow lock: %s\n", algo);
182 exit(1);
183 }
184
185 for(int i=0; i<n; i++)
186 pthread_create(&stats[i].tid, NULL, f, (void*)(uintptr_t)i);
187
188 for(int i=0; i<n; i++) {
189 void* retval;
190 pthread_join(stats[i].tid, &retval);
191 }
192
193 double total = 0;
194 for(int i=0; i<n; i++)
195 total += stats[i].elapsed;
196
197 double av = (total*1e9 / (n * it)) - cd;
198 printf("Average: %0.01f ns per loop (with a delay of %u ns => %0.01f ns)\n", av,
199 csd, av - csd);
200
201 if(z != n * it)
202 printf("Integrity check failed: %u for %u\n", z, n * it);
203
204 return 0;
205 }

```

## 6 futex 锁头文件 (futexlock.h)

Listing 4: futexlock.h

```

1 #ifndef _FUTEX_LOCK_H_
2 #define _FUTEX_LOCK_H_
3
4 #include <stdbool.h>
5 #include <stdatomic.h>
6 #include <stdint.h>
7 #include <pthread.h>
8
9 enum {
10 FREE = 0, /* the lock is free */
11 BUSY_NO_WAITER = 1, /* the lock is BUSY and I don't have waiters => no
 futex_wake at the end of unlock */
12 BUSY_WITH_WAITERS = 2 /* the lock is BUSY and I have waiters => futex_wake
 at the end of unlock */
13 };
14
15 typedef struct {
16 uint64_t _Atomic state;
17 } futex_lock_t;
18
19 extern void futex_lock(futex_lock_t* lock);
20 extern void futex_unlock(futex_lock_t* lock);
21
22 #endif

```

## 7 futex 锁实现 (futexlock.c)

Listing 5: futexlock.c

```

1 #define _GNU_SOURCE
2 #include <unistd.h>
3 #include <sys/syscall.h>
4 #include <linux/futex.h>
5 #include <time.h>
6 #include <stdio.h>
7 #include "futexlock.h"
8
9 long futex(void *addr1, int op, int val1, struct timespec *timeout,
10 void *addr2, int val3) {
11 return syscall(SYS_futex, addr1, op, val1, timeout, addr2, val3);
12 }
13
14 void futex_wait(void* addr, int val) {
15 futex(addr, FUTEX_WAIT, val, NULL, NULL, 0); /* wait if *addr == val */
16 }
17
18 void futex_wake(void* addr, int nb_threads) {

```

```
19 futex(addr, FUTEX_WAKE, nb_threads, NULL, NULL, 0);
20 }
21
22 void futex_lock(futex_lock_t* lock) {
23 uint64_t expected = FREE;
24
25 if(!atomic_compare_exchange_strong(&lock->state, &expected, BUSY_NO_WAITER)) {
26 restart:
27 if(expected == BUSY_WITH_WAITERS || atomic_compare_exchange_strong(&lock->state,
28 &expected, BUSY_WITH_WAITERS))
29 futex_wait(&lock->state, BUSY_WITH_WAITERS);
30
31 expected = FREE;
32 if(!atomic_compare_exchange_strong(&lock->state, &expected, BUSY_WITH_WAITERS))
33 /* take the lock and we don't know if we have waiters */
34 goto restart;
35 }
36 }
37
38 void futex_unlock(futex_lock_t* lock) {
39 uint64_t expected = BUSY_NO_WAITER;
40
41 if(!atomic_compare_exchange_strong(&lock->state, &expected, FREE)) {
42 /* I have waiters */
43 atomic_store(&lock->state, FREE);
44 futex_wake(&lock->state, 1);
45 }
46 }
```

## 补充：如何在 Overleaf 调整“正文字号”

你在这份模板已经是：

- 全局字号：[12pt]；
- 正文字号：\zihao{-4}（比默认大一档，适合阅读报告）。

如果以后想再调：

- 想更大：改成 \zihao{-3} 或 \zihao{-2}；
- 想小一点：改成 \zihao{4} 或去掉 \zihao 直接用 12pt。

## 1 非阻塞算法 (Non-blocking Algorithms)

### 1.1 1. 锁保护队列 (Lock-based Queue)

#### 题目要求

- (a) 编写主函数创建  $N$  个线程 ( $N$  为命令行参数), 并等待所有线程结束。
- (b) 实现一个用 `pthread_mutex_lock` 保护的整型队列 (包含 `enqueue` 和 `dequeue` 函数)。
- (c) 在每个线程中执行: 入队  $K$  个节点, 再出队  $K$  个节点, 其中  $K$  为命令行参数。
- (d) 运行时使用  $N = 10$ ,  $K = 1,000,000$ , 验证程序结束时队列为空。

#### 提示

可使用简单的链表结构表示队列, 并通过互斥锁保护临界区。

```
1 // Created by liyif on 2025/11/2.
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5 #include <pthread.h>
6
7 #define N_VAL 10
8 #define K_VAL 1000000
9
10 // 节点
11 struct noed {
12 int val;
13 struct noed *next;
14 };
15
16 struct noed *head = NULL;
17 struct noed *tail = NULL;
18 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
19
20 // 入队
21 void push(int v)
22 {
23 struct noed *n = malloc(sizeof(struct noed));
24 n->val = v;
25 n->next = NULL;
26
27 pthread_mutex_lock(&lock);
28 if (head == NULL) {
29 // 队列原来是空的
30 head = tail = n;
31 } else {
32 // 队列非空, 挂到尾巴后面, 并且更新尾巴
33 tail->next = n;
34 tail = n; // 你之前少了这句
```

```
35 }
36
37 }
38
39 // 出队, 成功返回0, 空返回-1
40 int pop(int *out)
41 {
42 pthread_mutex_lock(&lock);
43 if (head == NULL) {
44 pthread_mutex_unlock(&lock);
45 return -1;
46 }
47
48 struct noed *n = head;
49 head = head->next;
50 if (head == NULL)
51 tail = NULL;
52 pthread_mutex_unlock(&lock);
53
54 // 一定要先用再free
55 if (out)
56 *out = n->val;
57 free(n);
58 return 0;
59 }
60
61 // 线程参数
62 struct targ {
63 int id;
64 int K;
65 };
66
67 void *worker(void *arg)
68 {
69 struct targ *t = arg;
70 int K = t->K;
71
72 // 先入 K 个
73 for (int i = 0; i < K; i++) {
74 push(t->id);
75 }
76
77 // 再出 K 个
78 for (int i = 0; i < K; i++) {
79 int x;
80 while (pop(&x) != 0) {
81 // 队列暂时空了就继续试
82 }
83 }
```

```

84
85 return NULL;
86 }
87
88 int main(void)
89 {
90 int N = N_VAL;
91 int K = K_VAL;
92
93 pthread_t *ths = malloc(sizeof(pthread_t) * N);
94 struct targ *args = malloc(sizeof(struct targ) * N);
95
96 for (int i = 0; i < N; i++) {
97 args[i].id = i;
98 args[i].K = K;
99 pthread_create(&ths[i], NULL, worker, &args[i]);
100 }
101
102 for (int i = 0; i < N; i++) {
103 pthread_join(ths[i], NULL);
104 }
105
106 if (head == NULL)
107 printf("OK: empty\n");
108 else
109 printf("ERR: 队列不为空\n");
110
111 return 0;
112 }

```

## 1.2 2. 无锁队列 (Lock-free Queue)

### 题目要求

- (a) 使用课上讲解的算法实现一个 **无锁整型队列**。
- (b) 使用 C11 原子操作 (stdatomic.h) 和 Compare-And-Swap (CAS) 操作。
- (c) 同样运行  $N = 10$ ,  $K = 1,000,000$ , 并验证结束时队列为空。
- (d) 暂时不要尝试释放节点内存 (free), 稍后我们将讨论为什么这会引发问题。

### 调试建议

可增加一个打印队列状态的辅助函数, 用于调试队列正确性。

```

1 // 1_c.c —— 无锁队列 (课堂版, 简化原子用法)
2 // 不在出队时释放节点
3
4 #include <stdio.h>
5 #include <stdlib.h>

```

```
6 #include <stdatomic.h>
7 #include <pthread.h>
8
9 #define N_THREADS 10
10 #define K_OPS 1000000
11
12 // 节点
13 typedef struct node {
14 _Atomic(struct node *) next;
15 int value;
16 } node_t;
17
18 // 队列
19 typedef struct {
20 _Atomic(node_t *) head;
21 _Atomic(node_t *) tail;
22 } lfqueue_t;
23
24 lfqueue_t q;
25
26 // 初始化: 建一个虚拟节点
27 void lfqueue_init(lfqueue_t *q)
28 {
29 node_t *fake = malloc(sizeof(node_t));
30 fake->value = 0;
31 atomic_store(&fake->next, NULL);
32
33 atomic_store(&q->head, fake);
34 atomic_store(&q->tail, fake);
35 }
36
37 // 入队: 按“尾指针可能落后, 要帮忙推进”的算法
38 void lfqueue_enqueue(lfqueue_t *q, int v)
39 {
40 // 1. 新节点, next = NULL
41 node_t *node = malloc(sizeof(node_t));
42 node->value = v;
43 atomic_store(&node->next, NULL);
44
45 while (1) {
46 // 2. 读当前 tail
47 node_t *old_tail = atomic_load(&q->tail);
48 node_t *tail_next = atomic_load(&old_tail->next);
49
50 if (tail_next != NULL) {
51 // 3. 说明 tail 落后了, 帮忙推进
52 // CAS(&q->tail, old_tail, tail_next);
53 atomic_compare_exchange_weak(&q->tail, &old_tail, tail_next);
54 // 然后重新来一轮
```

```

55 continue;
56 }
57
58 // 4. 到这里说明 old_tail 真的是最后一个了, 它的 next 是 NULL
59 node_t *expected = NULL;
60 // CAS(&old_tail->next, NULL, node)
61 if (atomic_compare_exchange_weak(&old_tail->next, &expected, node)) {
62 // 5. 挂成功了, 再顺手把 tail 推到新节点 (失败无所谓)
63 atomic_compare_exchange_weak(&q->tail, &old_tail, node);
64 return;
65 }
66 // 如果没挂上, 说明别的线程也刚好挂了一个, 重新来
67 }
68 }
69
70 // 出队: 成功返回 0, 空返回 -1, 不 free 节点
71 int lfqueue_dequeue(lfqueue_t *q, int *out)
72 {
73 while (1) {
74 node_t *head = atomic_load(&q->head);
75 node_t *next = atomic_load(&head->next);
76
77 // 只有 fake, 没有别的, 就是空
78 if (next == NULL) {
79 return -1;
80 }
81
82 // 尝试把 head 前移到 next
83 if (atomic_compare_exchange_weak(&q->head, &head, next)) {
84 if (out) {
85 *out = next->value;
86 }
87 // 不 free(head);
88 return 0;
89 }
90 // 否则被别人抢了, 重来
91 }
92 }
93
94 // 线程参数
95 typedef struct {
96 int id;
97 int K;
98 } targs_t;
99
100 void *worker(void *arg)
101 {
102 targs_t *t = arg;
103 int K = t->K;

```

```

104
105 // 先入 K 个
106 for (int i = 0; i < K; i++) {
107 lfqueue_enqueue(&q, t->id);
108 }
109
110 // 再出 K 个
111 for (int i = 0; i < K; i++) {
112 int x;
113 while (lfqueue_dequeue(&q, &x) != 0) {
114 // busy wait
115 }
116 }
117
118 return NULL;
119 }
120
121 int main(void)
122 {
123 lfqueue_init(&q);
124
125 pthread_t ths[N_THREADS];
126 targs_t args[N_THREADS];
127
128 for (int i = 0; i < N_THREADS; i++) {
129 args[i].id = i;
130 args[i].K = K_OPS;
131 pthread_create(&ths[i], NULL, worker, &args[i]);
132 }
133
134 for (int i = 0; i < N_THREADS; i++) {
135 pthread_join(ths[i], NULL);
136 }
137
138 // 队列“逻辑为空”应当: head == tail && head->next == NULL
139 node_t *h = atomic_load(&q.head);
140 node_t *t = atomic_load(&q.tail);
141 node_t *nxt = atomic_load(&h->next);
142
143 if (h == t && nxt == NULL) {
144 printf("OK: 无锁队列最终为空 (只剩 fake) \n");
145 } else {
146 printf("ERR: 无锁队列最终不为空! \n");
147 }
148
149 return 0;
150 }

```

### 1.3 3. 性能对比 (Benchmarking)

#### 题目要求

使用 `clock_gettime(CLOCK_MONOTONIC)` 计时，比较以下两种实现的性能：

- 使用 `pthread_mutex` 保护的队列；
- 无锁队列。

统计运行时间，比较哪种方式在多线程环境下性能更优。

---

### 1.4 4. ABA 问题与崩溃分析

#### 题目要求

在无锁队列的 `dequeue` 操作中尝试 `free` 被移除的节点。

**结果：**程序应当崩溃 (Segmentation Fault)。

#### 分析问题

- 思考为什么会发生崩溃。
  - 调查 **ABA Problem**：在 CAS 操作中，当一个地址被修改后又被改回时，CAS 仍认为“没变”，从而产生错误。
  - 与教师讨论确认你的理解是否正确。
- 

### 1.5 5. 使用 Hazard Pointers 解决 ABA 问题 (可选，困难)

#### 背景原理

参考论文：

Maged Michael, “Hazard Pointers: Safe Memory Reclamation for Lock-Free Objects”, TPDS 2004.

核心思想：

- 当线程读取一个指针  $X$  并打算对该位置执行 CAS 修改时，必须保证  $X$  在这段期间不会被释放。
- 为此，每个线程使用一个线程局部变量 `hazard` 注册当前正在使用的指针。
- 只要任意线程的 `hazard` 指针仍指向  $X$ ， $X$  就不能被释放。

#### 在队列中的应用

- (a) 在 `dequeue` 中，线程读取 `head` 指针并存入 `hazard`。
- (b) 再次读取 `head` 验证一致性。
- (c) CAS 成功后，不直接 `free`，而是清除 `hazard` 并将节点加入一个 **无锁栈** `free_nodes`。
- (d) 在 `enqueue` 中同样要防止 `tail` 被释放，使用相同机制。

## 内存复用逻辑

分配新节点时:

1. 尝试从 `free_nodes` 栈中弹出节点 `Z`;
2. 若为空则使用 `malloc`;
3. 若发现 `Z` 出现在任意线程的 `hazard` 中, 表示仍被使用, 应重新压回 `free_nodes`;
4. 否则, 该节点已安全, 可重用。

**思考题:** 为什么同一个节点不能在 `free_nodes` 栈中出现两次?

## 2 无锁 Radix Tree (可选)

### 2.1 6. 实现要求

- 实现一个无锁版本的 Radix Tree;
- 可参考公开的 radix tree 结构描述;
- 假设不需要删除元素。

```

1 //
2 // Created by liyif on 2025/11/2.
3 //
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <stdint.h>
7 #include <stdatomic.h>
8 #include <pthread.h>
9
10
11 #define RADIX 16 // 每层分16叉 (4比特)
12 #define THREADS 4
13
14 typedef struct Node
15 {
16 _Atomic(struct Node *) child[RADIX];
17 _Atomic(void*) values;
18 }Node;
19
20 Node* node_new()
21 {
22 Node* n = malloc(sizeof(Node));
23 if (!n) exit(1);
24 for (int i = 0; i < RADIX; i++)
25 {
26 atomic_store_explicit(&n->child[i], NULL, memory_order_relaxed);
27 }

```

```

28 atomic_store_explicit(&n->values, NULL, memory_order_relaxed);
29 return n;
30 }
31
32 void insert(Node* root, uint64_t key, void* val)
33 {
34 Node* node=root;
35 for (int shift=60; shift>=0; shift-=4)
36 {
37 int id =(key>>shift) & 0xF;
38 Node *next=atomic_load_explicit(&node->child[id], memory_order_relaxed);
39 if (!next)
40 {
41 Node* newNode = node_new();
42 Node* expected =NULL;
43 if (atomic_compare_exchange_strong_explicit(&node->child[id], &expected,
44 newNode, memory_order_acq_rel, memory_order_acquire)){
45 next = newNode;
46 }else
47 {
48 free(newNode); // 有别的线程已经放进去了
49 next = expected;
50 }
51 node = next;
52 }
53 atomic_store_explicit(&node->values, val, memory_order_relaxed);
54 }
55 void* lookup(Node* root, uint64_t key)
56 {
57 Node* node=root;
58 for (int shift=60; shift>=0; shift-=4)
59 {
60 int id = (key>>shift) & 0xF;
61 node=atomic_load_explicit(&node->child[id], memory_order_relaxed);
62 if (!node)
63 {
64 return NULL;
65 }
66 }
67 return atomic_load_explicit(&node->values, memory_order_acquire);
68 }
69 static _Atomic int thread_id = 0;
70 void* thread_task(void* arg)
71 {
72 Node* node = (Node*)arg;
73 int id;
74 id =atomic_fetch_add(&thread_id, 1);
75 for (uint64_t k=id*100; k<(id+1)*100; k++)

```

```

76 {
77 int* val=malloc(sizeof(int));
78 *val=k*2;
79 insert(node,k,val);
80 }
81 return NULL;
82 }
83 int main()
84 {
85 Node* root = node_new();
86 pthread_t t[THREADS];
87 for (int i = 0; i < THREADS; i++)
88 {
89 pthread_create(&t[i], NULL, thread_task, root);
90 }
91 for (int i = 0; i < THREADS; i++)
92 {
93 pthread_join(t[i], NULL);
94 }
95 for (uint64_t k = 50; k < 55; ++k) {
96 int* v = lookup(root, k);
97 if (v)
98 printf("key=%lu, value=%d\n", k, *v);
99 else
100 printf("key=%lu not found\n", k);
101 }
102 return 0;
103 }

```

### 3 无锁链表 (Lock-free Linked List)

#### 3.1 7. 实验要求

- 重新实现第一个练习（队列），但结构改为链表；
- 暂时不要尝试释放节点；
- 若已实现 Hazard Pointers，可尝试将其用于解决链表中的 ABA 问题（难题）。

```

1 typedef struct Node {
2 int data;
3 struct Node *next;
4 } Node;
5
6 typedef struct {
7 Node *front;
8 Node *rear;
9 size_t size; // 仅用于最终校验
10 pthread_mutex_t mu;

```

```

11 } Queue;
12
13 /* ----- 队列：仅用互斥锁保护 ----- */
14 static void queue_init(Queue *q) {
15 q->front = q->rear = NULL;
16 q->size = 0;
17 pthread_mutex_init(&q->mu, NULL);
18 }
19
20 static void enqueue(Queue *q, int data) {
21 Node *n = (Node*)malloc(sizeof(Node));
22 if (!n) {
23 perror("malloc");
24 abort();
25 }
26 n->data = data;
27 n->next = NULL;
28
29 pthread_mutex_lock(&q->mu);
30 if (q->rear == NULL) {
31 q->front = q->rear = n;
32 } else {
33 q->rear->next = n;
34 q->rear = n;
35 }
36 q->size++;
37 pthread_mutex_unlock(&q->mu);
38 }
39
40 // 非阻塞出队：若空返回0，否则 *out 设置为值并返回1
41 static int try_dequeue(Queue *q, int *out) {
42 pthread_mutex_lock(&q->mu);
43 Node *f = q->front;
44 if (f == NULL) {
45 pthread_mutex_unlock(&q->mu);
46 return 0;
47 }
48 *out = f->data;
49 q->front = f->next;
50 if (q->front == NULL) q->rear = NULL;
51 q->size--;
52 pthread_mutex_unlock(&q->mu);
53 free(f);
54 return 1;
55 }
56
57 static int queue_is_empty(Queue *q) {
58 pthread_mutex_lock(&q->mu);
59 int empty = (q->front == NULL) && (q->rear == NULL) && (q->size == 0);

```

```

60 pthread_mutex_unlock(&q->mu);
61 return empty;
62 }
63
64 /* ----- 线程参数与工作函数 ----- */
65 typedef struct {
66 int id;
67 int K;
68 Queue *q;
69 } TArg;
70
71 static void *worker(void *arg) {
72 TArg *t = (TArg*)arg;
73 Queue *q = t->q;
74 const int K = t->K;
75
76 // 入队 K 次: 放入一些可区分的数据 (高16位写线程ID)
77 for (int i = 0; i < K; ++i) {
78 int val = (t->id << 16) | (i & 0xFFFF);
79 enqueue(q, val);
80 }
81
82 // 出队 K 次: 空则自旋
83 int got, v;
84 for (int i = 0; i < K; ++i) {
85 do {
86 got = try_dequeue(q, &v);
87 if (!got) sched_yield(); // 降低忙等占用
88 } while (!got);
89 }
90 return NULL;
91 }
92
93 /* ----- 主函数: N、K 为应用参数 ----- */
94 static int parse_pos_int(const char *s, int *out) {
95 errno = 0;
96 char *end = NULL;
97 long v = strtol(s, &end, 10);
98 if (errno || end == s || *end != '\0' || v <= 0 || v > 1000000000L) return -1;
99 *out = (int)v;
100 return 0;
101 }
102
103 int main(int argc, char **argv) {
104 if (argc != 3) {
105 fprintf(stderr, "用法: %s <线程数N> <每线程次数K>\n", argv[0]);
106 fprintf(stderr, "示例: %s 10 1000000\n", argv[0]);
107 return 1;
108 }

```

```

109 int N, K;
110 if (parse_pos_int(argv[1], &N) || parse_pos_int(argv[2], &K)) {
111 fprintf(stderr, "参数错误: N、K 必须为正整数\n");
112 return 1;
113 }
114
115 Queue q;
116 queue_init(&q);
117
118 pthread_t *ths = (pthread_t*)malloc(sizeof(pthread_t) * (size_t)N);
119 TArg *args = (TArg*)malloc(sizeof(TArg) * (size_t)N);
120 if (!ths || !args) { perror("malloc"); return 1; }
121
122 for (int i = 0; i < N; ++i) {
123 args[i].id = i;
124 args[i].K = K;
125 args[i].q = &q;
126 int rc = pthread_create(&ths[i], NULL, worker, &args[i]);
127 if (rc != 0) {
128 fprintf(stderr, "pthread_create 失败: %d (i=%d)\n", rc, i);
129 N = i; // 只 join 已创建的
130 break;
131 }
132 }
133
134 for (int i = 0; i < N; ++i) pthread_join(ths[i], NULL);
135
136 if (queue_is_empty(&q)) {
137 printf("OK: 队列为空 (N=%d, K=%d, 总操作=%lld)\n",
138 N, K, (long long)N * (long long)K * 2LL);
139 } else {
140 printf("ERR: 队列不为空! \n");
141 }
142
143 free(ths);
144 free(args);
145 // 这里不再遍历释放剩余节点, 因为我们已经检查为空
146
147 return 0;
148 }

```

```

1 //
2 // Created by liyif on 2025/11/3.
3 //
4 // build: gcc -O2 -Wall -Wextra -pthread -std=c11 -o lf_queue lf_queue.c
5 // run: ./lf_queue 10 1000000
6 // 说明: 为简单起见, 不做内存回收 (dequeue 不 free), 避免 HP/RCU/EBR 复杂性。
7
8 #include <stdio.h>

```



```

55 memory_order_release, memory_order_relaxed); //
56 ATOMIC CAS
57 return;
58 }
59 // 尾指针落后了, 帮它前进
60 atomic_compare_exchange_weak_explicit(
61 &q->tail, &tail, next,
62 memory_order_release, memory_order_relaxed); //
63 ATOMIC CAS
64 }
65 // 失败则自旋重试
66 }
67 }
68
69 // 返回 1 表示成功出队到 *out, 返回 0 表示当前空
70 static int lfqueue_try_dequeue(LFQueue *q, int *out) {
71 while (1) {
72 Node *head = atomic_load_explicit(&q->head, memory_order_acquire); //
73 ATOMIC
74 Node *tail = atomic_load_explicit(&q->tail, memory_order_acquire); //
75 ATOMIC
76 Node *next = atomic_load_explicit(&head->next, memory_order_acquire); //
77 ATOMIC
78
79 if (head != atomic_load_explicit(&q->head, memory_order_acquire)) //
80 ATOMIC
81 continue; // 发生变化, 重试
82
83 if (next == NULL) {
84 return 0; // 空
85 }
86 if (head == tail) {
87 // tail 落后, 推进
88 atomic_compare_exchange_weak_explicit(
89 &q->tail, &tail, next,
90 memory_order_release, memory_order_relaxed); //
91 ATOMIC CAS
92 continue;
93 }
94 int val = next->value; // 读取数据
95 // 把 head 前进到 next (逻辑删除 head)
96 if (atomic_compare_exchange_weak_explicit(
97 &q->head, &head, next,
98 memory_order_acq_rel, memory_order_relaxed)) { //
99 ATOMIC CAS
100 // 简化: 不 free(head), 避免回收复杂性 (ABA/悬挂指针)
101 *out = val;

```

```

96 return 1;
97 }
98 // 否则重试
99 }
100 }
101
102 static int lfqueue_is_empty(LFQueue *q) {
103 Node *head = atomic_load_explicit(&q->head, memory_order_acquire); //
104 ATOMIC
105 Node *next = atomic_load_explicit(&head->next, memory_order_acquire); //
106 ATOMIC
107 return next == NULL;
108 }
109
110 /* ----- 工作线程 ----- */
111
112 typedef struct {
113 int id;
114 int K;
115 LFQueue *q;
116 } TArg;
117
118 static void *worker(void *arg) {
119 TArg *t = (TArg*)arg;
120 LFQueue *q = t->q;
121 const int id = t->id;
122 const int K = t->K;
123
124 // 先入队 K 次 (编码线程ID和序号, 便于调试)
125 for (int i = 0; i < K; ++i) {
126 int v = (id << 8) ^ i;
127 lfqueue_enqueue(q, v);
128 }
129
130 // 再出队 K 次 (自旋直到成功)
131 int x, ok;
132 for (int i = 0; i < K; ++i) {
133 do {
134 ok = lfqueue_try_dequeue(q, &x);
135 if (!ok) sched_yield();
136 } while (!ok);
137 }
138 return NULL;
139 }
140
141 /* ----- 工具: 参数解析与主函数 ----- */
142
143 int main(int argc, char **argv) {

```

```

143 if (argc != 3) {
144 fprintf(stderr, "用法: %s <线程数N> <每线程K>\n示例: %s 10 1000000\n", argv
 [0], argv[0]);
145 return 1;
146 }
147 int N, K;
148
149
150 LFQueue q;
151 lfqueue_init(&q);
152
153 pthread_t *ths = (pthread_t*)malloc(sizeof(pthread_t) * (size_t)N);
154 TArg *args = (TArg*)malloc(sizeof(TArg) * (size_t)N);
155 if (!ths || !args) { perror("malloc"); return 1; }
156
157 for (int i = 0; i < N; ++i) {
158 args[i].id = i;
159 args[i].K = K;
160 args[i].q = &q;
161 int rc = pthread_create(&ths[i], NULL, worker, &args[i]);
162 if (rc != 0) {
163 fprintf(stderr, "pthread_create 失败: %d (i=%d)\n", rc, i);
164 N = i; // 只等待已创建的
165 break;
166 }
167 }
168 for (int i = 0; i < N; ++i) pthread_join(ths[i], NULL);
169
170 if (lfqueue_is_empty(&q)) {
171 printf("OK: 无锁队列为空 (N=%d, K=%d, 总操作=%lld)\n",
172 N, K, (long long)N * (long long)K * 2LL);
173 } else {
174 printf("ERR: 无锁队列不为空! \n");
175 }
176
177 free(ths);
178 free(args);
179 // 注意: 未进行节点回收, 存在内存泄漏; 用于课堂实验可接受。
180
181 return 0;
182 }

```

## 4 简单内存分配器 (A Simple Memory Allocator)

### 4.1 8. 数据结构定义

分配器内部管理内存块 (chunk):

Listing 1: chunk 结构

```

1 struct chunk {
2 size_t size;
3 union {
4 struct chunk* next; /* next node when the chunk is free */
5 char content[0]; /* the content of the chunk when allocated */
6 };
7 };

```

接口函数:

```

1 struct chunk* alloc_chunk(size_t size);
2 void free_chunk(struct chunk* c);

```

全局变量 `struct chunk* free_list` 存储所有空闲块。

## 4.2 9. 基于锁的分配器 (Lock-based Version)

### 设计思路

- 使用全局互斥锁 `pthread_mutex_t mutex`。
- `free_chunk`: 释放一个 chunk, 将其加入 `free_list`。
- `create_large_chunk()`: 创建 64MB 大块, 返回未链接的 chunk。

### 函数要求

(a) `create_large_chunk()`: 使用 `malloc` 分配  $64 \times 1024 \times 1024$  字节, 初始化 `size` 字段。

(b) `alloc_chunk()`:

- 在 `free_list` 中找到第一个足够大的块;
- 若找不到, 调用 `create_large_chunk()` 新建;
- 若块比需求大, 拆分为两个块以避免浪费;
- 使用锁保护整个过程。

## 4.3 10. 无锁分配器 (Lock-free Version)

### 总体思路

将 `free_list` 视为无锁栈, 用原子操作替代互斥锁。

### 函数要求

(a) `pop()`: 若 `free_list` 为 `NULL`, 返回 `NULL`; 否则弹出首元素。

(b) `push(struct chunk* head, struct chunk* tail)`: 将 `head..tail` 链整段压入 `free_list`; 不需遍历。

(c) `free_chunk()`: 将释放的块压入栈。

(d) `alloc_chunk()`: 通过 `pop()`、`push()` 和 `create_large_chunk()` 组合实现; 无需锁。

## 公共数据结构说明

题目给出的 `chunk` 结构如下, 用来管理内存块:

Listing 2: 公共数据结构: `chunk` 和 `free_list`

```

1 #include <stddef.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4 #include <stdatomic.h>
5
6 /* 每一个 chunk 表示一段连续的可分配内存区域 */
7 struct chunk {
8 size_t size; /* 该 chunk 可用的 payload 大小 (字节数, 不含自己的头部) */
9 union {
10 struct chunk* next; /* 当 chunk 处于“空闲状态”时, 用 next 把空闲块串成链表
11 */
12 char content[0]; /* 当 chunk 被分配出去时, 用户的数据从这里开始 */
13 };
14 };

```

后续会分两部分:

- Part 1: 基于锁的版本——使用 `pthread_mutex_t mutex` 保护全局空闲链表 `free_list`;
- Part 2: 无锁版本——将 `free_list` 视为 lock-free 栈, 用原子 CAS 实现 `pop` / `push`。

## 5 Part 1: 基于锁的版本

本部分中, 我们实现一个使用互斥锁保护的分配器版本。需要的全局变量如下:

Listing 3: Part 1 全局变量 (带锁)

```

1 /* 全局空闲链表头指针: 只在持有 mutex 时读写 */
2 static struct chunk* free_list = NULL;
3
4 /* 全局互斥锁, 保护对 free_list 的访问 */
5 static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
6
7 /* 统一定义一块大 chunk 的总大小 (包含头部和 payload) */
8 #define LARGE_CHUNK_TOTAL_SIZE (64UL * 1024 * 1024)

```

### 5.1 Question 4.a 自由块: free\_chunk

要求: 给出 free\_chunk(struct chunk\* c) 的代码, 把 c 插入到 free\_list 中。

Listing 4: Q4.a 基于锁的 free\_chunk

```

1 /*
2 * 将一个已经不再使用的 chunk c 重新挂回全局 free_list。
3 * 注意:
4 * - 这里假设 c 是通过本分配器分配出来的;
5 * - free_list 是一个 LIFO 单链表, 我们简单地头插入即可。
6 */
7 void free_chunk(struct chunk* c)
8 {
9 if (c == NULL) {
10 /* 传进来空指针就什么都不做, 防御式编程 */
11 return;
12 }
13
14 /* 加锁, 保护对全局 free_list 的访问 */
15 pthread_mutex_lock(&mutex);
16
17 /* 头插法: 让 c->next 指向原来的表头, 然后把表头改成 c */
18 c->next = free_list;
19 free_list = c;
20
21 /* 解锁 */
22 pthread_mutex_unlock(&mutex);
23 }

```

### 5.2 Question 4.b 创建大块: create\_large\_chunk

要求: 给出 struct chunk\* create\_large\_chunk() 的代码。该函数:

- 使用 malloc 分配一块 64 \* 1024 \* 1024 字节的大内存;
- 正确设置其 size 字段并返回;
- 不负责把新块挂入 free\_list (只返回)。

Listing 5: Q4.b create\_large\_chunk

```

1 /*
2 * 分配一块“超大 chunk”, 总大小为 64MiB。
3 * 返回值:
4 * - 成功: 返回指向新 chunk 头部的指针
5 * - 失败: 返回 NULL
6 *
7 * 注意:
8 * - LARGE_CHUNK_TOTAL_SIZE 包含头部和 payload 两部分;
9 * - size 字段只记录 payload 的大小。
10 */

```

```

11 struct chunk* create_large_chunk(void)
12 {
13 /* malloc 返回的是一大块连续的原始内存 */
14 struct chunk* c = (struct chunk*)malloc(LARGE_CHUNK_TOTAL_SIZE);
15 if (c == NULL) {
16 /* 内存不足, 直接返回 NULL, 由调用者处理 */
17 return NULL;
18 }
19
20 /* 可用的 payload 大小 = 总大小 - 头部大小 */
21 c->size = LARGE_CHUNK_TOTAL_SIZE - sizeof(struct chunk);
22
23 /* 作为自由块时 next 暂时置为 NULL, 方便后续挂入 free_list */
24 c->next = NULL;
25
26 return c;
27 }

```

### 5.3 Question 4.c 分配: alloc\_chunk

要求: 给出 alloc\_chunk 的代码:

- 在 free\_list 中找第一个够大的 chunk;
- 若没有足够大的块, 则调用 create\_large\_chunk 新建一块, 加入 free\_list 后重试;
- 若找到的块太大, 则将其拆成“被分配的部分”和“剩余空闲部分”两个 chunk;
- 返回用于分配的那一块。

Listing 6: Q4.c 基于锁的 alloc\_chunk

```

1 /*
2 * 从 free_list 中分配一个 payload 至少为 size 字节的 chunk。
3 * 简化策略:
4 * - first-fit: 从头到尾找到第一个 size 足够大的块;
5 * - 如找不到合适块, 则创建一个 64MiB 的大块挂到 free_list, 再重新搜索;
6 * - 如果找到的块过大, 会在其尾部切出一个新的空闲块, 避免浪费。
7 */
8 struct chunk* alloc_chunk(size_t size)
9 {
10 /* size 为 0 就没有意义, 直接返回 NULL (也可按需求改成最小分配单元) */
11 if (size == 0) {
12 return NULL;
13 }
14
15 for (;;) { /* 外层死循环: 如果当前 free_list 不够, 就创建大块后重试 */
16 pthread_mutex_lock(&mutex);
17
18 struct chunk* prev = NULL; /* 记录当前节点的前驱, 便于从链表中摘除 */

```

```

19 struct chunk* cur = free_list; /* 遍历 free_list */
20
21 /* 1. 在 free_list 中寻找第一个 size 足够大的 chunk (first-fit) */
22 while (cur != NULL && cur->size < size) {
23 prev = cur;
24 cur = cur->next;
25 }
26
27 if (cur == NULL) {
28 /* 说明现有 free_list 中没有任何块能满足 size 需求 */
29 pthread_mutex_unlock(&mutex);
30
31 /* 2. 申请一个新的“大块”，并通过 free_chunk 挂到 free_list 中，再重试 */
32 struct chunk* big = create_large_chunk();
33 if (big == NULL) {
34 /* 内存确实不够了，直接失败返回 */
35 return NULL;
36 }
37 /* free_chunk 内部会自行加锁，因此此处不能持有 mutex */
38 free_chunk(big);
39 /* 然后重新进入 for(;;) 搜索，此时 free_list 中多了一个大块 */
40 continue;
41 }
42
43 /* 走到这里，cur 指向第一个 size 足够大的空闲 chunk */
44 size_t original_size = cur->size;
45 size_t remaining = original_size - size;
46
47 /*
48 * 3. 判断是否需要“切块”：
49 * 如果剩余空间足够存放一个新的 struct chunk 头部，
50 * 则在当前 chunk 的 content 区域尾部构造一个新的空闲块，
51 * 并将其重新挂回 free_list。
52 */
53 if (remaining > sizeof(struct chunk)) {
54 /* 在 cur 的 content 部分偏移 size 字节处，构造一个新的 chunk 头部 */
55 struct chunk* rest =
56 (struct chunk*)(cur->content + size);
57
58 rest->size = remaining - sizeof(struct chunk);
59 /* rest 还在 free_list 中，因此它的 next 应当接在 cur->next 后 */
60 rest->next = cur->next;
61
62 /* 把 rest 接回链表，替代 cur 的位置 */
63 if (prev != NULL) {
64 prev->next = rest;
65 } else {
66 /* 如果 cur 正好是表头，那么新的表头就是 rest */

```

```

67 free_list = rest;
68 }
69 } else {
70 /*
71 * 剩余空间太小，不够再建立一个新的 chunk 头部，
72 * 这时就直接把整个 cur 分配出去，不再切块，
73 * 仅仅需要把 cur 从 free_list 中摘除即可。
74 */
75 if (prev != NULL) {
76 prev->next = cur->next;
77 } else {
78 free_list = cur->next;
79 }
80 }
81
82 /* 此时 cur 不再在 free_list 中，可安全返回给用户使用 */
83 cur->size = size; /* 记录真实分配的 payload 大小 */
84
85 pthread_mutex_unlock(&mutex);
86 return cur;
87 }
88 }

```

## 6 Part 2: 无锁版本

在本部分中，我们逐步实现无锁版本的分配器。主要思想：将 `free_list` 视为一个 lock-free 栈，只在 `pop` / `push` / `free_chunk` 内部使用原子 CAS 操作，`alloc_chunk` 本身只调用这些函数即可。

### 全局变量（无锁版）

Listing 7: Part 2 全局变量（无锁）

```

1 /* 无锁版本中，free_list 是一个原子指针，表示 lock-free 栈的栈顶 */
2 static struct chunk* _Atomic free_list_atomic = NULL;

```

后续所有函数都基于 `free_list_atomic`。

#### 6.1 Question 4.d 流行 (pop): pop

要求：实现 `struct chunk* pop()`。

- 如果 `free_list` 为 `NULL`，返回 `NULL`；
- 否则，从栈顶弹出一个元素并返回；
- 使用 CAS 保证无锁并发安全。

Listing 8: Q4.d 无锁 pop

```

1 /*
2 * 从 lock-free 栈 free_list_atomic 中弹出一个 chunk。
3 * 多个线程可以并发调用，依靠 CAS 来保证正确性。
4 */
5 struct chunk* pop(void)
6 {
7 struct chunk* head; /* 当前观察到的栈顶 */
8 struct chunk* next; /* head->next */
9
10 for (;;) {
11 /* 1. 读取当前栈顶（可能同时被其他线程修改） */
12 head = atomic_load(&free_list_atomic);
13 if (head == NULL) {
14 /* 栈为空，直接返回 NULL */
15 return NULL;
16 }
17
18 /* 2. 读取 head 之后的节点，CAS 时要把 free_list 改成 next */
19 next = head->next;
20
21 /*
22 * 3. 尝试用 CAS 把 free_list_atomic 从 head 改成 next。
23 * - 成功：我们成功“抢到”了 head，直接返回它；
24 * - 失败：说明在我们 CAS 之前，其他线程已经修改了栈顶，
25 * 那就重新读栈顶，再试一次。
26 *
27 * 注意：atomic_compare_exchange_weak 在失败时会把 free_list_atomic
28 * 当前值写回 head，所以 head 的内容会被刷新。
29 */
30 if (atomic_compare_exchange_weak(&free_list_atomic, &head, next)) {
31 return head;
32 }
33
34 /* CAS 失败，继续循环重试 */
35 }
36 }

```

## 6.2 Question 4.e 推 (push): push

**要求：**实现 void push(struct chunk\* head, struct chunk\* tail)。

- head 是一段链表的头，tail 是该链表的尾；
- 函数将这段链表整体压入 free\_list 栈顶；
- 禁止从 head 遍历到 tail，因为已经给出了 tail。

Listing 9: Q4.e 无锁 push

```

1 /*
2 * 将一整段 [head .. tail] 单链表（已知尾结点）整体压到 free_list_atomic 栈顶。
3 * 假设：
4 * - 这段链表在调用前已经是一个“闭合”的链表（tail->next 未被使用）；
5 * - head/tail 非 NULL，且 head 到 tail 之间的 next 指针已经正确设置。
6 */
7 void push(struct chunk* head, struct chunk* tail)
8 {
9 struct chunk* old_top;
10
11 for (;;) {
12 /* 1. 读取当前的栈顶 */
13 old_top = atomic_load(&free_list_atomic);
14
15 /* 2. 把这段链表尾部接到当前栈顶上（先在本地写，不影响其他线程） */
16 tail->next = old_top;
17
18 /*
19 * 3. CAS 尝试让 free_list_atomic 指向新的 head。
20 * - 如果在此期间没有其他线程修改栈顶，则 CAS 成功；
21 * - 否则 CAS 失败，说明 old_top 已经过期，需要重新读取再试。
22 */
23 if (atomic_compare_exchange_weak(&free_list_atomic, &old_top, head)) {
24 return; /* 压栈成功 */
25 }
26
27 /* CAS 失败，循环重试 */
28 }
29 }

```

### 6.3 Question 4.f 自由块（无锁版）：free\_chunk

**要求：**给出无锁版本的 free\_chunk 代码。

**思路：**free\_chunk(c) 只是把单个结点当作一段长度为 1 的链表，用 push 压栈即可。

Listing 10: Q4.f 无锁 free\_chunk

```

1 /*
2 * 无锁版本的 free_chunk：
3 * - 只需把 c 作为一个单结点链表 [c..c] 压入 lock-free 栈即可。
4 */
5 void free_chunk(struct chunk* c)
6 {
7 if (c == NULL) {
8 return;
9 }
10
11 /* 单结点链表：先把 next 置为 NULL，然后调用 push(c, c) */
12 c->next = NULL;
13 push(c, c);

```

```
14 }
```

## 6.4 Question 4.g 分配 (无锁版): alloc\_chunk

**要求:** 给出无锁版本的 alloc\_chunk 代码。

**约束:** alloc\_chunk 本身不需要直接使用原子操作, 而是通过调用 pop、push 和 create\_large\_chunk (同 Part 1) 来完成。

Listing 11: Q4.g 无锁 alloc\_chunk

```
1 /*
2 * 无锁版本的 alloc_chunk:
3 *
4 * 策略:
5 * 1. 反复从 lock-free 栈中 pop 一个 chunk;
6 * 2. 如果栈空, 则调用 create_large_chunk() 生成一个大块, 再通过 free_chunk
7 * 把大块拆分管理后重新尝试;
8 * 3. 如果 pop 出的块足够大, 则根据需要进行“切块”,
9 * 把剩余部分再通过 free_chunk 放回栈中;
10 * 4. 如果 pop 出的块太小, 则把它再 push 回去, 继续尝试下一个。
11 *
12 * 注意:
13 * - lock-free 的正确性依赖 pop/push 的 CAS;
14 * - alloc_chunk 自身只是一个“纯算法”, 不做原子操作。
15 */
16 struct chunk* alloc_chunk(size_t size)
17 {
18 if (size == 0) {
19 return NULL;
20 }
21
22 for (;;) {
23 /* 1. 尝试从 lock-free 栈中弹出一个空闲块 */
24 struct chunk* c = pop();
25 if (c == NULL) {
26 /*
27 * 栈为空: 当前没有任何空闲块可用。
28 * 解决办法: 创建一个 64MiB 的大块, 并通过 free_chunk
29 * 挂回 free_list_atomic, 以便后续被拆分和分配。
30 */
31 struct chunk* big = create_large_chunk();
32 if (big == NULL) {
33 /* 系统真的没有内存了, 返回失败 */
34 return NULL;
35 }
36
37 /* big 现在是一个“巨大的空闲块”, 推回栈中, 供下次循环 pop 使用 */
38 free_chunk(big);
39 /* 回到 for(;;) 顶部重新尝试 */
40 continue;

```

```

41 }
42
43 /* 2. 检查这个块是否足够大 */
44 if (c->size < size) {
45 /*
46 * 这个块太小，不能满足当前请求。
47 * 为了不浪费，我们再把它放回 free_list 栈，然后继续尝试。
48 */
49 free_chunk(c);
50 continue;
51 }
52
53 /*
54 * 3. 块足够大，考虑是否切成“已分配块 + 剩余空闲块”。
55 * 如果剩余空间大于一个 struct chunk 头部，就认为可以再拆出一个空闲块。
56 */
57 size_t remaining = c->size - size;
58 if (remaining > sizeof(struct chunk)) {
59 /*
60 * 在 c 的 payload 尾部构造一个新的 chunk 头部 rest，表示剩余的空闲部
61 * 分：
62 * [c(头) | <---- size ----> | rest(头) | 剩余 payload...]
63 */
64 struct chunk* rest =
65 (struct chunk*)(c->content + size);
66
67 rest->size = remaining - sizeof(struct chunk);
68 rest->next = NULL;
69
70 /*
71 * 通过 free_chunk(rest) 把剩余部分重新压回 lock-free 栈。
72 * 这里不会产生循环，因为 rest 与 c 不再在同一段链表中。
73 */
74 free_chunk(rest);
75
76 /* c 的 size 更新为实际分配的大小 */
77 c->size = size;
78 } else {
79 /*
80 * 剩余空间不够形成新的 chunk 头部，直接把整个 c 分配出去，
81 * 不进行切块，可以减少内存碎片管理的复杂度。
82 */
83 /* c->size 保持原样或者直接更新为 size，按需求选择 */
84 c->size = size;
85 }
86
87 /* 4. 返回最终分配到的 chunk 给用户 */
88 return c;

```

89	}
90	}

## Exercise 1\*

在本次实验中，我们将实现一个用于 **软件事务内存 (Software Transactional Memory, STM)** 的小型运行时。

### Question 1.a\*

与往常一样，编写一个主函数 (`main`)，创建  $N$  个线程，其中  $N$  作为程序的参数传入。主函数必须等待所有线程执行完毕后再退出（即使用 `pthread_join` 等待线程）。

### Question 1.b\*

我们将内存建模为一个由多个单元格 (`cell`) 组成的数组。每个单元格既包含一个整数值 (`value`)，又包含一个计数器 (`counter`)。

因此，需要定义如下结构体（文件名：`mem.h`）：

Listing 1: `mem.h`

```

1 #include <sys/types.h>
2
3 #define NB_CELLS 65536
4
5 /* 单元格结构 */
6 struct cell {
7 uintptr_t value; // 存储的整数值
8 size_t counter; // 版本号或计数器
9 };
10
11 /* 整个内存结构 */
12 struct memory {
13 struct cell* cells[NB_CELLS]; // 指向所有 cell 的指针数组
14 size_t clock; // 全局时钟或版本号
15 };

```

我们规定，索引为 0 的单元格 (`cells[0]`) 即为变量 `x`。每个线程需要将该变量 `x` 增加  $1024 \times 1024$  次，使用原子操作实现。

程序结束时，请检查 `x` 的最终值是否与预期一致：

$$x = 1024 \times 1024 \times \quad .$$

不要忘记编写一个初始化函数，用于分配所有单元格并将其初始值设为 0 (`value=0`, `counter=0`)。

```

1 // mem.c — 带上原子操作即是那个d问的同样道理
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <stdint.h>
5 #include <inttypes.h>
6 #include <pthread.h>
7
8 #define NB_CELLS 65536
9 #define INC_PER_THREAD (1024*1024)
10

```

```

11 struct cell {
12 uintptr_t value;
13 size_t counter;
14 };
15
16 struct memory {
17 struct cell* cells[NB_CELLS];
18 size_t clock;
19 };
20
21 static struct memory *gmem = NULL;
22
23 static void memory_init(void) {
24 gmem = (struct memory*)calloc(1, sizeof(*gmem)); // 清零并malloc
25 if (!gmem) { perror("calloc memory"); exit(1); }
26 for (size_t i = 0; i < NB_CELLS; ++i) {
27 gmem->cells[i] = (struct cell*)calloc(1, sizeof(struct cell));
28 if (!gmem->cells[i]) { perror("calloc cell"); exit(1); }
29 gmem->cells[i]->value = 0;
30 gmem->cells[i]->counter = 0;
31 }
32 gmem->clock = 0;
33 }
34
35 static void memory_destroy(void) {
36 if (!gmem) return;
37 for (size_t i = 0; i < NB_CELLS; ++i) free(gmem->cells[i]);
38 free(gmem);
39 gmem = NULL;
40 }
41
42 static void* worker(void *arg) {
43 (void)arg;
44 uintptr_t *x = &gmem->cells[0]->value; // 变量 x 是第 0 个 cell 的 value
45 for (int i = 0; i < INC_PER_THREAD; ++i) {
46 __atomic_fetch_add(x, (uintptr_t)1, __ATOMIC_SEQ_CST); // 原子 +1
47 }
48 return NULL;
49 }
50
51 int main(int argc, char **argv) {
52 if (argc != 2) { fprintf(stderr, "Usage: %s <num_threads>\n", argv[0]); return
 1; }
53 int n = atoi(argv[1]); if (n <= 0) { fprintf(stderr, "num_threads must be > 0\n"
); return 1; }
54
55 memory_init();
56
57 pthread_t *ts = (pthread_t*)malloc(sizeof(pthread_t)*(size_t)n);

```

```

58 if (!ts) { perror("malloc threads"); memory_destroy(); return 1; }
59
60 for (int i = 0; i < n; ++i)
61 if (pthread_create(&ts[i], NULL, worker, NULL)) { perror("pthread_create");
62 return 1; }
63
64 for (int i = 0; i < n; ++i) pthread_join(ts[i], NULL);
65
66 uintptr_t x = __atomic_load_n(&gmem->cells[0]->value, __ATOMIC_SEQ_CST);
67 uintptr_t expected = (uintptr_t)n * (uintptr_t)INC_PER_THREAD;
68
69 printf("%s: x=%" PRIuPTR " , expected=%" PRIuPTR "\n",
70 (x==expected) ? "OK" : "MISMATCH", x, expected);
71
72 free(ts);
73 memory_destroy();
74 return (x==expected) ? 0 : 2;
75 }

```

### Question 1.c\*

使用 `clock_gettime` (模式为 `CLOCK_MONOTONIC`) 来测量每个线程执行循环的平均耗时。  
建议在主线程中:

- 在启动所有线程前记录起始时间;
- 在所有线程结束后记录结束时间;
- 计算平均执行时间 (例如总耗时除以线程数, 或总迭代次数)。

### Question 1.d\*

为程序添加一个命令行参数, 用于指定保护变量 `x` 所采用的算法。  
在第一个版本中:

- 参数值应为 `"atomic"`;
- 若输入的算法参数不是 `"atomic"`, 程序应直接报错并退出。

示例:

```
./tm_test 4 atomic
```

### Question 1.e\*

增加第二个版本, 使用 `pthread_mutex_lock` 来保护变量 `x` 的更新操作。运行程序并比较此版本与原子操作版本的执行时间, 验证互斥锁版本明显更慢。

提示: 你可以像前一题一样, 用命令行参数区分版本:

```
./tm_test 4 mutex
```

```

1 // mem.c — 带上原子操作即是那个d问的同样道理
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <stdint.h>
5 #include <inttypes.h>
6 #include <pthread.h>
7
8 #define NB_CELLS 65536
9 #define INC_PER_THREAD (1024*1024)
10
11 struct cell {
12 uintptr_t value;
13 size_t counter;
14 };
15
16 struct memory {
17 struct cell* cells[NB_CELLS];
18 size_t clock;
19 };
20
21 static struct memory *gmem = NULL;
22
23 static void memory_init(void) {
24 gmem = (struct memory*)calloc(1, sizeof(*gmem)); // 清零并malloc
25 if (!gmem) { perror("calloc memory"); exit(1); }
26 for (size_t i = 0; i < NB_CELLS; ++i) {
27 gmem->cells[i] = (struct cell*)calloc(1, sizeof(struct cell));
28 if (!gmem->cells[i]) { perror("calloc cell"); exit(1); }
29 gmem->cells[i]->value = 0;
30 gmem->cells[i]->counter = 0;
31 }
32 gmem->clock = 0;
33 }
34
35 static void memory_destroy(void) {
36 if (!gmem) return;
37 for (size_t i = 0; i < NB_CELLS; ++i) free(gmem->cells[i]);
38 free(gmem);
39 gmem = NULL;
40 }
41
42 static void* worker(void *arg) {
43 (void)arg;
44 uintptr_t *x = &gmem->cells[0]->value; // 变量 x 是第 0 个 cell 的 value
45 for (int i = 0; i < INC_PER_THREAD; ++i) {
46 __atomic_fetch_add(x, (uintptr_t)1, __ATOMIC_SEQ_CST); // 原子 +1
47 }
48 return NULL;

```

```

49 }
50
51 int main(int argc, char **argv) {
52 if (argc != 2) { fprintf(stderr, "Usage: %s <num_threads>\n", argv[0]); return
 1; }
53 int n = atoi(argv[1]); if (n <= 0) { fprintf(stderr, "num_threads must be > 0\n"
); return 1; }
54
55 memory_init();
56
57 pthread_t *ts = (pthread_t*)malloc(sizeof(pthread_t)*(size_t)n);
58 if (!ts) { perror("malloc threads"); memory_destroy(); return 1; }
59
60 for (int i = 0; i < n; ++i)
61 if (pthread_create(&ts[i], NULL, worker, NULL)) { perror("pthread_create");
 return 1; }
62
63 for (int i = 0; i < n; ++i) pthread_join(ts[i], NULL);
64
65 uintptr_t x = __atomic_load_n(&gmem->cells[0]->value, __ATOMIC_SEQ_CST);
66 uintptr_t expected = (uintptr_t)n * (uintptr_t)INC_PER_THREAD;
67
68 printf("%s: x=%" PRIuPTR ", expected=%" PRIuPTR "\n",
69 (x==expected) ? "OK" : "MISMATCH", x, expected);
70
71 free(ts);
72 memory_destroy();
73 return (x==expected) ? 0 : 2;
74 }

```

## Question 1.f\*

现在我们将实现第三个版本——基于 **软件事务内存 (STM)** 的实现。需要提供如下 API 接口：

- void startTX(): 开始一个事务；
- uintptr\_t read(int n): 读取编号为 n 的单元格的值。若检测到冲突或中止，应返回保留值（例如 MAX\_INT）；
- void write(int n, uintptr\_t value): 向编号为 n 的单元格写入值；
- bool commitTX(): 尝试提交事务；若提交失败则返回 false 并中止。

暂时可先实现函数空壳以保证能编译。测试时可使用以下示例循环代码：

Listing 2: 事务性循环示例

```

1 for (size_t i = 0; i < 1024 * 1024; i++) {
2 restart:
3 startTX(); // 开始事务
4

```

```

5 int value = read(0); // 读取 x
6 if (value == MAX_INT) // 若返回保留值，则重新开始事务
7 goto restart;
8
9 write(0, value + 1); // 写入 x + 1
10
11 if (!commitTX()) // 提交事务
12 goto restart; // 若提交失败则重启
13 }

```

## Question 1.g\*

完善上述事务接口函数，实现课堂中讲过的算法。

为此需要：

- 定义一个线程局部（\_Thread\_local）的事务状态结构，用于保存每个线程当前事务的读集、写集和版本信息；
- 完成 startTX、read、write 和 commitTX 的具体实现；
- 此阶段暂不考虑释放旧 cell 的问题（即使新建了新的版本，也先不回收旧的内存）。

回忆提交阶段的关键操作：

1. 为要更新的变量分配一个新 cell，其中包含新的值和新的计数器；
2. 使用原子 CAS 操作将全局指针更新为新 cell，从而完成提交。

```

1 //
2 // stm_mem_stm_only.c
3 // 只保留 STM (Software Transactional Memory) 版本
4 // STM 实现采用延迟写 (redo log) + lazy 验证 + 提交期全局锁
5 //
6 #include <stdio.h> // 用于输出结果
7 #include <stdlib.h> // malloc/calloc/exit 等
8 #include <stdint.h> // uintptr_t 类型
9 #include <inttypes.h> // PRIuPTR 打印格式
10 #include <pthread.h> // 线程与互斥锁支持
11 #include <string.h> // memset
12
13 #define NB_CELLS 65536 // 主存中 cell 数量
14 #define INC_PER_THREAD (1024*1024) // 每个线程的加次数
15 #define MAX_INT_PTR ((uintptr_t)(~(uintptr_t)0)) // read() 冲突时返回的特殊失败值
16
17 // ===== 内存模型定义 =====
18
19 // 每个 cell: 保存一个值与其版本号 (counter)
20 struct cell {
21 uintptr_t value; // 实际的存储值
22 size_t counter; // cell 的“版本”，每次写入时更新
23 };

```

```

24
25 // 整个内存系统：包含若干 cell 指针和一个全局时钟
26 struct memory {
27 struct cell* cells[NB_CELLS]; // 每个元素指向一个 cell
28 size_t clock; // 全局时钟（事务时间戳）
29 };
30
31 // 全局内存指针
32 static struct memory *gmem = NULL;
33
34 // 全局互斥锁：STM 在提交阶段使用，保证提交过程的原子性
35 static pthread_mutex_t g_mutex = PTHREAD_MUTEX_INITIALIZER;
36
37 // ===== 初始化函数 =====
38
39 // 初始化整个内存模型
40 static void memory_init(void) {
41 gmem = (struct memory*)calloc(1, sizeof *gmem); // 分配并清零 memory 结构
42 if (!gmem) { perror("calloc memory"); exit(1); }
43
44 // 为每个槽位分配独立的 cell，初始值与计数均为 0
45 for (size_t i = 0; i < NB_CELLS; ++i) {
46 gmem->cells[i] = (struct cell*)calloc(1, sizeof(struct cell));
47 if (!gmem->cells[i]) { perror("calloc cell"); exit(1); }
48 }
49
50 gmem->clock = 0; // 全局时钟初始化为 0
51 }
52
53 // ===== STM 数据结构定义 =====
54
55 // 读集与写集的容量上限（固定表实现）
56 #define RS_CAP 1024
57 #define WS_CAP 1024
58
59 // 每个线程自己的事务状态结构
60 typedef struct {
61 size_t local_clock; // 启动事务时记录的全局时钟值
62 int aborted; // 标志：事务是否已被判定为中止
63 int rs[RS_CAP]; // 读集索引数组
64 size_t rs_len; // 当前读集长度
65 int ws_idx[WS_CAP]; // 写集索引数组
66 uintptr_t ws_val[WS_CAP]; // 写集对应写入的值
67 size_t ws_len; // 当前写集长度
68 } tx_state_t;
69
70 // 每线程独立的事务状态（线程局部存储）
71 _Thread_local static tx_state_t tx;
72

```

```

73 // ===== STM 基本操作 =====
74
75 // 启动事务：记录当前全局时钟，清空读/写集，重置中止标志
76 static void startTX(void) {
77 tx.local_clock = __atomic_load_n(&gmem->clock, __ATOMIC_SEQ_CST); // 快照全局时
 钟
78 tx.aborted = 0;
79 tx.rs_len = 0;
80 tx.ws_len = 0;
81 }
82
83 // 在写集中查找某个索引，返回下标或 -1
84 static int ws_find(int n) {
85 for (size_t i = 0; i < tx.ws_len; ++i)
86 if (tx.ws_idx[i] == n)
87 return (int)i;
88 return -1;
89 }
90
91 // 向读集添加一个索引（若已存在则不重复添加）
92 static void rs_add_once(int n) {
93 for (size_t i = 0; i < tx.rs_len; ++i)
94 if (tx.rs[i] == n)
95 return;
96 if (tx.rs_len < RS_CAP)
97 tx.rs[tx.rs_len++] = n;
98 }
99
100 // STM 读操作
101 static uintptr_t read(int n) {
102 // 1) 若写集中已有该 index（即本事务已写），返回写集中的值
103 int k = ws_find(n);
104 if (k >= 0)
105 return tx.ws_val[k];
106
107 // 2) 否则从主存读取 cell 指针快照
108 struct cell *c = __atomic_load_n(&gmem->cells[n], __ATOMIC_SEQ_CST);
109
110 // 3) lazy 验证：若该 cell 的版本号 >= 本地事务启动时的全局时钟，表示数据过新
 （冲突）
111 if (c->counter >= tx.local_clock) {
112 tx.aborted = 1; // 标记事务需要中止
113 return MAX_INT_PTR; // 返回保留失败值
114 }
115
116 // 4) 将 index 加入读集，供提交期再次验证
117 rs_add_once(n);
118
119 // 5) 返回读到的值

```

```

120 return c->value;
121 }
122
123 // STM 写操作（延迟写：只修改本地写集）
124 static void write(int n, uintptr_t value) {
125 int k = ws_find(n); // 查找写集中是否已有该 index
126 if (k >= 0) {
127 tx.ws_val[k] = value; // 若存在则覆盖旧值
128 return;
129 }
130
131 if (tx.ws_len < WS_CAP) { // 若写集未满足则追加
132 tx.ws_idx[tx.ws_len] = n; // 记录 index
133 tx.ws_val[tx.ws_len] = value; // 记录写入值
134 tx.ws_len++;
135 } else {
136 tx.aborted = 1; // 写集溢出：标记中止（简化处理）
137 }
138 }
139
140 // STM 提交事务
141 static int commitTX(void) {
142 if (tx.aborted) // 若已标记为中止，直接失败
143 return 0;
144
145 pthread_mutex_lock(&g_mutex); // 提交阶段全局加锁（简化串行化）
146
147 // 1) 验证读集：确保事务期间读过的 cell 没被修改
148 for (size_t i = 0; i < tx.rs_len; ++i) {
149 int idx = tx.rs[i];
150 struct cell *c_now = __atomic_load_n(&gmem->cells[idx], __ATOMIC_SEQ_CST);
151 if (c_now->counter >= tx.local_clock) {
152 // 有冲突：其他事务在我们期间修改了它
153 pthread_mutex_unlock(&g_mutex);
154 return 0;
155 }
156 }
157
158 // 2) 应用写集：将写入内容真正写回主存
159 size_t cur_clock = gmem->clock; // 当前全局时钟值
160 for (size_t i = 0; i < tx.ws_len; ++i) {
161 int idx = tx.ws_idx[i];
162 uintptr_t v = tx.ws_val[i];
163
164 // 为每个写入分配新的 cell 结构
165 struct cell *nc = (struct cell*)malloc(sizeof *nc);
166 if (!nc) {
167 pthread_mutex_unlock(&g_mutex);
168 return 0;

```

```

169 }
170 nc->value = v; // 写入值
171 nc->counter = cur_clock; // 使用当前时钟作为版本号
172
173 // 将全局指针更新为新的 cell
174 __atomic_store_n(&gmem->cells[idx], nc, __ATOMIC_SEQ_CST);
175 // 按题目要求：旧 cell 暂不回收
176 }
177
178 // 3) 提交成功后递增全局时钟，使后续事务的 local_clock 更大
179 gmem->clock = cur_clock + 1;
180
181 pthread_mutex_unlock(&g_mutex); // 解锁
182 return 1; // 提交成功
183 }
184
185 // ===== 工作线程函数 =====
186
187 // 每个线程对 cells[0] -> value 累加 INC_PER_THREAD 次
188 static void* worker(void *arg) {
189 (void)arg; // 未使用的参数
190
191 for (size_t i = 0; i < INC_PER_THREAD; ++i) {
192 restart: // 提交失败时重试
193 startTX(); // 启动事务
194 uintptr_t v = read(0); // 读出 cells[0] 的值
195 if (v == MAX_INT_PTR) goto restart; // 若读冲突，重启事务
196 write(0, v + 1); // 写入新值到写集
197 if (!commitTX()) goto restart; // 若提交失败则重试
198 }
199
200 return NULL; // 线程结束
201 }
202
203 // ===== 主函数 =====
204
205 int main(int argc, char **argv) {
206 if (argc != 2) { // 参数：线程数量
207 fprintf(stderr, "Usage: %s <num_threads>\n", argv[0]);
208 return 1;
209 }
210
211 int n = atoi(argv[1]); // 解析线程数
212 if (n <= 0) {
213 fprintf(stderr, "Error: num_threads must be > 0\n");
214 return 1;
215 }
216
217 memory_init(); // 初始化全局内存模型

```

```

218
219 pthread_t *ts = malloc(sizeof(pthread_t) * (size_t)n); // 分配线程数组
220 if (!ts) { perror("malloc threads"); return 1; }
221
222 // 创建所有线程
223 for (int i = 0; i < n; ++i)
224 if (pthread_create(&ts[i], NULL, worker, NULL) != 0) {
225 perror("pthread_create");
226 return 1;
227 }
228
229 // 等待所有线程完成
230 for (int i = 0; i < n; ++i)
231 pthread_join(ts[i], NULL);
232
233 // 检查结果是否正确
234 uintptr_t x = gmem->cells[0]->value;
235 uintptr_t expected = (uintptr_t)n * (uintptr_t)INC_PER_THREAD;
236
237 printf("[stm] %s: x=%" PRIuPTR " , expected=%" PRIuPTR " \n",
238 (x==expected)?"OK":"MISMATCH", x, expected);
239
240 free(ts); // 释放线程数组
241 return (x==expected)?0:2; // 返回结果状态
242 }

```

### Question 1.h\* (可选)

尝试在提交阶段回收旧的 cell，以实现内存复用。

但要注意：当某线程仍在访问某个 cell（例如执行读操作时），该 cell 不可立即释放。因此可以借助前一次实验中的 **Hazard Pointer（危险指针）** 技术：

- 每个线程维护一个线程局部的“危险指针集”，其中存放当前正在使用的 cell；
- 当准备释放某个 cell 时，只有当它未出现在任意线程的危险指针集中，才能安全地释放。

## Exercise 2: 课程问题 (2 分, 简单)

### Question 2.a (1 分)

**中文题目:** 为什么奔腾处理器的 `sfence` (存储栅栏) 指令足以实现抽象指令 `pfence`?

**中文答案:** 因为 `sfence` 指令确保所有在其之前的存储 (store) 操作都在其之后的存储操作之前完成并对其他处理器可见, 这正是抽象的 `pfence` 所要求的持久性顺序保证。

**English Question:** Why is the Pentium processor's `sfence` (store fence) instruction sufficient to implement the abstract `pfence` instruction?

**English Answer:** Because the `sfence` instruction guarantees that all previous store operations are completed and visible before any subsequent stores, providing the same persistence ordering semantics required by `pfence`.

### Question 2.b (1 分)

**中文题目:** 为什么奔腾处理器的 `sfence` (存储栅栏) 指令足以实现抽象指令 `psync`?

**中文答案:** 因为 `sfence` 确保在其之前的所有缓存写入都被刷新到持久性存储中, 从而保证数据在执行点之前已持久化, 满足 `psync` 的语义要求。

**English Question:** Why is the Pentium processor's `sfence` (store fence) instruction sufficient to implement the abstract `psync` instruction?

**English Answer:** Because `sfence` ensures that all previous cache writes are flushed to persistent storage before continuing, guaranteeing durability up to that point, which satisfies the semantics of `psync`.

## Exercise 1: 课程问题 (6 分)

### Question 1.a (2 分) NUMA

**中文题目:** 假设一个 NUMA 架构系统由四个节点 A、B、C、D 组成。每个节点包含 10 个核心, 并连接到一个最大带宽为 5 GB/s 的内存。节点之间有 4 条互连链路: A-B、B-C、C-D、D-A, 每条链路的最大带宽为 1 GB/s。假设一个包含 40 个线程的应用程序, 每个线程以恒定速率 0.1 GB/s 产生内存访问。应用在初始化阶段由初始线程分配并初始化其使用的内存。在这种情况下, 哪种内存分配策略能获得最佳性能: first-touch 还是 interleaved? 请说明理由, 并给出各链路的吞吐率。

**中文答案:** 若内存访问在虚拟地址空间上均匀分布, 40 个线程产生总负载 4 GB/s。- 若采用 interleaved 策略, 内存负载在 4 条链路上平均分配, 每条链路承载 1 GB/s, 均未达到瓶颈, 系统带宽为 4 GB/s。- 若采用 first-touch 策略, 全部线程初始访问由节点 A 完成。节点 A 最大可用内存带宽为 4 GB/s, 不饱和, 但其连接链路 AB、AC、AD 总共仅 3 GB/s, 因此系统总吞吐率仅为 3 GB/s。因此, interleaved 策略性能更佳。

**English Question:** We assume a NUMA architecture consisting of four nodes A, B, C, and D, each with 10 cores and a memory bandwidth of 5 GB/s. There are 4 interconnect links (A-B, B-C, C-D, D-A), each with a bandwidth of 1 GB/s. An application has 40 threads, each generating 0.1 GB/s of memory traffic. The application allocates and initializes its memory in the initial thread. Which allocation policy —first-touch or interleaved—provides the best performance? Justify and describe the throughput on each link.

**English Answer:** With interleaved allocation, the total 4 GB/s traffic is evenly distributed, giving 1 GB/s per link, which does not saturate any interconnect or memory controller, yielding 4 GB/s total. With first-touch, all threads' data are placed on node A, whose total outbound bandwidth is 3 GB/s (limited by links AB, AC, AD). Thus, interleaved policy performs better.

### Question 1.b (2 分) 内存模型

**中文题目：**考虑四个变量 `ok1`、`ok2`、`msg1` 和 `msg2`，初始值均为 0。有以下三个线程执行原子读写操作：

```
Thread 1: a. msg1 = 42; b. ok1 = 1;
Thread 2: c. while (ok1 == 0) {} d. msg2 = msg1; e. ok2 = 1;
Thread 3: f. while (ok2 == 0) {} g. printf("%d %d\n", msg1, msg2);
```

假设系统采用弱内存模型，请问所有可能的输出是什么？若系统不重新排序写操作或读操作，又有哪些可能输出？

**中文答案：**若系统不重排读或写操作，则必有  $a < b < c$  和  $d < e < f < g$ 。可能的输出有：

- 0 0 (当  $a < b < c$ )
- 42 0 (当  $d < a < e < f < g$ )
- 42 42 (当  $a < d < e < f < g$ )

在弱内存模型下仍可产生相同结果。注意：不可能出现 “0 42”，因为若 `msg2 = 42`，必然有 `msg1 = 42`。

**English Question:** We consider four variables `ok1`, `ok2`, `msg1`, and `msg2`, all initialized to 0. The following atomic code runs in three threads:

```
Thread 1: a. msg1 = 42; b. ok1 = 1;
Thread 2: c. while(ok1 == 0) {} d. msg2 = msg1; e. ok2 = 1;
Thread 3: f. while(ok2 == 0) {} g. printf("%d %d\n", msg1, msg2);
```

Under a relaxed memory model, what outputs are possible? What if the system does not reorder writes or reads?

**English Answer:** If the machine does not reorder reads or writes, we have the possible outputs:

- 0 0 when  $a < b < c$ ,
- 42 0 when  $d < a < e < f < g$ ,
- 42 42 when  $a < d < e < f < g$ .

Under a relaxed model, the same outputs can occur. The output “0 42” is impossible since if `msg2 = 42`, then necessarily `msg1 = 42`.

### Question 1.c (2 分) 持久内存

**中文题目：**假设计算机支持最新的 eADR 技术，该技术保证在系统故障时仍有足够能量将缓存中的数据刷新到持久内存。在这种机器上，是否仍需执行 `pwb` 和 `pfence` 指令？请说明理由。

**中文答案：**由于 eADR 能保证在掉电情况下缓存行数据最终写入持久内存，因此无需 `pwb`。但仍需 `pfence`，因为 `pfence` 主要用于指令顺序控制，与掉电传播机制无关。

**English Question:** Assume the computer uses eADR technology, which ensures enough power to flush dirty cache lines to persistent memory on failure. On such a machine, are the `pwb` and `pfence` instructions still necessary? Justify.

**English Answer:** `pwb` is no longer required since eADR guarantees all dirty cache lines will eventually reach persistent memory. However, `pfence` remains necessary because it enforces ordering of persistence operations, not related to power-failure propagation.

## Exercise 3: 课程问题 (3 分)

### Question 3.a (1 分)

**中文题目:** 在释放锁时, 为什么需要使用具有 `memory_order_release` 语义的原子操作?

**中文答案:** 因为必须保证在释放锁之前的所有写操作都已执行完成, 否则其他线程在获取锁后可能读到未更新的数据。

**English Question:** When releasing a lock, why must we use an atomic operation with `memory_order_release` semantics?

**English Answer:** Because it ensures that all previous writes are completed before releasing the lock, preventing other threads from seeing stale data once they acquire the lock.

### Question 3.b (1 分)

**中文题目:** 在 NUMA 架构中, 在哪种情况下 interleaved (交错) 内存分配比 first-touch (首次访问) 分配更合适?

**中文答案:** 当一个线程为其他线程分配内存并且首先访问这些内存时, interleaved 分配更优, 因为 first-touch 会将内存放在分配线程所在的节点上, 从而造成内存访问瓶颈。

**English Question:** On a NUMA architecture, in which case is interleaved placement preferable to first-touch placement?

**English Answer:** When one thread allocates memory for others and accesses it first, interleaved placement is better, as first-touch would place memory on the allocator's NUMA node, overloading it.

### Question 3.c (1 分)

**中文题目:** 简要说明 `psync` 和 `pfence` 的区别。

**中文答案:** `pfence` 保证 `pwb` 与内存读写不会乱序执行; 而 `psync` 除此之外还确保缓存中的写回数据确实已被刷新并持久化。

**English Question:** Briefly explain the difference between `psync` and `pfence`.

**English Answer:** `pfence` enforces ordering between `pwb` and memory operations, while `psync` additionally ensures that cache lines from the flush queue have actually been propagated to persistent memory.

## Exercise 1: 内存模型 (6 分)

### Question 1.a (1 分)

**中文题目:** 假设最初 `x` 和 `y` 都为 0。为什么下面的代码在 TSO 内存模型下一定会输出 42?

Thread 1:  
1.a `x = 42;`

```
1.b y = 1;
```

Thread 2:

```
2.a while (y == 0) { }
```

```
2.b printf("%d\n", x);
```

**中文答案：**在 TSO 模型中：- 写操作按程序顺序对其他线程可见（写后写顺序保证）。- 1.a 的写入在 1.b 之前完成传播，因此 x=42 对其他线程可见后才有 y=1。- 线程 2 在看到 y=1 后再读取 x，因此必定得到 42。

**English Question:** Assuming initially  $x = 0$  and  $y = 0$ , why does the following code necessarily print 42 under a TSO memory model?

**English Answer:** Under TSO, writes are observed in program order. Write to  $x$  (1.a) becomes visible before write to  $y$  (1.b). Thread 2 reads  $x$  only after it sees  $y == 1$ , so it must read the value 42.

### Question 1.b (1 分)

**中文题目：**为什么在更宽松的内存模型下，上述程序可能打印 0？

**中文答案：**在宽松模型中允许读操作重排。线程 2 中的读取  $x$ （在 2.b）可能在 2.a 之前执行，于是在  $y==0$  时提前读取了旧值  $x=0$ ，从而可能输出 0。

**English Question:** Why, under a more relaxed memory model, could the previous program print 0?

**English Answer:** Relaxed memory models may reorder reads. In Thread 2, the read of  $x$  in 2.b can be executed before the read of  $y$  in 2.a, so it may observe  $x=0$  before  $y$  becomes 1, thus printing 0.

### Question 1.c (2 分)

**中文题目：**假设初始时  $x = y = t1 = t2 = 0$ ，且采用顺序一致（sequential consistency）模型，即读写在各线程间按执行顺序传播。以下三线程执行完毕后， $x$ ， $y$ ， $t1$ ， $t2$  各自可能取什么值？

Thread 1:

```
1.a x = 2;
```

```
1.b t1 = y;
```

Thread 2:

```
2.a y = 1;
```

```
2.b x = 1;
```

Thread 3:

```
3.a t2 = x;
```

**中文答案：**在顺序一致模型下，所有线程的操作按执行顺序全局串行化。最终  $y=1$  必然成立。对  $x$  有两种情况：- 若  $x=2$ ，说明 1.a 在 2.b 之后可见；- 若  $x=1$ ，说明 2.b 在 1.a 之后。因此：

- 当  $x=2$  时， $t1=1$ ， $t2=0$  或 2；
- 当  $x=1$  时， $t1=0$  或 1， $t2=0$  或 1。

所以所有可能组合为:  $x \in \{1, 2\}$ ,  $y=1$ ,  $t1 \in \{0, 1\}$ ,  $t2 \in \{0, 1, 2\}$ 。

**English Question:** Assuming initially  $x=y=t1=t2=0$  and a sequentially consistent memory model (all reads and writes are observed in program order across threads), what possible values can the four variables take after all threads finish?

**English Answer:** Under sequential consistency, all operations are globally ordered.  $y=1$  always holds. For  $x$ , either: -  $x=2$  if Thread 1's write occurs last, -  $x=1$  if Thread 2's write occurs last. Thus:

- If  $x=2$ :  $t1=1$ ,  $t2 \in \{0, 2\}$ ;
- If  $x=1$ :  $t1 \in \{0, 1\}$ ,  $t2 \in \{0, 1\}$ .

Possible outcomes:  $x \in \{1, 2\}$ ,  $y=1$ ,  $t1 \in \{0, 1\}$ ,  $t2 \in \{0, 1, 2\}$ .

### Question 1.d (2 分)

**中文题目:** 若现在采用 TSO 模型, 上述四个变量可能取哪些值?

**中文答案:** 在 TSO 下, 读操作可在前面的写操作之前执行 (写后读重排允许)。因此, 1.b 可能在 1.a 之前执行, 从而  $t1$  可读取旧值  $y=0$ 。因此比顺序一致模型多一种情况:  $x=2$ ,  $y=1$ ,  $t1=0$ ,  $t2 \in \{0, 1, 2\}$ 。

**English Question:** Assuming now a TSO memory model, what values can the four variables take?

**English Answer:** Under TSO, a read can be executed before a previous write (write-read reordering). So  $t1$  may read  $y=0$  before  $x=2$  is written. Compared to sequential consistency, we add the extra case:  $x=2$ ,  $y=1$ ,  $t1=0$ ,  $t2 \in \{0, 1, 2\}$ .

## Exercise 1: 课程问题 (5 分)

在本题中, 如果被要求“说明理由”, 请用简洁回答 (2-3 行即可)。

### Question 1.a (1 分, 简单)

**中文题目:** 在一台 NUMA 机器上, 哪种情况更会减慢应用程序: (1) 互连网络因为所有核随机访问所有内存 bank 而饱和; (2) 某个内存控制器因为所有核都访问同一个内存 bank 而饱和? 请说明理由。

**中文答案:** 瓶颈更严重的是单个内存控制器饱和。若所有核随机访问所有内存 bank, 负载会在各个 NUMA 节点和链路之间大致均匀分布; 而若所有访问都集中在同一内存 bank 上, 则所有流量都压在同一个内存控制器上, 该控制器成为系统的唯一瓶颈。

**English Question:** On a NUMA machine, which situation slows down an application more: (1) the interconnect network becomes saturated because all cores randomly access all memory banks, or (2) a single memory controller becomes saturated because all cores access the same memory bank? Justify.

**English Answer:** The worst case is when a **single memory controller** is saturated. With random accesses to all banks, the traffic is roughly balanced across links and controllers. When all cores access the same bank, all traffic goes through one controller, which becomes the bottleneck and limits performance.

### Question 1.b (1 分, 简单)

**中文题目:** 假设使用 TSO 内存模型。线程 1 执行函数  $t1$ , 线程 2 执行函数  $t2$ 。为什么线程 2 可能永远不会执行输出? 需要对代码做什么修改来修正问题?

```

int s = 0;
char* msg = NULL;

void t1() {
 msg = "coucou";
 s = 1;
}

void t2() {
 while (s == 0) {
 }
 printf("Coucou !\n");
}

```

**中文答案：**编译器会发现 `t2` 中变量 `s` 在该函数里从未被修改，于是可以把循环优化成：

```

if (s == 0)
 while (true) { }

```

这样一来，一旦进入循环就再也不会重新读取 `s`，所以永远到不了 `printf`。要修正问题，只需把 `s` 声明为 `volatile`（或改成原子变量），强制每次循环都从内存重新读取它。

**English Question:** Assume a TSO memory model. Thread 1 executes `t1` and thread 2 executes `t2`. Why may thread 2 never perform its print? What modification must be made to the code to fix the problem?

**English Answer:** The compiler can optimize `t2` by noticing that `s` is never modified in this function, and therefore replace the loop by:

```

if (s == 0)
 while (true) { }

```

Thread 2 then never re-reads `s` and may spin forever, never reaching the `printf`. To fix this, declare `s` as `volatile` (or make it atomic) so that each iteration reloads `s` from memory.

### Question 1.c (1 分, 简单)

**中文题目：**为什么对上题代码进行修正（例如把 `s` 标记为 `volatile`）后，在 TSO 内存模型下程序一定会打印 “Coucou”，而在更宽松的内存模型下却可能因为段错误结束？请说明理由。

**中文答案：**在宽松内存模型下，`t1` 中对 `s` 的写入可能比对 `msg` 的写入更早对其他核可见；或者在 `t2` 中读取 `msg` 可能被重排到读取 `s` 之前。这样 `t2` 可能看到 `s != 0` 但 `msg == NULL`，导致访问空指针而段错误。在 TSO 模型下，不允许读后读和写后写的重排序，因此一旦 `t2` 看到 `s != 0`，就必然也能看到对 `msg` 的写入，所以一定会安全地打印 “Coucou”。

**English Question:** Why does the corrected version of the previous code (e.g., with `s` declared `volatile`) always print “Coucou” under a TSO memory model, whereas under a more relaxed memory model it may end with a segmentation fault? Justify.

**English Answer:** On a relaxed memory model, the write to `s` in `t1` may become visible to other cores before the write to `msg`, or in `t2` the read of `msg` may be reordered before the read of `s`. Then `t2` can observe `s != 0` while `msg == NULL`, and dereferencing `msg` causes a segmentation fault. With TSO, reads cannot be reordered with other reads and writes cannot be reordered with other writes, so once `t2` sees `s != 0`, it is guaranteed to see the prior write to `msg`, and the program safely prints “Coucou”.

### Question 1.d (1 分, 简单)

**中文题目:** 在一个虚拟机监控器 (hypervisor) 中, 为什么可以“本地执行”(原生运行) 一个进程, 但却不能以同样方式原生运行来宾操作系统? 请说明理由。

**中文答案:** 普通进程不执行特权指令, 只通过系统调用接口与操作系统交互, 因此 hypervisor 可以直接在硬件上运行该进程, 并把它的系统调用重定向到来宾系统的内核代码。而来宾操作系统本身需要执行大量特权指令, 这些指令可能与 hypervisor 的控制操作冲突, 所以必须被截获和虚拟化, 不能简单地原生执行。

**English Question:** In a hypervisor, why can we run a process natively, whereas we cannot run the guest operating system itself natively in the same way? Justify.

**English Answer:** A user process does not execute privileged instructions; it interacts with the system through system calls. The hypervisor can therefore run the process directly on the hardware and redirect its system calls to the guest OS kernel. The guest OS, however, executes privileged instructions that conflict with the hypervisor's own control of the hardware, so these instructions must be trapped and virtualized instead of being run natively.

### Question 1.e (1 分, 简单)

**中文题目:** 解释为什么在硬件事务内存 (TM) 中, 即使只有一个线程在执行, 事务也可能被中止 (abort)?

**中文答案:** 硬件事务通常把事务读写的数据缓存在缓存中, 如果事务访问的数据太多导致事务工作集超出缓存容量, 硬件会因为缓存溢出而强制中止事务。因此, 只要执行一个访问大量内存的事务, 即使该线程完全独占机器, 也可能反复触发事务回滚。

**English Question:** Explain why, with hardware transactional memory, a transaction may abort even when the thread executes alone on the machine.

**English Answer:** Hardware transactions keep their read and write sets in the cache. If a transaction touches too much memory so that its working set exceeds the cache capacity, the hardware aborts the transaction due to cache overflow. Thus, a single thread running a transaction that accesses a large amount of memory can repeatedly cause the transaction to abort, even without any interference from other threads.

## Exercise 2: 课程问题 (6 分)

### Question 2.a (2 分)

**中文题目:** 在 TSO 内存模型中, 下面的代码只有一个线程 `sender` 和一个线程 `receiver`。为什么该程序是正确的?

```
char* msg = 0;
int go = 0;

void sender() {
 msg = "coucou";
 go = 1;
}

void receiver() {
```

```

while (!go) {
}
printf("%s\n", msg);
}

```

**中文答案：**在 TSO 中，同一线程的写操作按程序顺序对其他线程可见（写后写不能重排）。因此只要 receiver 能看到 go==1，就保证已经先看到对 msg 的写入；于是打印时一定使用的是初始化后的 msg，程序正确。

**English Question:** In a TSO memory model, why is the following program, with a single sender thread and a single receiver thread, correct?

**English Answer:** Under TSO, a thread's writes become visible to other threads in program order (no write-write reordering). Once the receiver observes go == 1, the write to msg is already visible, so printf always uses the initialized msg, making the program correct.

## Question 2.b (4 分)

**中文题目：**考虑一个 NUMA 架构：每个核都有统一的 L1 缓存，同一 NUMA 节点上的若干核共享统一的 L2 缓存。从某变量 x 的虚拟地址 @v 出发，逐步描述硬件是如何完成一次对该变量的写访问的，要说明各级缓存中发生了什么。假设机器有 4 个 NUMA 节点；访问由“节点 0 的核心 5”发起；变量 x 归属节点 1，并且当前处于“节点 2 的核心 3 的 L1 缓存”中，状态为 Modified。

**中文答案：**一个可能的执行步骤如下：

1. 处理器通过页表把虚拟地址 @v 翻译成物理地址 @p。
2. 核心 5 向本地 L1 请求对地址 @p 的独占（写）访问。L1 未命中，将请求转发给本节点的 L2。
3. L2 也未命中，于是向变量所在的 home 节点（节点 1）发送对 @p 的独占访问请求。
4. 节点 1 的目录记录更新，登记该行将由“节点 0/核心 5”以独占方式持有，并把请求转发给当前持有者“节点 2/核心 3”。
5. 节点 2 的核心 3 在其 L1 中找到该行，状态为 Modified，将其失效并把包含数据的 cache line 返回给“节点 0/核心 5”。
6. 节点 0 的 L2 接收该行并以独占状态存入自己的 L2 缓存。
7. 核心 5 的 L1 从 L2 取入该行，以独占状态存入 L1。
8. 处理器在 L1 中对该 cache line 进行写入访问，完成对 x 的写操作。

**English Question:** We consider a NUMA architecture: each core has a unified L1 cache and all cores of a NUMA node share a unified L2 cache. Starting from the virtual address @v of a variable x, describe step by step how the hardware performs a write access to the variable, explaining what happens in the different caches. Assume there are four NUMA nodes; the access is issued by core 5 of node 0; x is homed on node 1 and currently resides in the L1 cache of core 3 of node 2 in the Modified state.

**English Answer:** One possible sequence is:

1. The CPU translates virtual address @v to physical address @p using the page tables.
2. Core 5 requests an exclusive (write) access to @p from its local L1. On a miss, L1 forwards the request to the node-0 L2.

3. L2 also misses and sends an exclusive request for  $@p$  to the home node of the line, node 1.
4. Node 1's directory records that node 0/core 5 will own the line exclusively and forwards the request to the current owner, node 2/core 3.
5. Core 3 of node 2 finds the line in its L1 in state **Modified**, invalidates it, and sends the data (the cache line) back to node 0/core 5.
6. Node 0's L2 inserts the line in its cache in exclusive state.
7. Core 5's L1 loads the line from L2 and stores it in exclusive state.
8. The processor finally performs the write to  $x$  in its L1 cache line.